IMPERIAL

Imperial College London
Department of Computing

Protecting User Secrets in Hostile Environments

Sirvan Almasi

Supervised by Prof William J. Knottenbelt

Submitted in partial fulfilment of the requirements for the award of *Doctor of Philosophy* from Imperial College London, February 2025

Statement of Originality

I declare that this thesis was written by myself, and the presented work is my own, except stated otherwise.

Copyright Declaration

Abstract

User secrets, such as passwords, personally identifiable information, and financial data, are increasingly vulnerable to browser-based malware, data breaches, and password guessing attacks. Motivated by an increase in cyber crime, this dissertation explores the life cycle of these user secrets and introduces novel methods to mitigate and circumvent security threats. We examine the journey of user secrets through three stages: password composition, client-side interaction, and server-side storage.

We make three contributions with the ultimate aim of protecting user secrets on the web. Firstly, during the password composition phase, we investigate the entropy and resilience of user-generated passwords. Despite their vulnerability to exploits and inherent weaknesses that lead to many data breaches, password-based authentication systems remain prevalent. Motivated by the lack of consensus on password composition policy and strength, we ask if there exists a secure, human-chosen, and memorable password. To address this, we introduce the first-ever human-labelled password dataset and utilise Large Language Models to expedite the labelling process.

Secondly, in the client-side stage, we examine the threats associated with the transmission and handling of user secrets within the browser environment. We design and implement an out-of-band method, FormL3SS, to circumvent Man-in-the-Browser malware.

Finally, we focus on the server-side security phase—the final stage in the journey of user secrets. We explore whether identity-based cryptosystems can replace traditional hash functions for password-based authentication. Using these cryptosystems, we design and implement a password-less protocol—deeID.

It is clear that modern web authentication, dominated by password, and its trajectory of progress is unsustainable. The current stopgap approach of bolstering password weaknesses through additional layers, such as MFA, has resulted in user frustration and, paradoxically, in many cases introduced new vulnerabilities. We hope that our work will inspire a paradigm shift and encourage a bold leap towards new passwordless and usable authentication systems.

Acknowledgements

The PhD journey has been both fulfilling and the most demanding challenge I have faced so far. I am proud of this achievement, and as I reach the end, I feel even more motivated to continue learning. Of course, this journey has not been one I have undertaken alone, and I am deeply grateful to the following individuals.

First and foremost, I would like to thank my supervisor, Prof William Knottenbelt, for his support and trust in me. I vividly recall our first meeting in 2017—the very first step in this journey. Prof Knottenbelt's leap of faith in me has been truly life-changing, and for that, I am profoundly grateful.

I joined the Financial Computing CDT under the leadership of Prof Philip Treleaven, whom I am thankful for his guidance, leadership, and mentorship over the years. Prof Treleaven is truly an inspiring leader, and I am thankful for his encouragement to reach the finish line. I acknowledge the grants from the Financial Computing CDT and the Engineering and Physical Sciences Research Council (EPSRC) that made this PhD happen.

I acknowledge the wider Imperial community, especially the Computing Department and members of the ACEX 358 lab: Lewis, Sam, Alexei, Katerina, Rami, Dragos, Dominik, Manon, Toshiko, and Luca. No journey is done alone and everyone we meet has an influence on us.

The love and support of my parents and sisters kept me going during the most challenging moments of this journey. I will forever be grateful for their ongoing encouragement and unconditional love.

To Justina, whose love, support, and encouragement were instrumental in keeping my spirits high. Her ability to listen and provide a comforting presence during stressful times enriched this experience in ways that words cannot capture.

A special thank you goes to Killian. His steadfast friendship and our shared PhD journey deepened my academic experience. Lastly, I acknowledge my friends who, with their constant reminders and light-hearted banter, kept me accountable and motivated.

You are made of stardust and dreams,
woven with the light of endless possibilities.

For Nora

May your journey be filled with wonder, your heart with courage, and your spirit with the magic of infinite adventures. May you always find light in the darkest nights and hear the whispers of dreams calling you forward.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 Motivation

The *cyberspace* has fuelled the growth of economies and civilisations, but like every innovation, there is a darker side to it; its more ominous aspects are vividly embodied by the idea and existence of the "dark net." The "cyber" prefix in words such as cyberspace, cybersecurity, cybercrime, or cyberwarfare has become an element of our everyday lingo. However, it was not too long ago that this reality was considered science fiction. William Gibson coined the term "cyberspace" in his 1984 novel *Neuromancer* [87], describing a virtual and networked grid of data. Our cyberspace is the internet and the World Wide Web, or simply the web; these technologies have become a ubiquitous part of daily life and a key component of national infrastructure. Today, many people in developed economies rely on mobile banking, e-commerce, and digital communication. The internet and the web have become indispensable tools for productivity, communication, and economic growth. However, this growth has also opened new avenues for cybercriminals to exploit vulnerabilities and profit from sensitive user data. Safeguarding user secrets in an increasingly complex and hostile digital environment has become more critical, especially as we conduct more of our sensitive transactions online.

We are primarily motivated by countering cybercrime and improving systems that enrich our lives to be more trustworthy and usable. In this dissertation, we focus on distinct steps in the journey of user secrets through cyberspace and devise strategies to circumvent threats. Taking the "journey of secrets" perspective differs from similar dissertations [31], [193], [249] (primarily password security), and we believe this approach has the added benefit of considering other factors, such as usability and other weaknesses.

Cyber and _Cyberspace_ Etymology

It is widely believed [19], [52], [245] that "cyberspace" was coined by the science fiction writer William Gibson in 1984 [87], which in turn was motivated by the term *cybernetics*. Cybernetics was popularised by Norbert Wiener in 1948 [264]. Cybernetics is a Latin form of the Greek word κυβερνητικός (kybernetikos), which means "good at steering" — a metaphor used to signify the governance of people.

1.1.1 Cybercrime

Cybercrime refers to offensive and illegal activities in cyberspace. Examples include fraud, identity theft, data breaches, and data leaks, which can result in significant financial and reputational damage. Cybercrime has a material impact on organisations, individuals, and even nations, leading to economic losses, privacy violations, and national security threats. As cyber threats continue to evolve, understanding their implications becomes increasingly important. We provide some key implications of cybercrime on these entities below.

A **data breach** refers to unauthorised access to sensitive data, such as credentials and user information, posing a significant threat to individuals and organisations. IBM's *Cost of a Data Breach Report 2023* [102] indicates that the average cost of a data breach for victim organisations is USD 4.45 million—an increase from the 2020 estimate of USD 3.86 million [103].

Beyond the costs of containing the threat and conducting investigations, organisations must also face regulatory penalties. For instance, British Airways was initially issued a £183 million fine, which was later reduced to £20 million [238], for the Man-in-the-Browser (MitB) attack in 2019 (examined in more detail in Chapter 3).

Furthermore, these costs are often transferred to customers through price increases in products and services, as noted in the IBM report [102]. Users also suffer from potential data leaks, leading to identity theft, account takeovers, and numerous other possible consequences.

Data breaches often lead to **data leaks**, which then create a vicious cycle of further attacks. Leaked credentials are used for account takeovers or to carry out Credential Stuffing Attacks (CSA) or Credential Tweaking Attacks (CTA). The IBM report [102] suggests that previously stolen credentials and phishing attacks were responsible for most breaches. Verizon's 2024 Data Breach Investigations Report [250] presents similar findings. Verizon's report indicates that credential misuse through web applications and phishing via email were the most common attack vectors. We build on this analysis by conducting our own empirical investigation of common data breaches in Chapter 3.

Organised crime constitutes the majority of **threat actors** [250]. However, state-sponsored actors exist and are responsible for most sophisticated attacks. While non-state actors are motivated by financial gain, state-sponsored actors are motivated by information gain (spying on other countries) and damaging and weakening their adversaries. For example, the STUXNET virus had the aim of slowing down the progress of uranium enrichment in Iran.

Cybercrime has damaging and even paralysing consequences for economies regardless of the actor. Therefore, it is no surprise that governments around the world have created state-funded agencies to counter cybercrime. For example, the UK government's cyber security investment was £1.9 billion between 2016–2021, then increased to £2.6 billion over a three-year period in the 2021 budget review [246]. This is reflective of the growing cyber threat that requires further investment at the national level.

1.1.2 Authentication

Authentication, especially password-based authentication, is a key component of web security, and exploits in these systems are a key threat vector [250]. These systems and protocols serve as the gateway to our identity and secret data online. For these reasons, authentication receives

Case Study: Authentication Scheme Design Requirements

When designing authentication schemes, we must also consider scenarios where users deliberately share their authentication secrets. Here is a case study of such an instance, highlighting how the design of an authentication system can directly impact a company's revenue. Netflix, a subscription-based video-on-demand internet streaming service, has been attempting to address the issue of paying users sharing their credentials with individuals outside their home. This sharing reduces the number of paying customers and thus decreases revenue. Recently, Netflix reported a significant increase in revenue after their password crackdown [178]. In this scenario, Netflix needs to prevent users from sharing their credentials with others.

Consider when a user A purchases a subscription linked to their identity via an email address E and password x, which form their authentication secrets $\langle E, x \rangle_A$. These secrets, of course, can be shared with other users. Note that user A can also have multiple devices themselves. Netflix then limited the number of devices that could stream concurrently. However, this did not solve their primary issue of acquiring more paying users. Let D_A represent a set of devices authenticated with $\langle E, x \rangle_A$. We now need to classify the devices that belong to user A and those that are considered to be *shared*. In this scenario, the authentication becomes $\langle E, x, d \rangle_A$, where d is a vector of device attributes such as IP address, MAC address, device type, etc.

considerable attention in both the literature and industry. Authentication is the act of party A (the *prover* or *claimant*) proving their identity or who they claim to be to party B (the *verifier*). After authentication, the two parties (*principals*) should be assured that they are communicating with each other and not a malicious actor [35]. This is also known as *identification* or *entity authentication* [150]. In password-based schemes (weak authentication), a username or email address serves as a claim of identity.

Notable related dissertations in the last decade [31], [193], [249] have primarily focused on the guessability of passwords or alternative forms of authentication, such as behavioural biometrics [61] or EEG-based biometrics [168]. We take a holistic view by examining the stages of password-based authentication protocols and the journey of other user secrets. This method allows us to capture the weaknesses of the current approach and the limitations of existing research. For example, we find that the literature on password guessing is often disconnected and rarely contributes to the practical security of passwords—we investigate this in Chapter 5.

Password and the Journey of Secrets

Passwords are secret strings stored in our memories. They are considered to be a *what you know* form of authentication, which makes them highly usable. If a password is not memorable and is written on a post-it or stored in a password manager, for example, it becomes *what you have*. *What you are* is the last form of authentication. Biometric-based authentication systems are examples of such a form of authentication. The practicality of *what you know* comes at a cost to security, because of the limitations of how much we can remember. Humans also tend to embed predictable patterns in memorable secret strings, which can be exploited to guess them.

Passwords are a weak form of authentication as human-generated secrets are susceptible to being guessed. Passwords encapsulate a paradox where they have to be strong enough to be resistant against being guessed and yet memorable by the user. The user typically generates passwords that are easily guessable. The password distribution exhibits a power law, specifically Zipf's law [252]. This makes passwords statistically exploitable and computationally feasible to guess. This raises a research question of **whether a strong password that is human-generated and memorable exists**—we explore this in Chapter 4.

The first line of defence in generating secure passwords is during the composition phase. Tools such as **Password Strength Meters (PSM)** (such as zxcvbn [261]) and **Password Composition Policies (PCP)** are used to help the user generate stronger passwords, but there is a vast amount of research [11], [84], [85], [129], [133], [136], [140], [203], [267] showing that very few web applications follow any guidance on PCP. Moreover, there is disagreement on the choice of PCP. The choice of PCP is an open problem, which we also explore in Chapter 4.

Passwords are also propped up by other techniques such as **Multi-factor Authentication (MFA)**. Typically, this is facilitated by SMS, hardware keys, or phone-based OTP. These methods add an extra level of security to mitigate the weaknesses of the password. They, however, can have their own weaknesses too. For example, SMS-based MFA can suffer from SIM swap attacks, which is also a form of identity theft. By impersonating the victim to the telecom provider, the attacker digitally swaps the victim's SIM with their own. This is of course enabled

by the fact that organisations still use knowledge about individuals as a form of authentication. Information such as date of birth and address is often enough to conduct identification on the phone. This rudimentary form of authentication is a major contributor to identity theft. The following is a real example of a SIM swap attack: The $400 million theft of cryptocurrency in 2022 from FTX [17], [123], known as the "El Swapo" attack.

The use of MFA and difficult PCPs comes at a cost to **usability** and user experience. The use of MFA has of course made password-based schemes more secure, but at a cost to usability and user experience. In many cases, usability trumps security and privacy. The Cookie Policy and regulation was meant to give back rights to the users. The cookie policy pop-ups are a nuisance, so much so that the Brave browser, one that is security and privacy-focused, is blocking them. Users often choose the path of least resistance, and that might not be the most secure path.

The **storage method** of passwords has a tremendous impact on their security. Storage is the last step in the journey of this secret, and it is a crucial step that directly affects the economy of a guessing attack. Passwords are stored as salted hashes, the salt is a random sequence to defend against rainbow table attacks (pre-computing the hashes of passwords). For simplicity assume passwords are stored as hash values y using a cryptographic hash function h. This function is a one-way function where it takes the input (password) x and creates the hash value output, $h(x) = y$. Typical cryptographic hash functions were designed to be efficient and computationally easy to compute. To deter an attacker from brute-forcing (i.e., computing $h(x) = y$ at speed), we want a function that is resistant to such an attack. These functions are known as *key-derivation functions*. Data breaches often lead to the hash values y being leaked, which are then cracked and revealed to the public. A research question for us is **whether alternatives to cryptographic hash functions and key-derivation functions exist**. This motivated us to explore zero-knowledge proofs, specifically identity-based cryptosystems in Chapter 6, which takes us into the password-less authentication territory.

Going beyond passwords is also an active but slow area of research. **Password-less authentication** is motivated by overcoming the weaknesses of passwords and the usability issues of MFA. The ubiquity of mobile phones and the decrease in the cost of electronics are motivating the

use of *what you have* and *what you are* forms of authentication. However, the standards that are being introduced are dogged by confusion and complexity; therefore, the implementation and adoption have been slow. Passkeys were introduced by Apple in 2022. There are privacy concerns in using cloud infrastructure for the storage of cryptographic secret keys. Password-less startups [119], [265], [266] are being actively invested in. This signals that there is demand for password-less technology. Services also use email-based authentication; here a secret token is sent to the user's email address to verify ownership/access.

The above summary of password-based authentication systems and the journey of passwords as secrets reveals many weaknesses that demand a holistic view of security protocols. Perhaps FTX would still be around today had they not lost $400 million to a SIM swap attack. This dissertation thus takes such an approach. We hope this approach can motivate researchers to be more daring in designing new systems and protocols in a field that is cautious and continuously reminds everyone to stick to what exists and not to innovate.

Password Guessing Attacks

It is important to consider guessing methods and algorithms because they help us create stronger passwords and improve password composition policies. Password guessing encapsulates all aspects of a password-based authentication system, where the choice of storage function, the composition of passwords through composition policies, and the appropriate strength meter determine the overall robustness of the password. All these factors influence a password's resistance to being guessed. They receive significant attention in the literature, and with advances in deep learning techniques and architectures, we have seen more guessing methods utilising machine learning. It should be noted that all password guessing research relies on leaked password datasets, and the increase in data breaches has, to some extent, contributed to its growth. Password guessing has also become a "sport", as exemplified by the following.

Every year, KoreLogic [117] hosts a password cracking challenge at the DEFCON[1] conference, and as the premier event of its kind, it attracts the best teams from across the world. Team

[1] defcon.org

Hashcat (the developers of the Hashcat [228] password guessing software) have been the winners for the last three years. Their 2023 lineup consisted of 20 hackers, 47 FPGA boards, 78 GPUs, and cloud infrastructure—not your average team. Their post-challenge writeup[2] provides a strong illustration of how passwords are handled in the real world. Seldom were all the high-performance GPUs running at full capacity because password guessing is more than simply feeding a hash into a machine and waiting for the cleartext output. There are exploitable patterns in these secrets. The KoreLogic challenge mimics this real-world scenario of human-chosen secrets. The goal then becomes discovering the underlying patterns rather than brute-forcing the password. This art is known as *password guessing* in the literature.

1.2 Objectives

We aim to contribute to web security by taking a holistic view of the journey of user secrets. The objectives of this dissertation are as follows:

1. **Password Security.** Investigate the weaknesses of the password string through a composition analysis of empirical datasets. Consequently, use this data to compare the different password strength measures and composition policies and provide a recommendation on a usable and secure password composition policy.

    - Create a labelled dataset of passwords showing the composition process of the password. This dataset will contain the password *chunks* and the *transformations*.
    - Propose new methodologies to expedite the password labelling process. Compare the cost and speed of different methods.
    - Investigate how users embed predictable and identifiable information about themselves and their surroundings in their passwords. Identify patterns that have not been clearly identified before.

[2] `https://github.com/hashcat/team-hashcat/tree/main/CMIYC2023`

2. **Server-side Security.** Evaluate the applicability of identity-based cryptosystems in proposed authentication systems in the context of compromised clients or servers.

3. **Client-side Security.** Propose how the ubiquitousness of mobile phones and cheap micro-boards can be used for the transaction of secrets in the context of a leaky or compromised user device or browser.

4. **Data Breaches.** Collect and summarise reported data breaches.

    - Identify the cause of the attacks and categorise each examined report.
    - Examine the role of authentication systems, especially the role of the password before and after the data breach.
    - Propose recommendations on how to counter future threats based on the data.

1.3 Contributions

We contribute to the field of web security. By taking a novel perspective of the journey of user secrets, we provide an integrated narrative of potential security pitfalls and methods to overcome them. Our contributions are as follows.

- *Breached data:* Using news reports from open-source media outlets, we analysed 60 web security and data breaches that occurred in 2020. We gathered the cause, date, amount and type (PII, finance, health, etc.) of data released, and other information such as fines and the actors behind the attack.
- *Lessons identified and opportunities:* Using the breached dataset, we identified a set of lessons-to-be-learnt and research opportunities. We believe our contributions will be valuable to industry professionals and the research community.

Password Security. Human-generated and memorable secrets, passwords, are crucial a crucial web security components. We contribute to the security of passwords by analysing decades of

research, providing new datasets, providing a method to compare password composition policies and consequently recommend the strongest and most usable password composition policy.

- *Human-labelled password corpus:* We decompose and label the hotmail-2009 [153] password strings (8 292) into chunks, words, structure, tags, and transformations, where applicable. Using the the methods developed in Chapter 4, we machine label the Lizard-Squad 2015 dataset [157]. To our knowledge, this is the first dataset of its kind. Insights derived from our labelled dataset are discussed, and the dataset is made available for research. This dataset in itself is a novel contribution, we go on to show how it can be used to improve our knowledge of password composition policy.

- *Password composition:* Utilising our labelled dataset, we evaluate secure and practical password composition strategies. We find that password structures, such as three-word passphrases without semantic links between words, offer both security and practicality. Using the hotmail-2009 dataset and the Lizard-Squad 2015 dataset offers an opportunity to investigate changes in password composition with respect to dates and thus reflect on the age of these datasets.

- *Large Language Models and human labellers:* Labelling data is time-consuming and costly, prompting exploration of alternatives to expedite the process. We engaged freelancers from Fiverr [74] and assessed their work against Large Language Models in terms of accuracy, cost, and time. The preliminary results indicate that language models are a promising tool for further experimentation.

- *Password guessing algorithms.* We provide the first ever systematic exposition of password guessing methods and algorithms. Our aim is to unify the scattered body of knowledge and the disconnect between academic and industry methods in order to create a more rigid and reproducible future experiments.

Server-side Security. Data leaks often contain large amounts of users' PII and password information; this leads to credential stuffing attacks (CSAs), impersonation, and identity theft.

We design and implement deeID, which overcomes the problem of poor credential storage by using a zero-knowledge proof system and a dynamic system in which many organisations can issue identities.

- *Fiat–Shamir cryptosystem extension:* We propose a new approach to the Fiat–Shamir cryptosystem where an ecosystem can have multiple identity issuers. The cryptosystem is explained in with its design into deeID in Chapter 6.
- *Design and implementation of* deeID*:* The proposed system is a mobile phone and blockchain-based authentication and identification scheme. We design the system with the Fiat–Shamir identification scheme that utilises a blockchain network as a decentralised PKI. Design of deeID is explained in Chapter 6. The implementation constitutes of the following components: mobile phone application, Ethereum smart-contracts, web application and accompanying code.
- *Mobile phone and blockchain based authentication scheme:* We compare this new class of authentication scheme through deeID using the Semi-structured Evaluation of User Authentication Schemes Framework [30].

Client-side Security. In Chapter 3 we find the devastating effect of Man-in-the-Browser (MitB) malware, exemplified by the British Airways case in 2019 [115]. So, our contribution here is a novel out-of-band method to protect user secrets from this malware whilst increasing usability at the same time.

- *Design and implementation of* FormL3SS*:* An out-of-band form and data transmission system. Implementation of the proposed system that can circumvent untrustworthy browsers and systems with respect to sending sensitive information to a trusted web server.
- *Standardising form requests:* We introduce a messaging standard that currently supports payment and sensitive information. Other type of information can be requested by signing the message's `form_type` as `custom`.

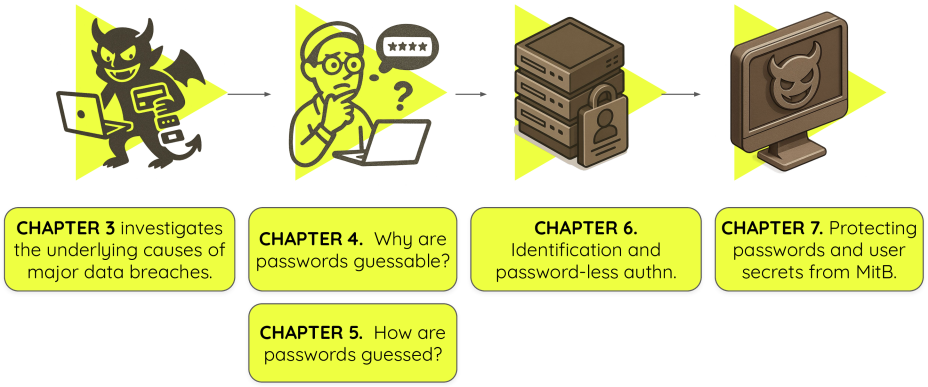


Figure 1.1: Graphical representation of the dissertation's outline. We explore the journey that user secrets take from the moment a user composes a password to their storage in the server.

1.4 Dissertation Outline

Figure 1.1 presents a graphical representation of this dissertation's structure. To establish a foundation for the dissertation, we conduct our own empirical research on data breaches, which informs the subsequent chapters. The dissertation then explores key checkpoints in the journey of user secrets. The outline of the dissertation is as follows.

- **Chapter 2** provides background information and a literature review on authentication schemes, topics related to password security, identity-based cryptosystems, and an overview of common threats to user secrets on both the client and server side. Given the extensive scope of this dissertation, we will include essential background information in the respective chapters when necessary. The majority of the password guessing background and literature will be covered in Chapter 5.

- **Chapter 3** investigates the underlying causes of major data breaches. We attempt to validate the findings of IBM [102], [103] and Verizon's reports on data breaches using publicly available data. The findings from this chapter will guide the rest of the dissertation. For instance, we chose to conduct an experiment (Chapter 7) on Man-in-the-Browser (MitB) malware due to the high number of reported breaches attributed to this type of attack.

- **Chapter 4** aims to address the question of how to compose secure passwords. In doing

so, we re-examine why passwords are guessable using the first human-labelled password dataset. We use the data to compare different Password Composition Policies and assess their validity. We find that there is a lack of consensus on what constitutes a strong password, and very few web applications adhere to password guidelines. Using empirical analysis and a novel mathematical framework, we recommend a five-word PCP.

- **Chapter 5** continues the discussion on memorable secrets and aims to understand how they are guessed. We provide the first systematic exposition of password guessing methods and algorithms. Our goal is to unify the scattered body of knowledge and bridge the gap between academic and industry methods to create a more rigorous and reproducible framework for future experiments. In this chapter we also produce a framework that uses both quality of generated guesses and the speed at which they are generated. Understanding how passwords are guessed helps us create more robust password composition policies and password strength meters.

- **Chapter 6** investigates methods of secret storage on a server. We argue for the adoption of identity-based cryptosystems as a means of overcoming the limitations of cryptographic hash and key-derivation functions when storing memorable secrets. We also demonstrate how Blockchain technology can be used to create identity systems at the national level.

- **Chapter 7** evaluates the threats originating from the client-side, specifically the browser. Motivated by the findings in Chapter 3, we design and implement a method of circumventing Man-in-the-Browser malware using out-of-band devices. We also incorporate the work from Chapter 6, using the Blockchain infrastructure to perform identification between the server and the user's out-of-band device.

- **Chapter 8** provides a summary and conclusion of the dissertation. We also discuss the ethical issues, applications, and potential economic impact of this research. Lastly, we briefly discuss the personal journey and other work that was experimented with but not included in the dissertation.

1.5 Publications and Statement of Originality

I declare that this dissertation was composed by myself, and that the work it presents is my own, except where otherwise stated. The following publications arose from work conducted during the course of this PhD.

- S. Almasi and W.J. Knottenbelt. **Five Word Password Composition Policy**. NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb 2025), San Diego, California, USA, Feb 2025.
- S. Almasi and W.J. Knottenbelt. **Protecting Users from Compromised Browsers and Form Grabbers.** NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb 2020), San Diego, California, USA, Feb 2020.

Five Word Password Composition Policy is the result of Chapter 4. *Protecting Users from Compromised Browsers and Form Grabbers* is the result of Chapter 7.

Chapter 2

Background

This chapter presents an overview of authentication schemes, cryptographic prelims, and identity-based ciphers. Passwords, a human-chosen and memorable secret, forms the foundation of many modern authentication schemes, hence a detail coverage of it here and a dedication of two chapters on why they are guessable (Chapter 4) and how they are guessed (Chapter 5). Identity-based ciphers are used in Chapter 6; therefore we provide an overview of them here.

2.1 Entity Authentication

Entity authentication or *identification* is the process of verifying the identity of a user, device, or system before granting access to sensitive information or functionality. It allows the *claimant* to reassure another entity (the *verifier*) of their identity. In related literature [150], the terms identification and entity authentication are often used interchangeably. Identification protocols are ubiquitous; we encounter them most commonly when logging into web applications. They are also used when unlocking a car with a wireless key fob or accessing an ATM.

Password-based authentication is the most prevalent form of entity authentication, tracing its history back to the early days of computing. This form of authentication is considered weak due to the vulnerabilities associated with user-chosen passwords. The password itself is a

Protocol

A protocol in computing and telecommunications is a set of rules and conventions that govern the exchange of data between devices. It defines the procedures for data transmission, including how data is formatted, transmitted, and received. Protocols ensure that devices can communicate effectively and understand each other, regardless of differences in their underlying hardware or software.

string—often a minimum of six characters—selected and memorised by the user, serving as a shared secret between the user and the system (verifier). The password string is linked to a claim of identity, such as a unique username or email address. We explore password-related topics in depth in Section 2.6.

Personal identification numbers (PINs) are commonly used on mobile devices or in conjunction with banking cards to prevent unauthorised access to personal devices or other physical possessions. PINs are typically four or six digits long and are most commonly employed as a secondary form of authentication.

Fixed secret schemes, such as passwords and PINs, are susceptible to replay attacks due to eavesdropping. **One-time passwords (OTPs)** mitigate this risk by ensuring that each password is used only once per login attempt. There are two variants of OTP: (1) Hash-based OTP (HOTP) and (2) Time-based OTP (TOTP). HOTP relies on a counter that increments with each use, whereas TOTP is based on the current time, typically changing every 30 seconds. As a result, TOTP is short-lived and requires only reasonably synchronised clocks, whereas HOTP necessitates synchronisation between both the client and the server. Due to its simplicity and security benefits, TOTP is the more commonly used algorithm.

Protocols that require interaction between entity A (the *claimant*, who is proving their identity) and entity B (the *verifier*) are known as **challenge-response authentication**. These protocols utilise a time-variant challenge to defend against active attacks; the challenge is provided by entity B (the verifier—a system or device). Kerberos and Needham–Schroeder are examples of symmetric-key, challenge-response-based protocols. We discuss zero-knowledge-based schemes in Section 2.9 and utilise them in Chapter 6.

Figure 2.1: Photograph of a hardware-based OTP device that is capable of generating event-based and time-based one-time passwords. This form is a visual-based hardware OTP as it requires the user to identify the digits and enter them onto another device.

2.2 Attacks on Entity Authentication Protocols

There are various security threats to identification protocols [150]:

- *Impersonation*. Pretending to be a different person.
- *Replay attack*. Use of information from a previously used protocol process, e.g. re-use of a poor signature.
- *Interleaving attack*. Multiple concurrent execution of the protocol and using selective information for possible attacks.
- *Reflection Attack*. Attack on a challenge-response method to essentially tricking the challenger into providing itself with the answer.
- *Forced Delay*. Requires intercepting a message and delaying it until some later time (thus not a replay attack).
- *Chosen-text Attack*. Strategically choosing challenges in an attempt to extract information about the prover's secret information.

2.3 Multi-Factor Authentication

Multi-factor authentication (MFA) is a layered approach to security and access control, combining multiple authentication methods to enhance the overall security of a system. Applications

typically implement two-factor authentication (2FA), which pairs password-based authentication with an OTP or SMS verification. In practice, OTP methods are commonly implemented through mobile phone applications.

The distinction between what constitutes multi-factor authentication is not always clear-cut. Often, if the *user* is required to interact with multiple systems, it is considered multi-factor authentication. However, when metadata—such as location and IP address—is used, it is less clear whether this qualifies as multi-factor authentication.

Use of multi-factor authentication comes often at a cost to usability. Users are often required to use a secondary device or have access to a mobile phone with LTE access. Use of multi-factor authentication provides additional costs to organisations too–an example of this is when Twitter forced non-paying users to switch from SMS-based 2FA to other forms of 2FA [232]. The reasoning for this was both cost (to Twitter) and the fact that SMS-based 2FA can be vulnerable for most users. The user experience of MFA has hardly changed over the last decade [54], indicating a lack of research or adoption in the research.

The use of multi-factor authentication often comes at the cost of usability. Users are frequently required to utilise a secondary device or have access to a mobile phone with LTE connectivity. Additionally, multi-factor authentication imposes extra costs on organisations—an example of this is when Twitter required non-paying users to switch from SMS-based 2FA to alternative forms of 2FA [232]. The rationale behind this decision was both the cost to Twitter and the vulnerability of SMS-based 2FA for most users. The user experience of MFA has remained largely unchanged over the past decade [54], suggesting a lack of research in this area.

SMS-based multi-factor authentication (MFA) involves sending a one-time password (OTP) to the authenticating user via SMS. A mobile phone number is registered to the user's account prior to this authentication stage. However, SMS-based MFA is susceptible to SIM-swapping attacks and SMS interception. Despite these vulnerabilities, SMS-based MFA remains widely adopted due to its convenience and its straightforward usability for users.

One-time passwords (OTPs) are unique authentication secrets generated using a shared secret

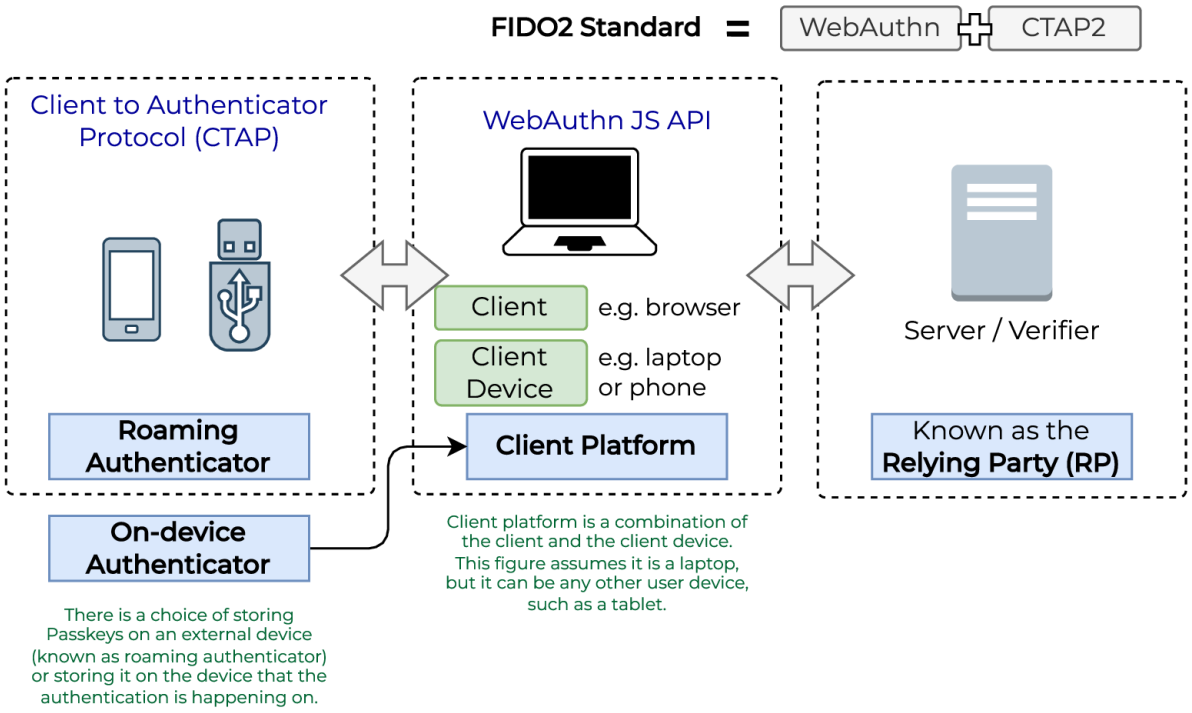


Figure 2.2: Diagram showing the core components of the FIDO2 standard. FIDO2's primary intent is to replace password-based authentication with public-key cryptography-based authentication. It consists of the WebAuthn JS API and the protocol to communicate with external devices known as roaming authenticators. Source: Own figure.

and are used only once per authentication transaction. Their uniqueness and time-sensitive nature make them resistant to replay attacks. In practice, OTPs are generated through mobile phone applications, dedicated hardware devices, or out-of-band channels.

Hardware tokens are physical devices that generate OTPs. Unlike SMS or app-based OTPs, these tokens operate independently of the user's mobile device or computer—reducing the risk of side-channel attacks. Hardware tokens are also less susceptible to hacking or phishing attacks, as they do not rely on potentially vulnerable network connections. Figure 2.1 illustrates an example of a hardware-based OTP generator.

2.4 FIDO2

FIDO2 is an authentication standard, which combines the WebAuthn [259] (Web Authentication API) and CTAP2 [50] specifications. It represents the primary ongoing effort to replace password-based authentication (i.e., weak authentication) with strong authentication: public-key cryptography-based authentication. It consists of specifications for public-key cryptography-based authentication. Figure 2.2 provides an illustration of the standard's core components.

The credentials in this ecosystem are known as WebAuthn credentials, FIDO credentials, or Passkeys. We will refer to them as Passkeys. Passkeys were introduced by Apple during WWDC 2022 [147] and have attracted considerable attention since then. The advantage of Passkeys for companies like Apple is that they can utilise the biometric features on the device to enhance the authentication experience for the user. However, a drawback is that users are locked into their platforms, as Passkeys are stored on iCloud, or they may lack the knowledge to transfer keys.

The Web Authentication API (WebAuthn) [259] is the specification and JavaScript API developed by W3C and the FIDO Alliance to enable public-key cryptography-based authentication. It is an extension of the Credential Management API. WebAuthn provides the necessary functions to interact with the verifier (server or relying party) and authenticators (which can be on-device or external—roaming authenticators). The Client to Authenticator Protocol (CTAP), currently in version 2 (CTAP2), is the specification that enables authenticators to interact with the WebAuthn API and the client.

FIDO2 is an initiative of the FIDO (Fast IDentity Online) Alliance, an open standard association formed through collaboration among various companies. The association was launched in February 2013 with the aim of creating and promoting a password-less authentication standard within the community. They define their mission as *"authentication standards to help reduce the world's over-reliance on passwords."* The executive leadership of the alliance includes members from Google, Amazon, Intel, Thales, Apple, Microsoft, and Docomo.

The work of the FIDO Alliance has been hindered by confusion and complexity, leading to a

lack of widespread adoption. The maze of specifications, insufficient developer support, and evolving terminologies have made it challenging for developers to implement FIDO2 effectively. The alliance and its output exemplify a design-by-committee approach. Some literature [69], [86] has examined the usability of FIDO2 systems in practice, concluding that users often revert to passwords or password managers due to their speed and convenience. Currently, FIDO2 is primarily used for second-factor authentication.

The journey of the FIDO Alliance began with the `Universal Authentication Framework (UAF)` and `Universal 2nd Factor (U2F)` standards in 2014. `Universal 2nd Factor (U2F)` is the standard used to enable two-factor authentication (2FA) through USB HID or NFC. It was succeeded by the Client to Authenticator Protocol 1 (`CTAP1`) standard, which provides a generalised interaction between the client device and secondary devices.

2.5 Secure Hardware

Isolated hardware with its own operating system can be used to protect user secrets, particularly on mobile phones and wearable devices. These devices store sensitive information such as passwords, financial data, and biometrics, and are ubiquitous; however, they also host numerous untrusted applications that may attempt to access this information. Most importantly, such devices can easily fall into the wrong hands. A response to these threats is dedicated hardware, or a segment of a System-on-Chip (SoC), separated from the rest of the system. This dedicated hardware is referred to as a Trusted Execution Environment (TEE) or a Secure Enclave.

A Trusted Execution Environment (TEE) is a dedicated area of the main processor with its own operating system (for example, on Android, the operating system running on the secure enclave is Trusty TEE [14]). The purpose of such architectures is to isolate critical functions and protect cryptographic secrets in isolated contexts (enclaves) from the rest of the system. The remainder of the system is often referred to as the Rich Execution Environment (REE).

Most smartphone manufacturers now build secure enclaves directly into their devices' hardware. Apple design and manufacture their own SoCs and refer to the dedicated secure hardware as

the *Secure Enclave*. Other manufacturers, such as Samsung, use ARM-based processors and therefore adopt the *ARM TrustZone* architecture. TEEs in mobile devices store passwords, PINs, biometric data, and other similar secrets. They protect user data by, for example, rate-limiting PIN attempts (with exponential delays) even when a malicious actor has physical access to the device. Biometric data can be stored and processed within the secure enclave; for instance, Touch ID, Face ID, and Optic ID on Apple devices [15] are stored and processed exclusively within the secure enclave. On Android, developers can access the TEE via the Keystore API [13]. Despite the availability of this API, it has been shown [26] that most Android applications do not use it for secret storage.

A Hardware Security Module (HSM) is dedicated hardware designed to manage cryptographic secrets. For example, in the cloud, an HSM may be used to avoid loading cryptographic keys into the memory of the main web server. Such isolation prevents leakage and unauthorised access to the keys by other applications.

2.6 Topics on Password

Passwords are obvious user secrets. Much of this dissertation focuses on human-chosen and memorable secrets, such as `il0vesk00l!`, as their weaknesses stem from predictable and frequently used patterns. Password security and authentication constitute a vast field encompassing multiple subfields. Some examples are as follows.

- Password usability and user experience
  - Password security in enterprise environments
  - Password creation and memorability
  - User education and awareness
  - Password statistics (length, count/freq, structure info)
- Password reset and recovery mechanisms

- Password modelling and similarity
  - Similarity (e.g. distance-like metrics)
- Password entropy and complexity analysis
  - Password composition policy (PCP) (or password creation policy)
  - Password strength meters (PSM)
- Password guessing algorithms
- Password hashing and encryption techniques
- Password-less authentication schemes
- Password authentication protocols
- Federated identity and single sign-on (SSO)
- Biometric and multi-factor authentication
- Password-based authentication in specific domains (e.g., web, mobile, IoT)
- Password managers and storage

Statistical and Lexical Password Analysis. Historical analyses of password characteristics have focused on *length*, *count*, and *character-level structures* [37], [55], [130], [131], [138], [194], [199], [248]. Our research advances this field by examining nuanced aspects of passwords with greater precision. Moreover, the evolution of language models, combined with our successful prototypes, presents a cost-effective approach for labelling additional passwords.

In the analysis of Chinese passwords [131], the authors conduct a broad examination of password syntax. Our research extends [131] by delving deeper into password structures. For instance, they identified $LLLLLL$ as the predominant structure within the `Rockyou` dataset, typically representing a word, as demonstrated in our results. Further structural analysis across various password datasets is presented in [130]. Das *et al.* [55] explored password syntax, notably

introducing an analysis of word or phrase transformations, which aligns with aspects of our research. However, we disagree with their classification of sequential keys, alternate keys, and sequential alphabets as transformations, instead viewing these as sequences. Riddle *et al.* [208] investigate the composition and semantics of passwords, with a particular emphasis on the psychological significance of words.

In "Investigating the Distribution of Password Choices" [139], the analysis of the hotmail dataset concluded that its distribution closely resembles Zipf's law. Our analytical efforts extend this observation by demonstrating that extracted words from unique passwords also follow a similar distribution—more of this is found in Chapter 4.

Password Modelling. Early password modelling techniques, such as the Markov n-gram model [171], focused on individual characters, positing that *"the distribution of letters in easy-to-remember passwords likely mirrors the letter distribution in the user's native language."* We validate this in Section 4 at the word level. The observation that letter distribution is not uniform holds true for words, exhibiting a power-law distribution akin to Zipf's law. Modelling passwords based on Probabilistic Context-Free Grammars (PCFG) [260] was the next innovation. Our insights into empirical password structures (§4.4) can refine `PCFG` [260] modelling. Incorporating W_n and N_n as variables for words and names, respectively, enhances the computational viability of modelling more intricate structures. Subsequent advancements have leveraged machine learning techniques in password modelling, including neural networks [36], GANs [97], RNNs [148], and Transformers [269], among others.

Password Composition Policy (PCP). Despite extensive research into password complexity measures and meters, recommendations for password policies remain scarce. Bonk *et al.* [29] provide guidance on constructing passphrases, effectively extending the NCSC's [172] three-word suggestion to longer sequences, such as seven words or more. `zxcvbn` [261] employs dictionaries and bespoke rules to gauge password strength on a scale from 0 to 4. However, `zxcvbn` [261] has its limitations; for instance, it incorrectly parses the password `gomythsun` as `"go myths un"`, which affects its strength rating. Additionally, it struggles with transformations such as `"numbr"` from `"number"`. Wang *et al.* [253] conduct a deeper evaluation of password strength

meters. Their endorsement of `zxcvbn` [261] and Markov n-gram models has informed our research approach.

The deployment of language models, particularly Large Language Models (LLMs), in this domain remains notably scarce. Rando *et al.* [206] explored password modelling using the GPT-2 framework. Given the broad embeddings that encompass multiple languages, the semantic breakdown and analysis of passwords represent, in our view, a pioneering application of LLMs.

2.7 Computational Hardness Assumptions

Much of modern security and cryptography is built upon one-way functions—functions that are easy to compute but difficult to invert. The *difficulty* is based on computational and algorithmic efficiency. An algorithm is *efficient* if it runs in polynomial time. More formally, an algorithm is considered efficient if its running time is bounded by a polynomial function of the input size. That is, for an input of size n, the running time is $O(n^k)$ for some constant k. Cryptographic primitives and protocols are often designed under the assumption that certain computational problems cannot be solved efficiently, i.e., in polynomial time. The hardness of these problems is crucial for maintaining the security of the cryptographic systems based on them. The hypothesis that a problem cannot be solved efficiently is known as the computational hardness assumption.

Trapdoor one-way function is a variant of the one-way function that requires a secret key. Using this secret key, one can invert the function; without it, it remains hard to invert.

The existence of one-way functions remains an open conjecture in computer science. However, there are several promising candidates that have withstood attempts at inversion to date. For these functions, no efficient algorithms to compute their inverses have been discovered. Examples of such candidate one-way functions include:

- **Integer factorisation problem:** Given a positive integer n, find its prime factors. For example, if $n = pq$, where p and q are distinct prime numbers, the factorisation problem is to find p and q given only n. Also known as *Factorisation Problem* or simply *Factoring*.

- **RSA problem:** The RSA algorithm was the first public key encryption algorithm; though it was also invented by Clifford Cocks at GCHQ in 1973, four years before Rivest, Shamir and Adleman. The RSA problem is finding m given the public key (n, e) and a ciphertext $c \equiv m^e \pmod{n}$. The security of the RSA algorithm is no harder than the integer factorisation problem, so if we can solve the factoring problem, then we can solve the RSA problem. However, there is no proof that solving the RSA problem will help us solve the factoring problem (D. Boneh and R. Venkatesan Breaking RSA may not be equivalent to factoring). Also known as *RSA Assumption* or *RSA Inversion*.

- **Quadratic residuosity problem:** Given a composite number n and an integer a, determine whether a is a quadratic residue modulo n. In other words, find whether there exists an integer x such that $x^2 \equiv a \pmod{n}$.

- **Discrete logarithm problem (DLP):** Given a finite cyclic group G, a generator g of the group, and an element $h \in G$, find an integer x such that $g^x = h$. The integer x is called the discrete logarithm of h with respect to g, denoted as $x = \log_g h$. This problem is considered hard in certain groups, such as the multiplicative group of a finite field or the group of points on an elliptic curve over a finite field.

- **Diffie-Hellman problem:** Given a finite cyclic group G, a generator g of the group, and elements g^a and g^b, where a and b are unknown integers, compute g^{ab}. This problem is closely related to the discrete logarithm problem and is used in the Diffie-Hellman key-exchange protocol. The assumption that this problem is hard is known as the Computational Diffie-Hellman (CDH) assumption.

Quantum computing does indeed pose a threat to some of the computational hardness assumptions, as quantum computers can solve some problems exponentially faster. Shor's algorithm [223] can solve both the integer factorisation and discrete logarithm problems in polynomial time. This algorithm, for example, threatens RSA- and DLP-based cryptosystems, as it can factor an integer n in $O((\log n)^3)$ time and $O(\log n)$ space.

Post-quantum cryptography deals with finding quantum-resistant problems. For example, *lattice*

problems (such as the Shortest Vector Problem) are believed to be quantum resistant. NIST has been actively standardising the process for post-quantum cryptography since 2016. In August 2024, NIST released ML-KEM (CRYSTALS-Kyber) for key establishment. They also released ML-DSA (CRYSTALS-Dilithium) and SLH-DSA (SPHINCS+) for digital signatures. Other national organisations, such as the UK's NCSC, are actively providing guidance to navigate the post-quantum cryptography journey [175].

Whilst quantum computers pose a threat to many cryptosystems, it is believed that cryptographic hash functions are quantum resistant. Thus, they do not undermine the security of password-based authentication systems. However, Grover's algorithm [93] could aid a malicious user in inverting a function. Grover's algorithm can accelerate unstructured search problems, providing a quadratic speed-up compared to classical algorithms.

2.8 Cryptographic Hash Functions

A hash function is an umbrella term for functions with the following property. A hash function h takes an arbitrary-length input x and outputs y, a fixed-length bit string. This is often represented as: $h(x) = y$, where x is the input, message, or preimage, and y is the output, hash value, or hash digest. Table 2.1 shows examples of hash functions and their key properties. In cryptography, we require the hash function to have specific properties for it to be useful. We differentiate it by calling it a cryptographic hash function or a collision-resistant hash function.

Key properties of a cryptographic hash function are as follows.

- **Preimage resistant**: Given a hash value y it should be computationally infeasible to find an input string x such that $y = h(x)$. i.e. it should be computationally infeasible to reverse the hash. Some literature may refer to this property as *one-way property.*
- **Second preimage resistant**: Given some input x_1 and a hash value y, it should be computationally infeasible to find another input x_2 that results in the same hash value y. Some literature may refer to this property as *weak collision resistant.*

Table 2.1: Overview of various hash functions and their key parameters. n is block size in bits and m is output size in bits.

Hash function	n	m	Preimage	Collision	Collision Resistant	Construction
SHA-1	512	160	2^{160}	2^{80}	No	Merkle-Damgård
SHA-256	512	256	2^{256}	2^{128}	Yes	Merkle-Damgård
SHA-512	1024	512	2^{512}	2^{256}	Yes	Merkle-Damgård
MD5	512	128	2^{128}	2^{64}	No	Merkle-Damgård
RIPEMD-160	512	160	2^{160}	2^{80}	Yes	Merkle-Damgård
Whirlpool	512	512	2^{512}	2^{256}	Yes	Miyaguchi-Preneel
Blake2b	1024	512	2^{512}	2^{256}	Yes	HAIFA
Blake2s	512	256	2^{256}	2^{128}	Yes	HAIFA

- **Collision resistant**: It should be computationally infeasible to find two inputs that result in the same output. Some may refer to this as the *strong collision resistant property*.

When designing a hash function, we also require small changes in the input to lead to a large change in the output. This is called the *avalanche effect*.

Passwords are hashed alongside a unique and random string called a *salt*; this is done per password to prevent *look-up table* or *rainbow-table* attacks. A rainbow table is a precomputed table used for caching the outputs of a cryptographic hash function. A secure method of password storage employs *key derivation functions*. These functions still use one-way functions as primitives but are slower to compute and resistant to rainbow table attacks.

2.8.1 Key Derivation Functions

Key derivation functions (KDFs) are one-way functions that generate cryptographic secret keys from inputs such as passwords. For their use case in password storage and authentication, they offer several advantages over standard cryptographic hash functions. Standard hash functions are designed to be efficient and fast to compute. For password storage, this efficiency makes them insecure, especially as dedicated hardware makes it trivial to compute a large number of hashes per second. This allows an adversary to guess a vast number of passwords. KDFs aim to

solve this problem by introducing CPU- and memory-intensive operations, thereby slowing down brute-force and other guessing attacks. Table 2.2 shows various KDFs and their key properties.

Definition 2.8.1 (A password-based key derivation function) *A password-based key derivation function, or PBKDF, is a function H that takes as input a password $w \in \mathcal{P}$, a salt $s \in \mathcal{S}$, and a difficulty $d \in \mathbb{Z}^{>0}$. It outputs a value in $0,1^\ell$ for some ℓ. We require that H is computable by an algorithm that runs in time proportional to d. We say that the PBKDF is defined over $(\mathcal{P}, \mathcal{S}, 0, 1^\ell)$.*

NIST's Storage Recommendation

Verifiers SHALL store memorized secrets in a form that is resistant to offline attacks. Memorized secrets SHALL be salted and hashed using a suitable one-way key derivation function. Key derivation functions take a password, a salt, and a cost factor as inputs then generate a password hash. Their purpose is to make each password guessing trial by an attacker who has obtained a password hash file expensive and therefore the cost of a guessing attack high or prohibitive.
Examples of suitable key derivation functions include Password-based Key Derivation Function 2 (PBKDF2) [SP 800-132] and Balloon [BALLOON]. A memory-hard function SHOULD be used because it increases the cost of an attack. The key derivation function SHALL use an approved one-way function such as Keyed Hash Message Authentication Code (HMAC) [FIPS 198-1], any approved hash function in SP 800-107, Secure Hash Algorithm 3 (SHA-3) [FIPS 202], CMAC [SP 800-38B] or Keccak Message Authentication Code (KMAC), Customizable SHAKE (cSHAKE), or ParallelHash [SP 800-185]. The chosen output length of the key derivation function SHOULD be the same as the length of the underlying one-way function output.

Memory-hard functions require a large amount of memory to be computed and impose computational penalties if less memory is used. `Scrypt` [200] is one of the first such functions to utilise this approach but suffers from two key vulnerabilities [77]: its password-dependent memory access patterns make it susceptible to cache-timing attacks, and its memory storage approach leaves it vulnerable to garbage-collector attacks, where an adversary can bypass the memory-hardness entirely by recovering stored values after computation. Subsequent key derivation functions, designed with password hashing in mind, aimed to be simpler, resistant to

Table 2.2: Overview of various key derivation functions (KDFs) or also some known as Password Hashing Functions.

Year	Function	Memory-Hard?	Primitive
1999	`Bcrypt` [205]	×	Blowfish
2000	`PBKDF2` [111]	×	HMAC
2009	`Scrypt` [200]	✓	Salsa20/8
2013	`Catena` [76]	✓	Blake2b
2015	`Makwa` [202]	×	Squaring Modulo
2015	`yescrypt` [177]	✓	SHA-256
2015	`Argon2` [23]	✓	Blake2b
2015	`Lyra2` [12]	✓	Blake2b
2016	`Balloon` [28]	✓	Blake2b

side-channel attacks, and memory-hard. The Password Hashing Competition[1] was an initiative to accelerate the development of such new functions.

The 2015 Password Hashing Competition gave birth to a host of functions with the following selection criteria[2]:

- Defense against GPU/FPGA/ASIC attackers
- Defense against time-memory tradeoffs
- Defense against side-channel leaks
- Defense against cryptanalytic attacks
- Elegance and simplicity of design
- Quality of the documentation
- Quality of the reference implementation
- General soundness and simplicity of the algorithm
- Originality and innovation

`Argon2` [23] was the winner of this competition which provided three variants to mitigate time-memory tradeoff attack: 1) *Argon2d* maximizes resistance to GPU cracking attacks but

[1] `https://www.password-hashing.net/`
[2] `https://www.password-hashing.net/report-finalists.html`

introduces potential side-channel attacks; 2) *Argon2i* is conversely optimized to resist side-channel attacks; 3) *Argon2id* is a hybrid version of the two prior variants. OWASP [188] recommends using the Argon2id variant with a minimum configuration of 19 MiB of memory, 1 degree of parallelism, and 2 iteration counts. `Catena` [76], `Lyra2` [12], `Makwa` [202], and `yescrypt` [177] were the runners up in this competition. `Catena` [76] was developed to address the vulnerabilities of `Scrypt` [200]by implementing password-independent memory access patterns and providing built-in protection against garbage collection attacks in its double-butterfly graph variant.

`Balloon` [28], developed in 2016, offers several key improvements over existing memory-hard functions like `Argon2` [23]. Unlike its predecessors that rely on heuristic security arguments, Balloon provides proven memory-hardness guarantees in the random-oracle model. Its password-independent memory access pattern prevents cache timing side-channel attacks that plague functions like `Scrypt` [200], while maintaining competitive performance. The authors demonstrate Balloon's practical advantages by showing an attack against Argon2i that allows computing it with less space than claimed, while Balloon maintains its security properties even under sophisticated analysis.

The OWASP password storage recommendations [189]

- Use Argon2id with a minimum configuration of 19 MiB of memory, an iteration count of 2, and 1 degree of parallelism.
- If Argon2id is not available, use scrypt with a minimum CPU/memory cost parameter of (2^17), a minimum block size of 8 (1024 bytes), and a parallelization parameter of 1.
- For legacy systems using bcrypt, use a work factor of 10 or more and with a password limit of 72 bytes.
- If FIPS-140 compliance is required, use PBKDF2 with a work factor of 600,000 or more and set with an internal hash function of HMAC-SHA-256.
- Consider using a pepper to provide additional defense in depth (though alone, it provides no additional secure characteristics).

Table 2.3: Table of Identity-based cryptosystems.

Year	Protocol		Type
1984	Shamir	[221]	Signature
1986	Fiat–Shamir	[71]	Identification
1988	Feige–Fiat–Shamir	[70]	Identification
1988	GQ	[94]	Identification
1988	Beth	[21]	Identification
1990	Schnorr	[216]	Identification
1991	Ong–Schnorr	[180]	Signature
1991	Girault	[88]	Identification
1993	Okamoto	[179]	Identification
2002	Choon–Cheon	[41]	Signature
2004	Ateniese–Medeiros	[16]	Hash
2005	Kurosawa–Heng	[125]	Identification
2009	Chin–Heng–Goi	[40]	Identification
2011	Tan–Heng–Phan–Goi	[231]	Identification
2020	Chia–Chin	[39]	Identification

2.9 Identity-based Cryptosystems

Identity-based cryptosystems or protocols are a type of public-key cryptography where a user's public key is derived directly from some unique identifier of the user, such as their email address, domain name, or phone number. They are *challenge-response* identification protocols and are considered to be strong authentication. Challenge-response identification involves a prover (or claimant) A authenticating to a verifier (or challenger) B.

The protocols to be discussed in this section are zero-knowledge protocols. A protocol (or proof system) has the *zero-knowledge property* if it is an *interactive proof system* where the verifier learns nothing beyond the validity of the assertion being proved. These *interactive proof systems* have the following general structure:

1. $A \rightarrow B$: witness
2. $A \leftarrow B$: challenge
3. $A \rightarrow B$: response

Adi Shamir first introduced an identity-based signature scheme in his seminal 1984 paper,

"Identity-based cryptosystems and signature schemes" [221]. This approach remains grounded in public-key cryptosystems but incorporates a distinctive modification. Instead of generating and publishing a traditional public key, users can utilise a set of unique personal identifiers (e.g., name, date of birth, place of birth) as a substitute for the public key.

Shamir only provides a signature implementation scheme for his identity-based system idea. Shamir returns to this idea in subsequent papers in the years following 1984. In 1986, Fiat and Shamir present the first concrete solution [71]. They note that *"The new identification scheme is a combination of zero-knowledge interactive proofs [90] and identity-based schemes [221]."* In this paper [71], they provide both the signature and proof-of-identity scheme that Shamir proposed in his 1984 paper [221]. Moreover, Shamir, along with Feige and Fiat, produces a further study on *"Zero-Knowledge Proofs of Identity"* [70]. This paper discusses and differentiates between zero-knowledge *"proofs of membership"* and *"proofs of knowledge."*

2.9.1 Feige–Fiat–Shamir Identification Protocol

This is the protocol as described in *"Zero Knowledge Proofs of identity"* [70]. We have made some adjustments to the notation to keep it consistent throughout this dissertation. Its security depends on the intractability of computing the square roots mod n.

Assuming the interaction is occurring between two entities, the *prover* A and the *verifier* B.

A's key generation protocol is as follows:

1. Choose k random numbers $s_1, ..., s_k$ in $\mathbb{Z}_n$
2. Choose each v_j (randomly and independently) as $\pm 1 \cdot (s_j^2)^{-1} \mod n$
3. Publish $v_1, ..., v_k$ and keep $s_1, ..., s_k$ secret

The generation and verification process (i.e. interactions) takes place as follows through a four-step process: Repeat t times:

Repeat steps 4 to 7 for $i = 1, \ldots, t$:

4. A picks a random $r_i \in [0, n)$ and sends $x_i = r_i^2 \pmod{n}$ to B.

5. B sends a random binary vector $(e_{i1}, \ldots, e_{ik})$ to A.

6. A sends to B:

$$y_i = r_i \prod_{e_{ij}=1} s_j \pmod{n}.$$

7. B checks that

$$x_i \equiv y_i^2 \prod_{e_{ij}=1} v_j \pmod{n}.$$

In their final remarks [70], Feige, Fiat, and Shamir note: *"An interesting modification can eliminate the public key directory and lead to a key-less identification scheme."* We use the identity string to public-key transformation provided in [70] to create public keys in the system.

There are other protocols similar to the Fiat–Shamir protocol, such as the Guillou–Quisquater (GQ) identification scheme [94], which differs by reducing the number of messages exchanged and the memory requirements for user secrets. The Schnorr identification protocol [216] relies on the intractability of the discrete logarithm problem, unlike the Fiat–Shamir and GQ protocols.

Chapter 3

Empirical Study of Data Breaches and an Anatomy of MitB Malware

We conduct an empirical analysis of data and user secret breaches to understand how secrets are compromised and leaked in the wild. This chapter collects and analyses 60 web security and data breaches from 2020, examining their causes and impact. We establish an evidence-based foundation that informs our subsequent chapters on the use of secrets (such as passwords), client and server-side protections, and vulnerabilities.

3.1 Introduction

Collecting and analysing data breaches enables us to understand the threat vectors and actors, facilitating vicarious learning for other observers, who can then apply these insights to better protect themselves, especially given that organisations are primarily responsible for protecting user secrets. Two main ways of obtaining insights and statistics on data breaches are: 1) when a breach is reported by the victim organisation to a regulatory body, such as the UK's Information Commissioner's Office (ICO); or 2) when breaches are discovered online and subsequently reported by journalists or experts. Self-reported incidents rarely provide detailed accounts of the attack; the most useful information is often gleaned from journalist reports or white-hat hackers. Thus, in this chapter, we will collect and analyse open-source reports and go deeper

than regulatory reports. We will draw insights from case studies and post-mortem reports.

We aim to understand MitB malware in more detail. It has been underreported, especially given that it operates silently and exfiltrates the most valuable user secrets, such as complete banking information. Additionally, some of the most prominent data breaches and cyber incidents have been MitB-based attacks; examples include the British Airways [115] and Ticketmaster case [53]. Section 3.3.2 provides a detailed exposition of MitB malware.

The literature on this topic of understanding breaches in detail is sparse. The work of Saleem *et al.* [211] is most similar to our intent; they conduct a systematic analysis and workflow of ten data breaches between 2013 and 2014. Bilge *et al.* [22] perform a similar systematic analysis but focus on zero-day attacks between 2008 and 2011. Attack2Vec [222] formalises attacks in the CVE type and time—a useful method for understanding the evolution of an attack. Thomas *et al.* [237] study the data and consequences of breaches themselves, including credentials, keyloggers, and phishing kits. Abramova *et al.* [3] analyse a specific breach and subsequently conduct a survey on user behaviour and attitudes towards data breaches.

We will use the cost of data breaches and their impact on businesses to motivate the need to understand and mitigate data breaches. In 2019–2020, UK data controllers reported (via the UK's ICO[1]) a total of 2353 cyber incidents [105], as shown in Figure 3.1. In Q2 of 2020—2021 (July 2020 to October 2020), UK data controllers reported 737 cases [105], which equates to 30% of the previous year's total cases. This increase in attacks is straining organisations, especially those that lack adequate resources to protect themselves. Some businesses believe they do not receive sufficient guidance [83]. These cyber incidents impose significant costs on organisations. Complex regulatory landscapes and emerging attack vectors, such as ransomware, make predicting these costs difficult. IBM reports that the average cost of a data breach is $3.86 million [103], and it takes an average of 280 days to resolve a data breach. There are direct costs, including legal fees and regulatory fines, as well as indirect costs, such as customer loss, reduced productivity, and other opportunity costs. A third of UK businesses lost customers following a security breach [83].

[1]Information Commissioner's Office

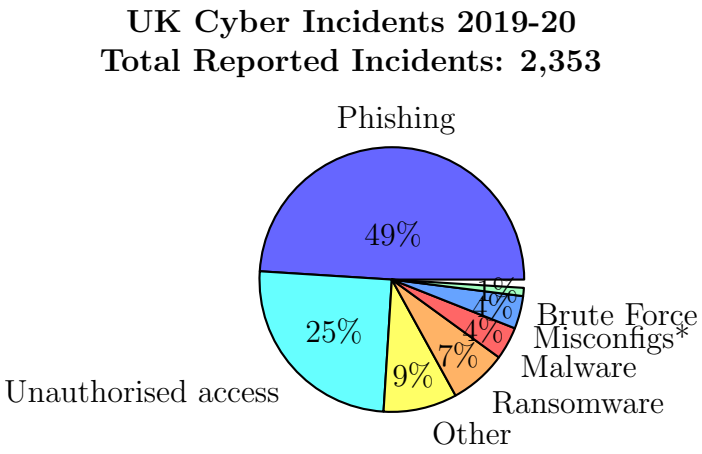


Figure 3.1: Reported cyber incidents to UK's Information Commissioner's Office from April 2019 to April 2020 [105]. These are reported incidents by data controllers. Does not reflect total incidents. Hardware and Software Misconfiguration

3.2 Method

This method facilitates the collection of reported data breaches online and their categorisation. Our goal is to understand the attack vectors and classify them accordingly. We gathered data from open-source news reports, focusing on the attack method, number of records breached, data types, and other relevant metadata.

Attack Classification. We categorised the attacks using the following taxonomy:

- **Man-in-the-Browser (MitB)**: Code injection attacks or malicious third-party scripts executing on the client or browser in order to exfiltrate user data as they are being entered.
- **Phishing**: Social engineering techniques used to trick users into revealing sensitive information or granting access to a system, often through spoofed emails or fake websites.
- **Unprotected Database**: Misconfigured or unsecured databases or storage systems (e.g., AWS S3 buckets) that are publicly accessible without proper access controls.
- **Credential Stuffing Attack (CSA)**: Automated injection of compromised credentials into login forms to gain unauthorised access to user accounts.

- **Unknown**: Incidents where the attack method could not be determined based on available open-source information.
- **Other**: Attacks not fitting into the above categories, such as SQL injection (SQLI), supply chain attacks, or exploitation of insecure code.

Data Categories. We classified the compromised data into the following categories:

- **Personally Identifiable Information (PII)**: Data that can be used to uniquely identify an individual, such as name, address, social security number, or biometric data.
- **Healthcare**: Sensitive medical information, including patient records, medical history, or insurance details.
- **Financial**: Full or partial bank card information, such as credit card numbers, expiration dates, or CVV codes.
- **Authentication**: User credentials, such as email addresses, usernames, or passwords (hashed or plaintext).

3.3 Results and Evaluations

Table 3.1 summarises our results. We analysed sixty attacks that were reported in 2020. We investigated each attack and data breach and came to a conclusion on the cause and type of attack. Our data sources were open-source material. This section follows the table structure in Table 3.1—we discuss the results in each category.

3.3.1 Credential Stuffing Attacks

Credential Stuffing Attack (CSA) is the consequence of password re-use across web applications. The insights here are limited due to the reporting nature of these attacks and most often the

Table 3.1: Analysis of 60 open source reports of data breaches and web attacks categorised based on method of attack.

	Victim	Records	Data	Incident Date From	Incident Date To	News Date	Src
Credential Stuffing Attack						Total: 4	
1	US–**J.CREW**	n/a	PII, $, Sensitive activity	n/a	Apr '19	Mar	[263]
2	US–**Amtrak**	n/a	PII	n/a	Apr	Jun	[141], [183]
3	US–**Spotify**	0.30	PII	n/a	Nov	Nov	[163]
4	US–**Zoom**	0.50	PII	n/a	Apr	Apr	[8]
Man-in-the-Browser						Total: 7	
5	US–**Warner Music Group**	n/a	PII, $	Apr	Aug	Sept	[34]
6	GB–**Sweaty Betty**	n/a	PII, $	Nov '19	Nov '19	Dec '19	[4], [229]
7	US–**Claire's**	n/a	PII, $	Apr	Jun	Jun	[213]
8	US–**Boom! Mobile**	n/a	PII, $	n/a	Oct	Oct	[184]
9	US–**Hanna Andersson**	0.20	PII, $	09 '19	11 '19	Jan	[126], [241]
10	US–**JM Bullion**	n/a	PII	Feb	Jul	Nov	[118]
11	US–**San Francisco Int Airport**	n/a	Auth	n/a	Apr	Apr	[2]
12	**Cardbleed - Int**	n/a	PII, Auth, $	Sept	Sept	Sept	[215]
Phishing						Total: 5	
13	US–**Tufts Health Plan & EyeMed**	0.06	PII	July	Nov	Dec	[244]
14	US–**Twitter**	130 Accounts	Auth, Msg	n/a	Aug	Aug	[234]
15	US–**Imperium Health**	0.14	PII, PHI	Apr	Apr	Sept	[110]
16	US–**Ambry Genetics**	0.23	PII	Jan	Jan	Apr	[24]
17	US–**Beaumont Health**	0.12	PII, Health	May	Jun	Jul	[1]
Ransomware						Total: 3	
18	US–**Magellan Health**	1.70	PII, Auth	Apr	Apr	May	[56]
19	DE–**Fresenius Group**	n/a	System	n/a	May	May	[122]
20	US–**Barnes & Noble**	n/a	PII, Sensitive activity	n/a	Oct	Oct	[5]
Unknown						Total: 10	
21	US–**Drizly**	2.50	PII, $	n/a	Feb	Jul	[262]
22	US–**Quidd**	4.00	PII, Auth	n/a	Oct	Mar	[176]
23	US–**MGM Resorts**	142	PII	n/a	Jul	Jul	[152]
24	JP–**Nintendo - JP**	0.30	PII, $	n/a	Apr	Jun	[132]
25	US–**Dental Care Alliance**	1.00	PII, Sensitive info	Sept	Oct	Dec	[57]
26	US–**Wishbone**	40.00	PII, Auth	Jan	May	May	[44]
27	HR–**Mathway**	25.00	PII, Auth	n/a	Jan	May	[42]
28	US–**Minted**	5.00	PII, Auth	May	May	May	[7]
29	US–**Chowbus**	0.80	PII,	n/a	Oct	Oct	[6]
30	GB–**EasyJet**	9.00	PII, $	April	April	May	[251]
Unsecured Database						Total: 19	
31	US–**Pray.com**	10.00	PII, Sensitive activity	n/a	n/a	Nov	[217]
32	US–**Paay**	2.50	Payment Info	Apr	Apr	Apr	[209]
33	US–**Clubillion**	¿1	PII, Activity	n/a	Apr	Apr	[207]
34	NL–**Pfizer (EMA)**	n/a	Docs	n/a	n/a	Dec	[201]
35	US–**Estee Lauder**	440	PII, Activity	n/a	Jan	Feb	[190]
36	US–**Whisper**	900	PII	2012	Mar	Mar	[187]
37	US–**MyCastingFile.com**	0.26	PII, Sensitive info	n/a	Apr	Jul	[186]
38	US–**PhotoSquared**	0.10	PII, Sensitive info	Nov	Jan	Feb	[167]
39	US–**Ancestry.com**	0.06	PII, Sensitive info	n/a	Jul	Jul	[162]
40	MY–**123RF**	8.30	PII, Auth	n/a	Oct	Nov	[95]
41	US–**Fragomen, Del Rey, Bernsen & Loewy**	–	PII	n/a	n/a	Oct	[78]
42	US–**Key Ring**	14.00	PII	n/a	Jan	Feb	[51]
43	GB–**Virgin Media**	0.90	PII	n/a	Mar	Mar	[48]
44	US–**U.S. Marshals Service**	0.39	PII	n/a	Dec	May	[47]
45	US–**Vertafore**	27.70	PII	Mar	Aug	Nov	[45]
46	US–**CouchSurfing**	17.00	PII	n/a	Jul	Jul	[43]
47	US–**Broadvoice**	350.00	PII, Sensitive Activity, Msg	n/a	Oct	Oct	[33]
48	US–**Avon**	19.00	PII	n/a	Jun	Jul	[166]
49	CN–**Peekaboo Moments**	n/a	PII, Media	n/a	Jul	Jul	[142]
Other						Total: 11	
50	US–**Slickwraps**	0.85	PII, HR, Sys	Feb	Feb	Feb	[137]
51	US–**BlueLeaks (Netsential)**	0.70	PII, $, Activity	n/a	Jun	Jun	[120]
52	US–**Beaumont Health**	0.00	PII, Health	Feb	Oct	Jan	[1]
53	US–**Dickey's BBQ**	3.00	$	May '19	Sep '20	Oct	[121]
54	US–**Health Share of Oregon**	0.65	PII++	n/a	Jan	Feb	[185]
55	US–**Walgreens**	¡10	PII	May	Jun	Jul	[25]
56	ES–**Freepik**	8.30	Auth	n/a	Aug	Aug	[80]
57	US–**Mashable.com**	n/a	PII, HR	n/a	Nov	Nov	[144], [258]
58	US–**Dave Mobile Banking App**	7.50	PII, Auth	n/a	Jun	Jul	[46]
59	US–**General Electric**	0.28	PII	Feb	Feb	Mar	[226]
60	IL–**Promo.com**	22.00	PII	n/a	Jul	Jul	[9]

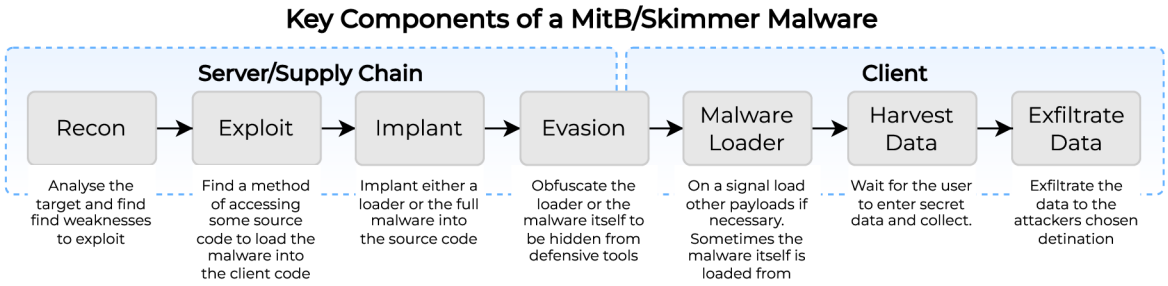


Figure 3.2: Graphical representation of the Man-in-the-Browser (MitB) malware components. Based on the observed MitB-based cyber incidents, we divide the malware into seven ordered components. The taxonomy is based on key components that enable the success or the failure of the malware. For example, when the data is being exfiltrated, the attacker has to overcome Content Security Policy (CSP). Source: Own figure.

organisations themselves were not aware of the attack. We expect CSA attacks to keep growing as more credentials are leaked online.

3.3.2 Man-in-the-Browser

The Man-in-the-Browser (also known as *form grabbers*, *skimmers*, or *magecart-style attacks*) incidents were highly sophisticated and targeted attacks. Our first insight is on the anatomy of a MitB malware: we divide the malware into six key components, as seen in Figure 3.2. We should note here that we are focusing on a variant of a MitB malware that originates from a compromised server. MitB malware can also originate from a compromised third-party script or the user's device itself could be compromised.

The suggested components of a Man-in-the-Browser (MitB) malware can be best explained through the life-cycle of such an attack, as shown in Figure 3.2.The attacker would begin by understanding the target, where the majority of them are e-commerce platforms or applications with a payment gateway. These applications are most valuable because of the frequency at which users enter their banking details (which are typically not stored in full on the server). The attacker would conduct a reconnaissance (recon) to find a route into the server (or to conduct a supply chain attack). E-commerce platforms that have poor admin security are easy targets. Once the server is *exploited*, the attacker would implant their malware or loader into the application. Their choices here will determine how well they would be able to *evade*

discovery. Their choice here also depends on the level of effort put into tailoring their malware for a specific target. Some attackers choose to use a *loader* to evade discovery–this loader has a smaller footprint and is timed to load the malware during checkout when the user is entering sensitive data. Next, the attacker needs to *harvest* the user data; they can do this by creating a fake checkout page or grabbing user data from the HTML input fields. Finally, the attacker would *exfiltrate* the data either through the compromised server or through methods such as the image GET request method.

Implant and Evasion Methods *Implant* refers to where and how a payload is embedded into a target's web application, typically through vulnerabilities in the application's code or configuration. *Evasion* encompasses the techniques used by attackers to avoid detection or prevention mechanisms while carrying out their attacks. Through empirical analysis of real-world attacks, we can better understand the methods devised by attackers and develop more effective defenses for web applications, ultimately improving their security posture.

- **Timed Response:** When an attacker only activates their malware at a specific point in the user's journey on a website, it is said to be *timed*. An example of this is where an attacker used a malicious icon and favicon website to only deliver malicious content on a checkout page [218]. This method helps the attacker evade static analysers or methods that do not take the full user journey into account.
- **EXIF Metadata:** Image headers being used to hide malicious credit card skimming JavaScript code. In one example [220], Malwarebytes Labs show how the copyright field (in the EXIF metadata) was used to transport the obfuscated JavaScript code. This technique allows attackers to bypass content security policies.
- **Image:** There are multiple examples [127], [219] of images being used to hide malicious JavaScript code that is later processed by obfuscated code.
- **CSS File:** Sinegubko [224] demonstrates how attackers conceal malicious JavaScript code within `.css` files. By exploiting CSS properties like `content` and `background-image`,

attackers can encode and embed malicious payloads. These payloads are subsequently decoded and executed by the victim's browser when the CSS file is processed, effectively bypassing traditional security controls that focus on JavaScript files.

- **Web Font:** Attackers can embed malicious JavaScript code inside web font files [143], such as `.woff` or `.ttf`, which are then loaded by the target website. When the browser processes these manipulated font files, the malicious code is executed.

- **HTML5 Local Storage:** Attackers can use HTML5 local storage to persist malicious JavaScript code on the user's browser. This stored code can be later retrieved and executed, even if the user navigates away from the compromised website, allowing the attacker to maintain a persistent presence on the user's system.

- **Service Worker Hijacking:** Service workers, a feature of progressive web apps, can be hijacked by attackers to intercept and modify network requests. By registering a malicious service worker, attackers can steal sensitive information, such as login credentials or credit card details, as they are transmitted between the user's browser and the web server.

- **Polyglot Files:** Attackers can create polyglot files, which are files that are valid in multiple formats. For example, an attacker could create a file that is both a valid image and a valid JavaScript file. When this file is loaded by the browser, the malicious JavaScript code is executed.

Exfiltration: This is the method attackers employ to transmit harvested data to their controlled infrastructure. The process presents several technical challenges, particularly bypassing security mechanisms like Content Security Policy (CSP). A common exfiltration technique involves disguising stolen data as parameters in image GET requests, making the data transfer appear as legitimate image loading. More sophisticated approaches leverage third-party infrastructure and messaging platforms' APIs, such as Telegram or Google App Script, to establish covert communication channels while evading detection. These methods often succeed because they utilize trusted services and legitimate APIs that are typically allowed in security policies.

- **Image GET Request:** Attaching the harvested data as URL parameters in a GET request for an image hosted on the attacker's server.
- **WebSocket Exfiltration:** Establishing a WebSocket connection to the attacker's server and transmitting the data over this channel.
- **DNS Tunneling:** Encoding the data within DNS queries and responses, allowing the data to bypass firewall restrictions.
- **Telegram Bot API:** Leveraging the Telegram Bot API to send the harvested data to a Telegram channel or chat controlled by the attacker.
- **Google App Script:** Utilizing Google App Script to exfiltrate data by sending it to a Google Sheet or other Google service controlled by the attacker.
- **ICMP Tunneling:** Encapsulating the data within ICMP echo request and reply packets, enabling data transmission through firewalls that allow ICMP traffic.

Case Studies: We have a detailed breakdown of the attack on **Claire's** and **Boom! Mobile**. In both of these attack's, the attackers were able to gain full access to the source code on the server and therefore implant their malware. Overview of each of the attacks are as follow.

- **Warner Music:** The server was compromised and malicious JavaScript were injected in order to carry out MitB attack.
- **JM Bullion**. Same as Warner Music. It took five months for the company to discover the breach and a further three months to alert the effected customers.
- **Claire's:** Sansec [214] indicates that the attackers were able to access the store's source code and manipulate any file. How they were able to access the code is yet unknown. However, it is noted that the stores are hosted on the Salesforce Commerce Cloud (previously known as Demandware). The obfuscated malware was added to the `app.min.js` file. It uses `jQuery` functions to attach to the checkout form's button. Once this checkout

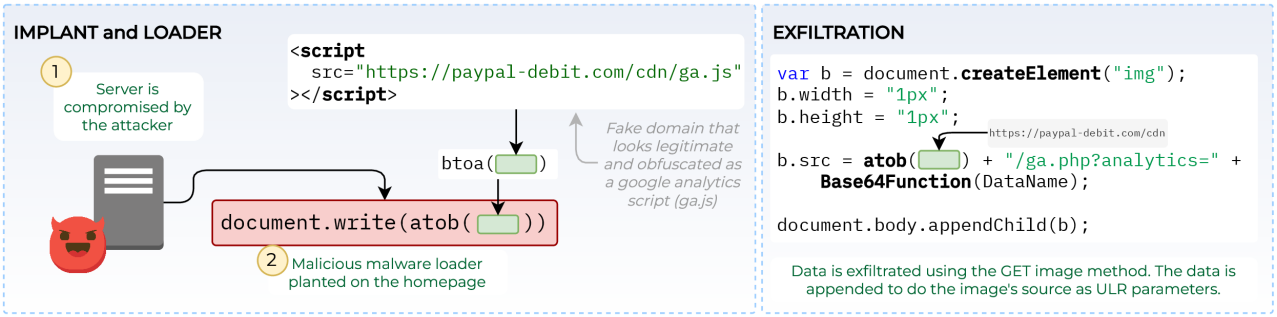


Figure 3.3: Components of the MitB malware used in the Boom! Mobile attack [233]. This is a simple example where the attacker is using a fake domain to load the malware from and exfiltrate user's data to. The malware itself is not obfuscated, instead a one line JavaScript loader is used. Source: Own figure.

button is clicked, an image from the attacker's server is loaded into the DOM, and the customer's information is appended to the image file and exfiltrated via a `GET` request.

- **Sweaty Betty:** A UK-based retailer had its source code infiltrated [4]. The attacker injected their malware into a `custom.js` [229] file. The store was—like Claire's—ran on Salesforce Commerce Cloud (Demandware) platform. The obfuscated JavaScript code communicated with the following URL. `https://www.cdcc02.com/widgets/main.js`.

- **Hanna Andersson:** A children's clothing company in the USA also suffered from a MitB attack; the extent of the attack is unknown but the court-filings suggest approximately 200 000 victims. Sources suggest that the firm was not aware of the attack until law enforcement notified them of the attack; this could be due to victims reporting fraud whom were consequently linked back to Hanna Andersson. The e-commerce platform that was used was operated by Salesforce [212] (as with *Sweaty Betty* and *Claire's*.) The main lesson here is that even if you outsource the IT operations and platform, there are business, legal and PR consequences to web and data breaches.

- **Boom! Mobile:** Malwarebytes Labs report [233] that a criminal related to the *Fullz House* entity had infiltrated the `boom.us` site and implanted a MitB malware. Figure 3.3 shows an overview of some of the components of the malware. The attacker had made no attempt at obfuscating much of the source code apart from encoding the URLs using Base64 encoding.

- **Cardbleed**. Thousands of Magento e-commerce sites that were running an end-of-life version 1 were exploited over few days. The sites then were injected with MitB malware. The malware exfiltrated payment and personal information to a Moscow-hosted site at `https://imags.pw/502.jsp`.

A limitation of our method is that we have only identified a particular set of MitB attacks — attacks that manipulate a site's source code in the server. This has a bigger reach and probablility of success as opposed to malware that infects individual end-user devices.

Opportunity: MitB Defence

Circumventing the threat: Can we circumvent the threats even if a website or device is compromised? FormL3SS (Chapter 7 and Fidelis [67] provide some evidence that we can circumvent threats originating from malicious frontends or malicious devices.

Provenance: Majority of the MitB attacks in Table 3.1 begun with the attackers gaining access to the source code. Therefore, testing the code-base and understanding where the changes are coming from will help mitigate risks. Tracking unwarranted code insertions through the analysis of its provenance will help fight intruders.

3.3.3 Unsecured Databases

An array of unsecured databases and repeated successful attacks imply a poor information security culture. Culture is a set of behaviours, and certain behaviours are necessary to ensure the smooth and secure continuation of any operation. The incidents at Beaumont Health led to attackers gaining access to email accounts, with breaches occurring twice within months. The key takeaway is the need to foster a security-aware culture. Government-backed schemes such as the UK's Cyber Essentials [247] and CISA in the U.S.A. [49] provide guidance and certification programmes aimed at building a cyber-aware culture at both industry and economic levels—this awareness must be communicated down to the lowest employee within an organisation. The SABSA Enterprise Security Architecture [236] serves as an example of a guiding framework for developing security architecture at various levels within an organisation.

Building a culture that is information security aware—from the beginning—is your best chance of avoiding cyber attacks. Based on our results, reasons why we believe having the right culture can help are as follows.

- *Training*. Phishing and poor passwords result from either a lack of discipline or insufficient knowledge. Organisations can communicate expected security standards and provide employees with the necessary knowledge to recognise and question poor information security practices. However, it is equally important to enforce these standards to ensure compliance and maintain a secure environment.
- *Behaviour*. The SolarWinds attacks [73] serve as evidence of poor information security discipline, even when there is awareness of the potential negative consequences of such behaviour. Therefore, fostering a self-reinforcing security culture is essential.
- *Sales and Procurement*. With an information security mindset, your employees will question the security of your potential suppliers.

In the UK, 40% of businesses do not have a response plan to inform customers of a breach [83]. Implementing a response plan for security breaches enables businesses to restore full operational capability more quickly.

Opportunity: Out-of-band Devices and Zero-Storage Policy

We can use out-of-band devices to transact secrets and sensitive information within self-contained and secure containers. Consider payment data: web applications typically store only a subset of payment information to protect users while maintaining application usability. However, we can eliminate the need for this approach and its associated risks by enabling users to transact their secrets using another device, such as a mobile phone. Through out-of-band transactions, we can preserve both user experience and security. Additionally, segregating codebases minimises the impact of breaches—a compromised frontend will no longer pose a direct threat to the user. This approach also eliminates the need for users to repeatedly re-enter their financial details.

3.4 Opportunities

3.4.1 The Data Security Estimate

For leaders of organisations, the security field can be daunting. Whilst they can get help in the form of consultants, the cash strapped startups and small businesses have to get help elsewhere. So, can we create a framework that can guide an individual in analysing and planning the security of their system and data? Such a framework can provide the following benefits through questioning and hand-railing.

- *Risk:* The framework helps to discover and plan for risks. What are the security and data related risks associated to your product, business and technology?
- *Systematic Processes:* We have seen that systems are breached because of human errors; creating new processes can help minimise human errors.
- *Contingency Plans:* Creating what-if scenarios can be helpful in thinking through potential scenarios.
- *Regulations:* What are the data and web security related regulations in my country? And how and when should I interact with them?
- *Response Plans*: Do you have a response plan in place to inform your customers of a possible data breach? What are the processes in plan to deal with a system compromise? How will you ensure business continuity?

3.4.2 PII and Identity

The majority of incidents in our results led to breaches of personally identifiable information (PII). Compromised organisations, such as Hanna Andersson, were forced to provide free credit checks and identity protection for the affected customers. This is because customers are at risk

of identity theft and fraud. A criminal only needs access to basic personal information—such as name, address, and date of birth—to steal an identity.

We have opportunities in *defending* data from being compromised and *defending* systems and organisations against fraud. Should we rely on *secrecy* and *what-you-know* identity checks? Can legal and decentralised national identity systems neutralise this attack vector? Identity checks, or know-your-customer (KYC) processes, slow down customer onboarding. Therefore, future innovation and research should seek a balance between security and business costs.

3.5 Discussion

Have I Been Pwned? [101] popularised the issue of data breaches and raised awareness of PII and credential leaks. Such awareness has most likely improved the security of many user accounts, as users have been able to change passwords when a leak has occurred. This chapter has, in the same vein, attempted to raise awareness of a large number of breaches in order to understand them and offer solutions on how to defend against them in the future.

We used public news reports instead of dark web publications and breaches because not all breaches are reported in time. Moreover, some breaches are not leaked publicly, so they do not have much presence on the dark web. Our aim has been to understand the nature of the breach rather than the leaked data from the breach.

Our results showed that the Magecart group was behind most of the Man-in-the-Browser (MitB) attacks, and ShinyHunters was behind most of the unknown attacks. This over-reporting of ShinyHunters could be because they openly sell their data online. Their odd behaviour of sometimes making the data available for free adds to the mystery and potential for their name appearing in open-source media. Nevertheless, the evidence shows a network of individuals who have the knowledge, capability, and willingness to carry out these attacks.

The results reinforce the necessity of regulatory oversight: from Virgin Media leaving their marketing database exposed to PhotoSquared storing a vast trove of information in an unpro-

tected AWS storage bucket, numerous organisations have failed to secure their databases and storage instances. The complacent behaviour of many organisations, coupled with a clear lack of learning from past incidents, underscores the need for stringent regulatory measures.

Reported security breaches covered by various news sources are mostly high-profile cases. As a result, we acknowledge that our dataset represents only a small portion of actual security breaches, making it impossible to correlate our findings with the total number of web attacks and data breaches. Nevertheless, our findings remain valuable given the sample size and the reported incidents. We have been able to extract key lessons and gather useful data for future research.

The cases we have covered have all been reported through English-speaking media outlets, and all the organisations involved are either European or North American. However, we do not believe that the nature of threats and attack vectors would differ significantly in other countries. Moreover, the reliance on publicly reported incidents creates a significant selection bias towards organisations that either voluntarily disclosed breaches or were legally required to do so, potentially missing a substantial "dark figure" of unreported incidents. Small and medium enterprises may be underrepresented, as they often lack the resources for comprehensive incident response and may avoid public disclosure to protect their reputation.

The categorisation of attack types and attributions depends on the initial news reports, which may later prove inaccurate as investigations develop. The complexity of modern supply chain attacks and the use of shared tools across different criminal groups can lead to misattribution or oversimplified categorisation of incidents.

There is a risk that data collected in 2020 might be irrelevant in 2025. However, we could conduct a historical analysis if we were to repeat the experiment today or in the future. The field of cybersecurity and the nature of threats are rapidly evolving (attack vectors, defence, and regulations). Such rapid change could mean that the results and outcomes will not be relevant to practitioners today. Our view is that the 2020 data provides a baseline for measuring progress (or lack thereof) in cybersecurity practices.

An LLM can be used to provide continual analysis. The data can be used to fine-tune an LLM

to scrape the web, collate data breaches, and conduct analysis on a daily basis. This approach could be used to provide real-time threat intelligence gathering and pattern recognition across multiple languages and regions.

3.6 Summary

With web security and data breaches on the rise, we sought to understand how we could learn from recent security incidents. We analysed 60 web security and data breach incidents from 2020 using open-source reports. Each incident was categorised based on the type of attack, type of data breached, number of records compromised, and other relevant metadata. Using this data, we identified key lessons learned and research opportunities. Our findings justify the necessity of data and privacy policies, despite the pressures and challenges they impose. Organisational leaders must prioritise security to foster a self-reinforcing culture at all levels. Additionally, research opportunities exist in developing a security framework—The Data Security Estimate—that can assist organisations in analysing and planning the information security aspects of their operations.

Form grabbers and Man-in-the-Browser threats are evolving in their delivery, evasion, and exfiltration techniques. This malware has the potential to remain hidden within various file types. There are opportunities to enhance browser security to detect and mitigate the presence of malicious files. Additionally, we identified opportunities for improving the handling of personally identifiable information (PII) through a PII URI scheme and for securing the communication of sensitive information using out-of-band systems. Credential stuffing attacks highlight the inherent weaknesses of password-based authentication. The pursuit of a more secure yet usable alternative authentication scheme to replace passwords remains an ongoing challenge.

Chapter 4

On Password Composition and Complexity

In this chapter, we empirically investigate the composition of passwords. Our aim is to understand why common passwords are weak and how we can strengthen them. The subsequent chapter will focus on how passwords are guessed. While numerous studies have provided high-level analyses of password structure and composition, we take this further by presenting the first human-labelled password dataset. We also address issues in the choice of Password Composition Policies (PCPs) and demonstrate how our dataset aids in creating stronger and more usable passwords—we recommend a five-word PCP.

4.1 Introduction

The lack of consensus on Password Composition Policies (PCPs) and the lack of adoption of best practices on the web [129] suggest that our knowledge of password security is not yet complete. Password Strength Meters (PSMs), such as the `zxcvbn` [261] PSM, which was found to be the best dictionary-based PSM by Wang *et al.* [254] and has 14.7K GitHub stars, still have obvious weak points. For example, `zxcvbn` [261] gives the password `girlsruledaworld` a maximum score of 4 (scores range from 1 to 4). It also assigns a strong score of 3/4 to `asdf1fdsa1`.

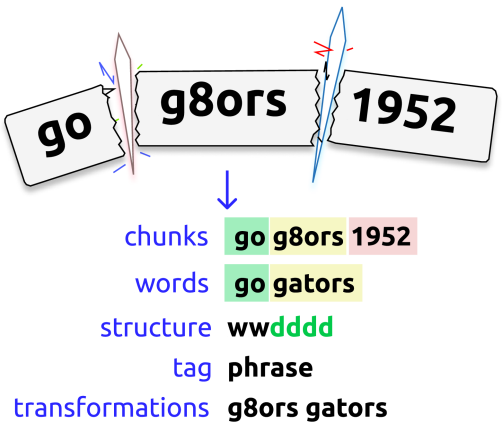


Figure 4.1: Example of the labelling process. The password `gog8ors1952` is sliced into core constituents, then the meaningful words are extracted. Such a labelling process can capture details such as transformations. Source: Own figure.

This is because PSMs such as `zxcvbn` [261] depend on the patterns and structures they are programmed to recognise. This raises the question of what other patterns we are missing. To advance our understanding of passwords, we provide the first human-labelled password dataset.

It is almost common knowledge that we embed personal information in our secrets and that they possess a certain structure. But have we studied and collected most of these patterns and structures? The implications are important, as we saw with the `zxcvbn` [261] example—users can be misled into believing their secrets are strong enough. Other fields benefit from a plethora of labelled datasets that can be analysed; unfortunately, such a dataset does not exist in web security, specifically in password security. The chief task of this chapter is to human-label such a dataset and propose new methods to advance our understanding of password compositions.

We manually label the hotmail (2009) dataset [153] and machine-label the LizardSquad [157] (2015) dataset. We are also in the process of fully labelling the Fortinet (2022) dataset [158]. The dataset itself is the key contribution of this chapter, and we discuss many implications in Section 4.7. In Section 4.4, we provide the results of an analysis on the hotmail dataset and consequently list a more comprehensive set of transformations and structures that can be applied to PSMs such as `zxcvbn` [261].

Understanding password composition is critical in password modelling, as it significantly impacts the precision and efficacy of predictive models. Traditional methods, such as [171], typically analyse passwords at the character level, considering the likelihood of each character's occurrence

in sequence. More sophisticated techniques, like chunking [268], break down passwords into larger, meaningful segments rather than isolated characters, offering a refined perspective on password composition. Nonetheless, these approaches do not provide the detailed insight needed to fully understand nuances such as word transformations, semantic contexts, and structural patterns within passwords.

Data labelling is highly time-consuming and expensive. Thus, we asked ourselves: how can we expedite the labelling process? We can either hire more human agents or develop algorithms and models for assistance. We tested both approaches, employing paid freelancers and utilising language models to aid in the labelling process (see Section 4.6).

Contributions of this chapter:

1. **Human-Labelled Password Corpus.** (Section 4.4) We decompose and label the hotmail 2009 [153] password strings (8292) into chunks, words, structure, tags, and transformations, where applicable. Using the methods developed in Section 4.6, we machine-label the LizardSquad [157] 2015 dataset [157]. To our knowledge, this is the first dataset of its kind. Insights derived from our labelled dataset are discussed, and the dataset is made available for research[1].

2. **Password Composition.** (Section 4.5) Utilising our labelled dataset, we evaluate secure and practical password composition strategies. We find that password structures, such as three-word passphrases without semantic links between words, offer both security and practicality. The use of the hotmail 2009 dataset and the Lizard Squad 2015 dataset provides an opportunity to investigate changes in password composition over time and to reflect on the relevance of these datasets given their age. Using a novel mathematical framework and empirical analysis we recommend a five-word PCP.

3. **Large Language Models and Human Labellers.** (Section 4.6) Labelling data is time-consuming and costly, prompting exploration of alternatives to expedite the process. We engaged freelancers from Fiverr [74] and assessed their work against Large Language

[1] `https://github.com/sirvan3tr/passwordninja`

Models in terms of accuracy, cost, and time. The preliminary results indicate that language models are a promising tool for further experimentation.

4.2 Password Strength and Composition Overview

Password Strength. This measures the difficulty of guessing or cracking a password. However, there is no consensus on how to quantify this difficulty. Some use dictionary-based methods, some use probabilistic methods. Nonetheless, some lower-bound measures are widely accepted, such as the number of characters, to make the password resistant to brute-force.

Shannon's entropy (Eq. 4.1) quantifies the uncertainty or information content associated with a random variable.

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log p(x) \tag{4.1}$$

In the context of password strength, it measures how unpredictable or "random" a password is. The higher the entropy, the more difficult the password is to guess, making it stronger. This metric estimates the minimum number of bits required to encode the password. However, the entropy measure for passwords is often given in the form of Eq. 4.2.

$$E = \log_2(R^L) \tag{4.2}$$

R represents the total number of characters in the character sets used in the password, and L is the length of the password.

The limitation of Eq. 4.2 is that it assumes all R characters are equally likely, disregarding the dependency between characters. This approach oversimplifies, as passwords like `8zm!U6gNL%` and `p4$sW0rd1!` are calculated to have the same entropy. However, such an assumption is unrealistic, as the latter password is essentially the word *password* with predictable transformations and common end-character additions.

Password Composition Policies (PCP). PCP is a set of rules, often expressed in natural

language, that restricts users to selecting passwords from a space assumed to be resistant to guessing attacks. The *National Cyber Security Centre (NCSC)* [172] in the UK and the *National Institute of Standards and Technology (NIST)* [173] in the USA are leading authorities in cybersecurity. Their recommendations, particularly on password policies, should serve as standards or benchmarks in the field. Specifically, the NCSC advocates for the use of three random words [145] and advises against mandatory password complexity requirements [174]. This approach, detailed in [113], is based on the principle that a sequence of three random words is both easier for users to remember and sufficiently complex to deter unauthorised access, striking a balance between security and user convenience.

The random words meet the following conditions:

- **Length**. Likely to satisfy the minimum length requirements.
- **Impact**. Easy to adopt and understand.
- **Novelty**. Unlikely to find it in a breached database.
- **Usability**. More likely to be remembered.

NIST's [92] PCP is as follows. Passwords should have a minimum length of 8 characters, but systems should support more than 64 characters. It is advised to compare the selected password against prior breach data, dictionary entries, and repetitive patterns. Context-specific terms should also be considered. Additionally, users should be aided with strength meters to gauge password robustness. While rate limiting should be enforced during authentication attempts, use of specific composition rules is discouraged.

4.3 Method

This is a method to calculate the pre-image of a password through decomposition. This involves collecting and cleaning leaked password datasets then manually labelling the individual passwords to decompose into chunks, words, structure, and consequently tagging them. Such a detailed decomposition has been lacking in the field, where academics have crudely modelled

password strings based on individual characters and chunks. The newly created labelled dataset then was used to train various machine learning models to do future password decomposition with little human intervention.

Lexical Analysis identifies the basic components, such as letters, numbers, and special characters. **Semantic Decomposition** breaks down the password into meaningful units or words (if any), such as recognising that "cat" and "dog" are two words in the password `catdog123`. **Pattern Recognition** detects common patterns, including repeated characters, sequential numbers, and typical substitutions (e.g., A $\rightarrow$ 4 or E $\rightarrow$ 3). **Structural Analysis** examines the overall composition of the password, such as the sequence of character types (e.g., letters followed by numbers or interspersed special characters). **Transformation Analysis** identifies modifications applied to parts of the password, such as leetspeak substitutions or specific capitalisation patterns.

Data. We began by compiling a list of frequently utilised datasets from prior studies and open-source repositories, such as SecLists [159]. The datasets previously employed are open-source and readily accessible. We have concentrated on datasets predominantly in English, Spanish, and German. We selected the hotmail dataset due to its substantial size of 8929 entries, the phishing method used for its breach, and its bilingual content in English and Spanish. We also selected the LizardSquad [157] dataset because it was compromised in 2005 and thus represents a younger dataset compared to hotmail. Additionally, LizardSquad [157] is relatively small (11 808 records), making it more manageable for labelling. Common datasets such as `Rockyou` [155], `phpbb` [154], and `000webhost` [156] also emerged; however, their vast sizes exceed our capacity for manual labelling given the limited human resources available.

The hotmail dataset, which leaked in 2009 [109], comprises 8 929 records of passwords. Public sources indicate that these passwords were procured through phishing. Abundant evidence within the dataset supports this claim. For instance, we observe numerous passwords with inverted capitalisation and similar passwords that contain typos, suggesting that users might have unintentionally activated `CAPS LOCK`.

Labelling. We segmented the 8 929 leaked passwords based on their structural characteristics.

Numeric-only passwords were excluded. The remainder was categorised into three groups: 1) Passwords following an L_nD_n pattern, consisting of letters followed by digits; 2) Passwords with an L_n structure, including letters and symbols; and 3) Those that did not fit the previous categories, labelled as $!L_nD_n$. We then employed the Markov n-gram entropy method to sort the passwords. This process involved semantically breaking down passwords into identifiable segments, extracting meaningful words, and then labelling the password structure based on these components. Each password received a tag reflective of its primary element—word, name, or phrase—outlined in Table 4.2. Passwords lacking overt semantic content were designated as R2, denoting a human-generated structure that was ambiguous to the labeller. Passwords identified as machine-generated or random received an R1 tag, while those with repetitive structures or keyboard patterns were marked as R3—see Table 4.1.

After labelling the hotmail dataset, we used it to fine-tune the `Mistral-7B-v0.1` model; the dataset was formatted as JSON with system/user/assistant messages (see Listing 1.) The model was fine-tuned using parameter-efficient fine-tuning (PEFT) with QLoRA. The data set LizardSquad [157] was machine-labelled using this fine-tuned model. For inference, the model generated structured outputs with a maximum of 80 new tokens and a repetition penalty of 1.2. Training and inference was conducted using Pytorch and Nvidia GPUs—these were our own hardware (2 × RTX 6000 ada GPUs) and cost was negligible.

```
{
  "messages": [
    {
      "role": "system",
      "content": "Given a password string, decompose the password string and provide the
      ↪  following data in a JSON dict: 'chunks', 'words', 'structure', 'tags',
      ↪  'transformations' (if applicable)"
    },
    {
      "role": "user",
      "content": "interinato"
    },
    {
      "role": "assistant",
      "content": "{\"chunks\": \"interinato\", \"words\": \"interinato\", \"structure\":
      ↪  \"w\", \"tags\": \"word\", \"transformations\": \"\"}"
    }
  ]
}
```

Listing 1: An example of a labelled hotmail record formatted as JSON.

Table 4.1: Labels for passwords that have a degree of randomness to them or the passwords are undecipherable to the human labeller.

Label	Description
R	Random passwords with no apparent structure.
R2	The password has a distinct non-random structure but we cannot decompose it into meaningful components.
R3	The password has a distinct repeating pattern such as `abcd123`

Table 4.2: Keywords used to tag and identify passwords based on their composition.

Serial	Tag	Description
1	word	Standard dictionary word or common.
2	name	Given real or fictional names.
3	phrase	Password is composed of multiple words.
4	location	Places such as towns, cities and countries.
5	date	If majority of the password is made up of a date.
6	org	Organisation
7	object	Specific objects such as car models or artefacts.
8	email	Email address like passwords.
9	website	Entire string composed as a web URL
10	kp	Keyboard pattern, e.g. `qwerty`

Table 4.3: Table showing a small subset of the labelled hotmail dataset. We decomposed the passwords from the hotmail dataset into chunks, words, structure, tags, and transformations (if applicable). Such decomposition allows for better password modelling and nuanced understanding of human behaviour when creating their chosen passwords.

pwd	chunks	words	structure	tags	transform
estatica4	estatica 4	estatica	wd	word	
manterola84	manterola 84	manterola	ndd	name	
elingeniero20	el ingeniero 20	el ingeniero	wwdd	word	
centella2	centella 2	centella	wd	word	
indiferencia8	indiferencia 8	indiferencia	wd	word	
veinti7	veinti 7	veinti	wd	word	
tere50	tere 50	tere	wdd	word	
Bertita1	Bertita 1	bertita	nd	name	
alohanene2	aloha nene 2	aloha nene	nnd	name	
andreateamo3	andrea te amo 3	andrea te amo	nwwd	phrase	
happyness	happyness	happiness	w	word	happiness

Training. In our experiment evaluating the effectiveness of different agents (both human and machine) in labelling password strings, we fine-tuned several Large Language Models (LLMs). The GPT-3 models were fine-tuned via the OpenAI API. The LLama2 and Mistral-7B models underwent fine-tuning with the QLoRa [58] method.

4.4 Analysis of the Labelled Dataset

This section presents an overview of some analysis of the labelled hotmail dataset. The dataset has many potential use-cases (as discussed in Section 4.7.1) First, we draw some insights from this manually labelled dataset.

We manually decomposed and labelled the entire hotmail dataset, comprising 8 929 records. Specifically, we labelled 7 235 records that were not solely numeric. Additionally, we labelled 2 000 records from the phpbb dataset for comparative analysis and further experiments (discussed in subsequent sections). Table 4.3 provides a snapshot of the labelled dataset. Given that we labelled the dataset into *chunks*, *words*, *structure*, *tags* and *transformations*, we will analyse each of these components separately in this section. This section will primary focus on the hotmail

dataset as it is completely labelled and therefore can be used to draw more useful insights.

The time it takes to label a password varies due to their complexity, but we estimate c.170 hours were spent labelling the hotmail and a portion of the phpbb dataset. A single labeller would likely need 35 to 45 days for such a task. In a subsequent experiment, two freelancers took about 5 days each to label 2 000 records.

Words. Here, words refers to a column in our dataset and it contains extracted (meaningful) information from the sliced passwords, such as words, names, locations, organisation names, and etc. If a word is transformed in the original password string then we would transform it back to the standard form, e.g. `g8ors` → *gators*. We observed 5 749 unique words. The frequency of the words in the unique password strings exhibit a power-law like distribution, specifically the Zipf's law. Figure 4.3a shows that the log-log plot of word frequency and rank is linear: the frequency of the n^{th} ranked word $f_n \propto n^{-\alpha}$ [116]. This builds on the hypothesis that raw passwords (not unique) in a given dataset also exhibit the Zipf's law [252]. This has implications for how we model passwords.

Structure. Structure of the password indicates the order and position of different types of characters, digits and words. Table 4.4 shows the most common structures for the hotmail dataset. Majority of the passwords are composed of words or combination of words (phrases). The most common structure is a w (word), which is prone to simple dictionary attacks.

Our method of labelling the structures is certainly up for debate, however, they are flexible and can be converted. For example, for the password `mialiasesANGELO1` has the structure of `wwnd`. This structure can be converted to one presented in the `PCFG` [260] model where the authors use L_n, D_n and S_n to represent alpha, digit and special variables (n represents the count of the specific variable). We can extend these to include W_n and N_n for **word** and **name** variables. So, for example, password (structure: `wwndd`) can be represented as $W_2N_1D_1$. So, one may be able to carry out a `PCFG` [260] style attack with the knowledge of these structures.

The distribution of the structures exhibit a power law–similar to Zipf's law if we plot the frequency of the structures against their sorted Rank (as shown in Figure 4.3b). This can

support our thesis that the core of the password is a word or a set of words (a phrase). A question for future work is whether other datasets follow a similar power law as seen in the hotmail dataset.

Tags. Our analysis reveals that most passwords are made up of words and names. 33% of the total 7 235 passwords were names, 32% words, 15% phrases, 13% random, 4% location, 2% organisation names, and 1% dates. These are followed by phrases and passwords classified as Random, which we further divided into R1, R2, and R3 categories. Among the 7 235 records analysed, 946 were labelled as R2, indicating a substantial proportion of passwords appear random but are vulnerable to brute-force attacks. The distribution of the remaining random passwords is as follows: 23 classified as R3 and only one as R1. Additionally, 35 passwords were identified as keyboard patterns.

Transformations. We observed 324 transformations in the hotmail dataset. Though the partial labelling of the phpbb dataset revealed more intriguing word transformations. Identifying the exact nature of a transformation can be challenging, especially for non-native speakers of the language used in the passwords, leading to less than 100% confidence in our identifications. Transformations with nearly zero confidence were labelled as *R2*.

Contextual information can also reveal more information about the transformation, for example: the string `01well` can simply be *zero*, *one* and the word *well*, but if we add `9801well` or `198401well` then it becomes apparent that it is *orwell* (referencing the book *Nineteen Eighty-Four* [182] by George Orwell). This example is a clear illustration of why word search and algorithmic means of labelling passwords are difficult. Such nuanced patterns make labelling difficult for a human who cannot make the connection between *84* and *01well* or simply doesn't have the knowledge of that specific book and author. Though the transformation is complex to the human, it still has a d^4w structure. This means that it has a predictable structure that can be guessed using a mask attack.

Transformations can also encompass multiple words, such as mingling them or altering their order. Such transformations are less frequent than those at the word level. Hence, we categorise these as *phrase-level* transformations. Examples of transformations are as follows.

Phrase level transformations

- **Hybridisation:**
  - **Overlapping letters**: Such as `r3al0v3` → `real love`, where numbers are used to represent letters that look similar, overlapping in meaning.
  - **Other** More complex mixing examples: `castilleo` → `leo castillo`: A rearrangement of the letters to form the correct name.
- **Change of order.** Changing the order of the words or chunks in the password. `pedro te amo` → `te amo pedro`.
- **Repetition.** e.g. `mememememe`

Word level transformations

- **Homophones.** These are words that sound similar but have different meaning, e.g. `2` → `to`
- **1337 speak** is the replacement of words with numbers and letters to create alternative spellings that resemble leet or 'elite' speak. This replacement would either make the word visually look or sound similar. Examples are shown in the text below.
  - **Visual.** `9801well` → `1984 Orwell`: Numbers are used to resemble the visual appearance of the corresponding letters.
  - **Sound.** `articul8` → `articulate`: The number '8' sounds like 'ate', making the word sound like 'articulate'.
  - **Sound.** `2rus` → `trust`: The number '2' represents 'to' and the reversal of 'rus' to 'sur' makes the word 'trust'.
  - **Sound.** `pahswrd` → `password`: Phonetic spelling that omits certain letters but still resembles the sound of 'password'.
- **Slang.** Shorthand or informal language.
  - `gurlz` → `girls`
  - `luvme` → `love me`: 'luv' is a common slang for 'love'.
  - `skure` → `secure`: Phonetic spelling of 'secure'.
- **Elision.** Deletion of one or more sounds in the word. `nbtwin` → `inbetween`: Shorthand for 'in between'.
- **Abbreviation.** `atl` → `Atlanta`.
- **Reversal.** Flipping the letters around, as in `drowssap` → `password`.
- **Syllable swap.** Mixing up the order of syllables or parts of the word, like `wordpass` or `word$pass`.
- **+/− characters.** Omitting one or more characters from a word, such as `bloo` → `bloom` or `asasin` → `assassin`.
- **Elongation.** Extending one or more characters in a word, like `psssss` or `sammmmm`, often to convey a drawn-out pronunciation.
- **Misspelling**: Intentional or accidental incorrect spelling of words.
- **Doubling letters.** Repeating one or more letters, as in `biiaankaa` → `bianka`.
- **Random *L* or *D* insertion.** Inserting letters at random points, such as `cancero1s` → `cancerous` or `plu01to` → `pluto`. Often they occur between syllables in a word.

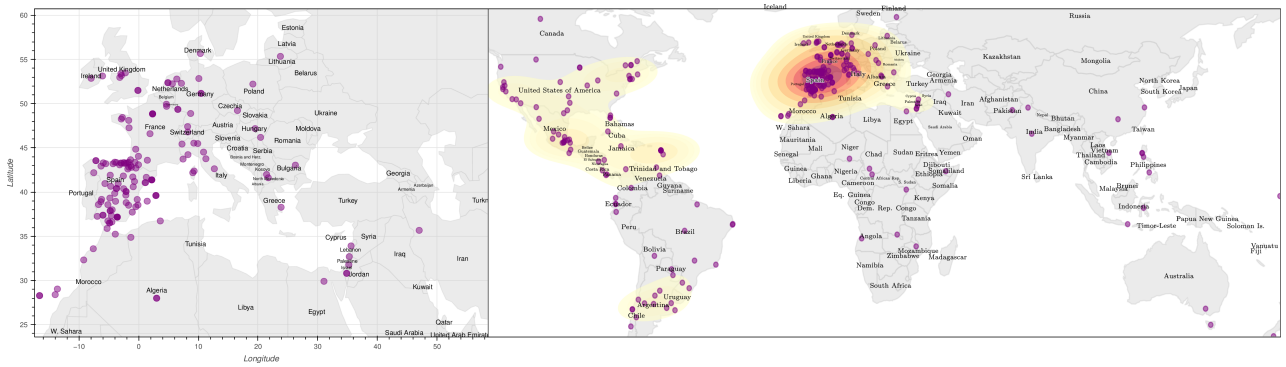


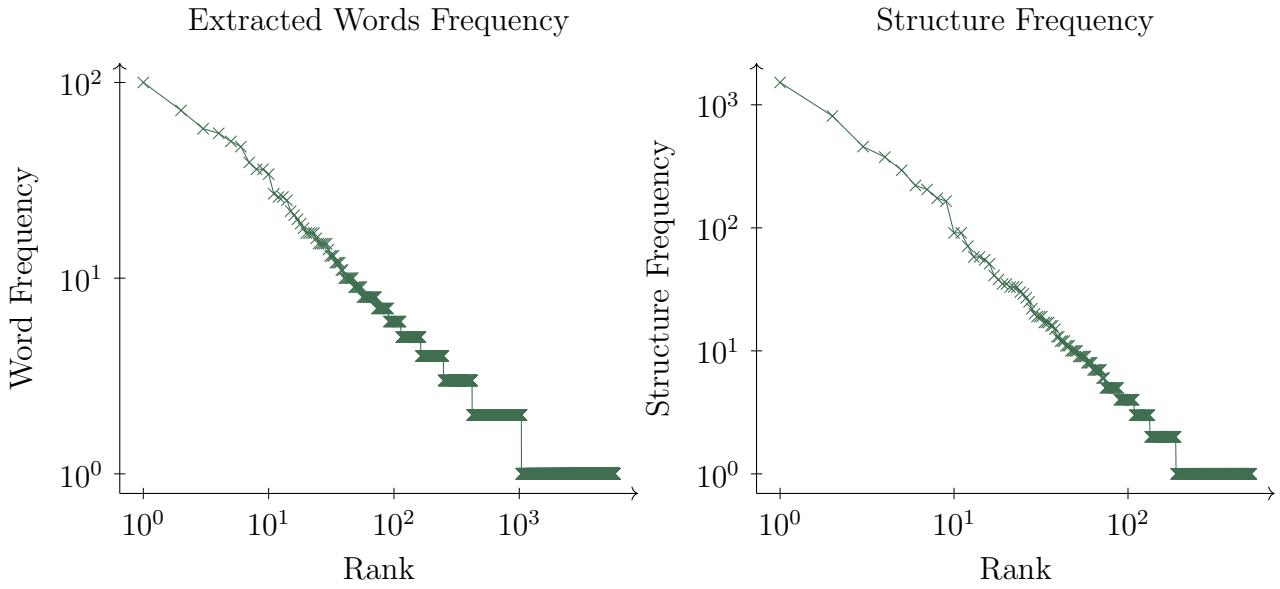
Figure 4.2: Map showing passwords that were tagged as location in the labelled hotmail dataset. Location names were extracted from passwords, geolocated and plotted. Southern Europe, especially Spain, is more dense than other regions. This evidence and the prominent language in the hotmail dataset indicates that the passwords came from Spanish speaking regions.
Source: Own figure.

Location. We also embed our knowledge of geography and our surroundings into our passwords. We find that the extracted locations are correlated with the language of the users. This indicates that users are more likely to use locations that they are familiar with or are close to. Figure 4.2 shows the extracted locations on a map. Such a location-based analysis can be beneficial for post-mortem investigations in a data breach and leak. It can help identify the threat vector and the intent of the attacker.

Age. The age of the hotmail dataset, 2009, raises the question whether it is still relevant. One can assume that password policies and other unknown factors would have changed our behaviour and therefore the lessons-learnt from the hotmail dataset would not be significant. The machine-labelled LizardSquad [157] as shown in Table 4.5 indicates that age of the dataset is less of a factor as we still notice similar patterns in password structures. Password composition policies however do change this pattern but not the underlying human behaviour of embedding information about ourselves and surrounding into these secrets. Moreover, we can still learn from older datasets and one can compare them to a newer labelled datasets in future studies.

4.5 Password Composition Policy

There is a lack of consensus on password composition policy amongst industry experts, academics and government agencies. This has created some confusion which is evident by the fact that most web applications do not follow any best practices [11], [84], [85], [129], [133], [136], [140],



(a) Unique Words (b) Structures

Figure 4.3: Charts showing the log-log plot of word and structure against their respective rank for the hotmail dataset. Words are from extracted unique passwords. The distribution of both words and structure exhibit a power law similar to Zipf's law.

[267]. This raises the question which PCP is the *right* one? Specifically, should we have a structure-based policy as recommended by UK's NCSC?

To answer this question we pay particular attention to the NCSC's PCP of three random words. We first develop a mathematical model that incorporates password structures, this is novel and enabled by the passwordNinja dataset.

4.5.1 Empirical Evaluation

To analyse how the password structure affects security, we draw on the data set passwordNinja as described in the previous section. This dataset provides something previously unavailable to researchers: individually labelled passwords that reveal their structural composition. This labelling is essential for our analysis, as it allows us to systematically evaluate how different password structures influence the security strength.

Our analysis of password datasets reveals several key vulnerabilities in how users choose passwords. First, users select words from a highly biased distribution, making passwords more

Table 4.4: The table presents the top 25 labelled password structures from the passwordNinja [10] dataset, accounting for 5 032 (69.5%) of the labelled dataset. Hashcat [228] benchmarks indicate the performance of consumer hardware in a hybrid attack. The dictionary size for $\boldsymbol{w}$ (words) and $\boldsymbol{n}$ (numbers) is set at 5×10^5. `zxcvbn` [261] serves as a password strength meter, with scores ranging from 0 to 4, and guess estimates are log based.

	Frequency		Hashcat using Nvidia RTX 4090				`zxcvbn` Mean	
Structure *log* **perm'**	**Count**	**%**	**MD5**	**PBKDF2-sha256**	**scrypt**	**bcrypt-sha512**	**Score**	**Guesses**
w	1520	21.0	~ 0	0.1 s	1 min	4 min	1.3	5.36
n	811	11.2	~ 0	0.1 s	1 min	4 min	1.0	4.72
nd	58	0.8	~ 0	0.6 s	12 min	42 min	1.4	5.68
wd	91	1.3	~ 0	0.6 s	12 min	42 min	1.5	5.97
nl	33	0.5	~ 0	1.5 s	30 min	2 h	1.2	5.09
wdd	294	4.1	~ 0	5.6 s	2 h	7 h	1.8	6.69
ndd	221	3.1	~ 0	5.6 s	2 h	7 h	1.6	6.33
nddd	71	1.0	~ 0	56.4 s	19 h	3 day	1.7	6.37
wddd	91	1.3	~ 0	56.4 s	19 h	3 day	1.9	6.84
wsdd	29	0.4	~ 0	3 min	3 day	9 day	2.0	7.28
ddddw	35	0.5	~ 0	9 min	8 day	29 day	2.4	7.95
wdddd	174	2.4	~ 0	9 min	8 day	29 day	2.3	7.59
ndddd	205	2.8	~ 0	9 min	8 day	29 day	2.0	6.90
nn	376	5.2	1.5 s	8 h	1 yr	4 yr	2.0	6.91
ww	457	6.3	1.5 s	8 h	1 yr	4 yr	2.0	7.04
nw	30	0.4	1.5 s	8 h	1 yr	4 yr	2.2	7.51
nddddd	51	0.7	3.0 s	16 h	2 yr	8 yr	2.5	8.05
wdddddd	33	0.5	3.0 s	16 h	2 yr	8 yr	2.8	8.84
wwdd	58	0.8	3 min	33 day	111 yr	402 yr	2.9	8.97
nndd	38	0.5	3 min	33 day	111 yr	402 yr	3.2	9.62
wwdddd	33	0.5	4 h	9 yr	1.11×10^4 yr	4.02×10^4 yr	3.3	9.81
nwn	41	0.6	9 day	447 yr	5.56×10^5 yr	2.01×10^6 yr	3.0	8.88
www	165	2.3	9 day	447 yr	5.56×10^5 yr	2.01×10^6 yr	3.0	9.26
nww	35	0.5	9 day	447 yr	5.56×10^5 yr	2.01×10^6 yr	2.7	8.18
nnn	27	0.4	9 day	447 yr	5.56×10^5 yr	2.01×10^6 yr	3.3	9.95
wwww	55	0.8	171 day	8.70×10^3 yr	1.08×10^7 yr	3.91×10^7 yr	3.5	11.24

predictable (Figure 4.3a). Second, commonly used password structures contain patterns that attackers can exploit using techniques such as Probabilistic Context-free Grammar (`PCFG` [260]) or mask attack mode in Hashcat [228].

As shown in Table 4.4, single words (w) and names (n) dominate password structures, making them vulnerable to dictionary attacks. The effectiveness of these attacks varies by hash function; our tests using consumer hardware demonstrate significantly different cracking times across hash functions. A newer dataset (LizardSquad [157] 2015) shows similar structural patterns (More detail in Table 4.5.)

Tables 4.4 and 4.5 use data from labelled passwords and were generated as follows. The datasets were labelled with the respective *Structure* to indicate their structural composition. We then filtered the top 25 most frequent structures and sorted them by dictionary-based permutations. The Hashcat-based attack shows, if we were to perform a dictionary attack and assume they were hashed using the given cryptographic hash functions, how long it would take to crack them using an Nvidia RTX 4090 GPU. Lastly, the `zxcvbn` [261] library for Python was used to

Table 4.5: The table presents the top 25 labelled password structures from the LizardSquad [157] dataset. Hashcat [228] benchmarks indicate the performance of consumer hardware in a hybrid attack. The dictionary size for $\boldsymbol{w}$ (words) and $\boldsymbol{n}$ (numbers) is set at 5×10^5. `zxcvbn` [261] serves as a password strength meter, with scores ranging from 0 to 4, and guess estimates are log based.

	Frequency		Hashcat using Nvidia RTX 4090				`zxcvbn` Mean	
Structure *log* **perm'**	**Count**	**%**	**MD5**	**PBKDF2-sha256**	**scrypt**	**bcrypt-sha512**	**Score**	**Guesses**
w	594	5.5	~ 0	0.1 s	1 min	4 min	0.9	4.24
n	189	1.7	~ 0	0.1 s	1 min	4 min	0.8	4.13
wd	254	2.3	~ 0	0.6 s	12 min	42 min	1.2	5.06
nd	135	1.2	~ 0	0.6 s	12 min	42 min	1.5	5.71
wdd	667	6.1	~ 0	5.6 s	2 h	7 h	1.4	5.64
ndd	457	4.2	~ 0	5.6 s	2 h	7 h	1.4	5.85
wddd	736	6.8	~ 0	56.4 s	19 h	3 day	1.6	6.10
nddd	409	3.8	~ 0	56.4 s	19 h	3 day	1.6	6.04
wdddd	465	4.3	~ 0	9 min	8 day	29 day	1.7	6.41
ndddd	359	3.3	~ 0	9 min	8 day	29 day	1.7	6.18
nddddd	58	0.5	0.3 s	2 h	81 day	293 day	1.8	6.51
wddddd	84	0.8	0.3 s	2 h	81 day	293 day	2.0	6.89
nn	139	1.3	1.5 s	8 h	1 yr	4 yr	1.6	6.03
nw	26	0.2	1.5 s	8 h	1 yr	4 yr	1.8	6.42
ww	666	6.1	1.5 s	8 h	1 yr	4 yr	1.5	5.84
ndddddd	51	0.5	3.0 s	16 h	2 yr	8 yr	2.3	7.24
wddddd	52	0.5	3.0 s	16 h	2 yr	8 yr	1.9	6.92
nnd	73	0.7	15.2 s	3 day	11 yr	40 yr	2.4	7.82
wwd	231	2.1	15.2 s	3 day	11 yr	40 yr	2.1	7.18
wwdd	307	2.8	3 min	33 day	111 yr	402 yr	2.4	7.61
nndd	86	0.8	3 min	33 day	111 yr	402 yr	2.8	8.64
wwddd	233	2.1	25 min	326 day	1.11×10^3 yr	4.02×10^3 yr	2.5	7.93
nnddd	48	0.4	25 min	326 day	1.11×10^3 yr	4.02×10^3 yr	2.9	8.96
nndddd	28	0.3	4 h	9 yr	1.11×10^4 yr	4.02×10^4 yr	3.1	9.54
wwdddd	76	0.7	4 h	9 yr	1.11×10^4 yr	4.02×10^4 yr	2.8	8.39
www	210	1.9	9 day	447 yr	5.56×10^5 yr	2.01×10^6 yr	2.3	7.33
wwwd	42	0.4	88 day	4.47×10^3 yr	5.56×10^6 yr	2.01×10^7 yr	2.3	7.45
wwwdd	42	0.4	2 yr	4.47×10^4 yr	5.56×10^7 yr	2.01×10^8 yr	3.1	9.43
wwwddd	39	0.4	24 yr	4.47×10^5 yr	5.56×10^8 yr	2.01×10^9 yr	2.9	9.02
wwww	54	0.5	1.21×10^4 yr	2.24×10^8 yr	2.78×10^{11} yr	1.00×10^{12} yr	3.1	9.77

calculate the average score and number of guesses for each structure in the table. This table and view would not be possible without labelling each password with their respective structure.

However, passwords containing multiple words (such as w^4 structures) demonstrate much stronger security, resisting hybrid attacks even when hashed with MD5. Yet this strength depends critically on word choice. When users select related words or popular phrases (such as song lyrics), passwords become more predictable. For example, the phrase `"mira que eres canalla"` is vulnerable, even though it is made up of four words. This is because it originates from Mexican folk music.

To maximise security, users should select words with minimal semantic or probabilistic relationships in their sequence. This prevents attackers from using language models to predict subsequent words in multi-word passwords or passphrases.

Our analysis of the passwordNinja datasets supports the NCSC guidelines. Testing with

Table 4.6: Top 10 password structures for each zxcvbn [261] score of 3 an 4. These are the password structures that are categorised in the most secure bin (4) by the zxcvbn algorithm; Avg Guesses is the zxcvbn guesses_log10.

Structure		Count	Avg Guesses	Avg Length
zxcvbn score = 4				
r2	$r2$	160	12.6	14
www	w^3	57	12.1	14
ww	w^2	48	12.0	14
wwww	w^4	34	13.1	15
w	w	25	11.3	12
nn	n^2	24	11.2	13
wdddd	wd^4	19	11.6	12
wwdddd	w^2d^4	16	11.3	14
nndd	n^2d^2	15	11.5	13
ndddd	nd^4	14	10.8	12
zxcvbn score = 3				
r2	$r2$	185	8.9	10
w	w	100	8.8	10
ww	w^2	86	8.8	11
nn	n^2	77	8.8	11
www	w^3	62	9.1	11
wdddd	wd^4	44	8.8	11
ndddd	nd^4	43	8.7	10
wdd	wd^2	38	9.0	10
wwdd	w^2d^2	31	9.1	11
n	n	26	8.9	10

zxcvbn [261] (Figure 4.5 and Table 4.6) confirms that multi-word passwords improve the security of human-chosen and memorable passwords.

4.5.2 Theoretical Evaluation

Password Structure Probability

Let S denote the structure of a password, where $S(p) = (e_1, e_2, \ldots, e_n)$ and each $e_i \in \Sigma$ (e.g., $\Sigma = \{w, n, d, s, \ldots\}$). The probability of a structure S is given by $P(S) = P(e_1, e_2, \ldots, e_n)$. Assuming dependence between the elements in the structure, this can be expressed as:

$$P(S) = P(e_1) \prod_{i=2}^{n} P(e_i \mid e_{i-1}) \tag{4.3}$$

Combined Relationship Probability

If e_i represents the type of the element, then let $\hat{e_i}$ represent the instance of the element, e.g. $\hat{e} \rightarrow$ *password* if $e = w$ (i.e. a word). We can represent both structure and instance relationships using a joint distribution. For a password p with elements $(e_1, ..., e_n)$, we can define:

$$P(p) = P(S(p),\, E(p)) \tag{4.4}$$

$$= P(S(p))\, P(E(p) \mid S(p)) \tag{4.5}$$

Where $E(p)$ represents the specific element instances.

$$P(E(p) \mid S(p)) = P(\hat{e}_1) \prod_{i=2}^{n} P(\hat{e}_i \mid \hat{e}_{i-1}, S(p)) \tag{4.6}$$

The final equation simplified is:

$$P(p) = P(e_1)P(\hat{e}_1) \prod_{i=2}^{n} [P(e_i \mid e_{i-1})P(\hat{e}_i \mid \hat{e}_{i-1}, S(p))] \tag{4.7}$$

Incorporating NCSC's Recommendation

Aligning with the NCSC's recommendation of using at least three random words, we define a specific structure S^* where $S^* = (w,\ w,\ w)$. We assume a fixed structure of just three words, thus $P(S^*) = 1$. Using Eq. 4.5, we are left with $P(p \mid S^*) = P(E(p) \mid S(p))$. We also assume that the words are chosen independently. Thus, we have:

$$P(p \mid S^*) = \prod_{i=1}^{3} P(w_i) \tag{4.8}$$

4.5.3 Non-uniform NCSC PCP

In practice humans do not select words at random and the passwordNinja [10] dataset shows that the selection of words follows a power-law like distribution. This has significant implications for

the NCSC's recommendation of using at least three random words for password composition. Below is a mathematical framework to model this scenario and assess the impact on the password composition policy.

Word Frequency Distribution

Let w denote a word from the vocabulary $\mathcal{W}$. Under Zipf's law, the probability $P(w)$ of selecting the r-th most frequent word is given by:

$$P(w_r) = \frac{1/r^s}{\sum_{k=1}^{|\mathcal{W}|} 1/k^s} \tag{4.9}$$

where s is the exponent characterizing the distribution (typically $s \approx 1$ for natural languages).

For a password p consisting of n words $(w_1, w_2, \ldots, w_n)$ selected independently according to Zipf's law, the structure probability $P(S)$ where $S = (w, w, \ldots, w)$ is:

$$P(S) = \prod_{i=1}^{n} P(w_i) = \prod_{i=1}^{n} \frac{1/r_i^s}{\sum_{k=1}^{|\mathcal{W}|} 1/k^s} \tag{4.10}$$

where r_i is the rank of the i-th word.

Entropy of the Password Distribution under Zipf's Law

The entropy H of the password distribution. Substituting $P(p)$ with the structure probability:

$$H = -\sum_{w_i \in \mathcal{W}}^{n} \left(\prod_{i=1}^{n} \frac{1/r_i^s}{\sum_{k=1}^{|\mathcal{W}|} 1/k^s} \right) \log \left(\prod_{i=1}^{n} \frac{1/r_i^s}{\sum_{k=1}^{|\mathcal{W}|} 1/k^s} \right) \tag{4.11}$$

Simplifying the logarithm:

$$H = -\sum_{i=1}^{n} \sum_{w_i \in \mathcal{W}} P(w_i) \log P(w_i) = n \cdot H_{\text{word}} \tag{4.12}$$

where H_{word} is the entropy per word. We will now use this to draw a comparison with the uniform distribution, which we take to be the theoretical version of the NCSC's PCP.

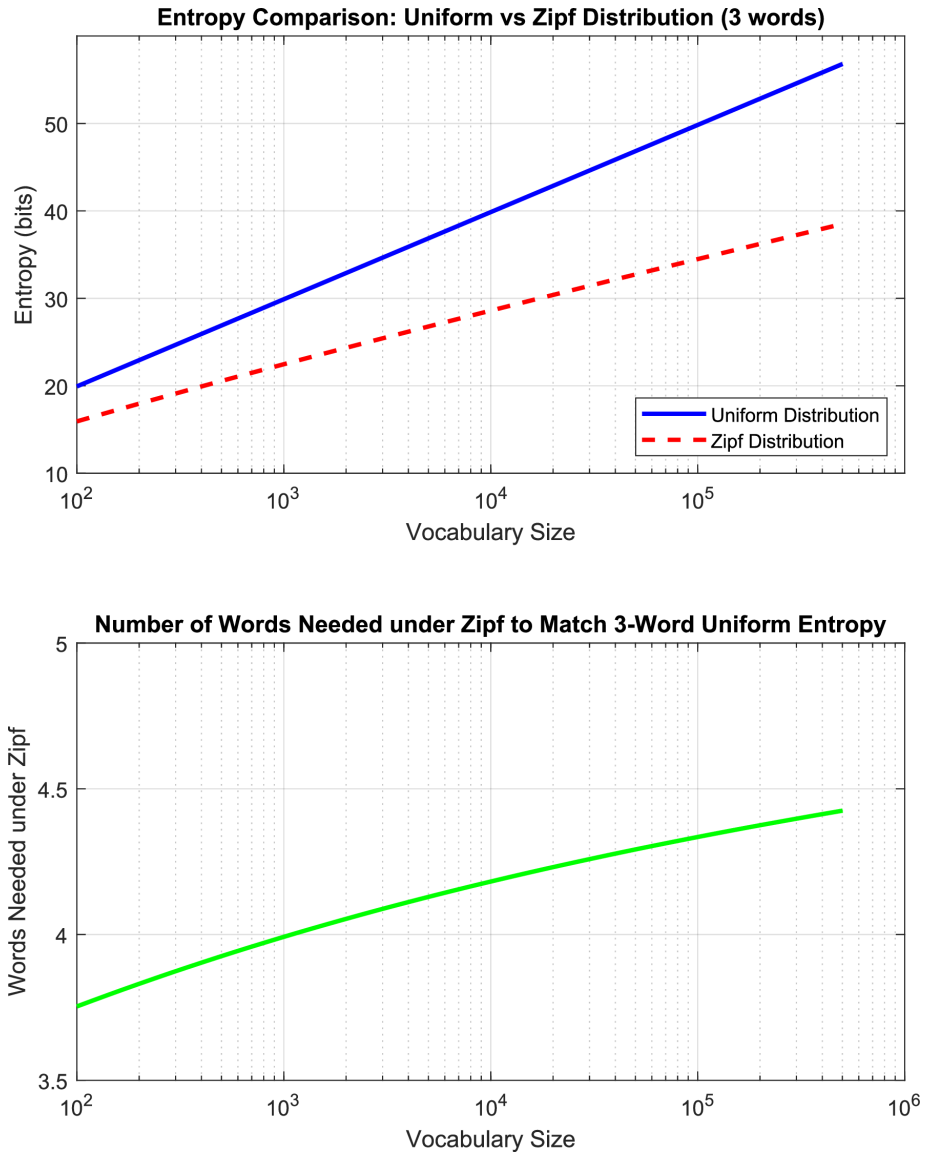


Figure 4.4: Chart showing the equivalent number of words required in practice to match the theoretical complexity of the National Cyber Security Centre's (NCSC) password composition policy. The NCSC recommends at least three random words, using our model we show that in practice a minium of 4 words is more secure. Source: Own figure.

Impact on NCSC's Recommendation

To align the password policy with enhanced entropy under Zipf's law, we need to determine the minimum number of words n required to achieve a desired entropy H_{desired}. This can be formulated as:

$$n \geq \frac{H_{\text{desired}}}{H_{\text{word}}}$$

Given that H_{word} is lower under Zipf's law compared to a uniform distribution (due to the presence of highly probable words), a larger n may be required to achieve the same entropy.

Under a uniform distribution where each word has $P(w) = \frac{1}{|\mathcal{W}|}$, the entropy per word is:

$$H_{\text{word}}^{\text{uniform}} = \log |\mathcal{W}|$$

Comparing with Zipf's law:

$$H_{\text{word}}^{\text{Zipf}} < H_{\text{word}}^{\text{uniform}}$$

Zipf's law requires more words than a uniform distribution to achieve the same entropy level. As Figure 4.4 demonstrates, when we increase the vocabulary size, the entropy difference between uniform and Zipf's distributions also increases. This comparison leads us to conclude that we need at least 4-5 words to match the entropy that NCSC's theoretical PCP describes. Moreover, this minimum word requirement grows as the vocabulary expands.

To maintain or exceed the entropy level recommended by NCSC using three random words under a Zipfian distribution, the policy should be adjusted to:

$$n_{\text{adjusted}} \geq \frac{3 \cdot \log |\mathcal{W}|}{H_{\text{word}}^{\text{Zipf}}}$$

This ensures that the password composition maintains sufficient entropy despite the skewed word frequency distribution.

Adjusted Password Structure Probability

If we adjust the structure to use n words to counteract the lowered entropy per word, the structure probability becomes:

$$P(S_n) = \prod_{i=1}^{n} P(w_i) = \left(\frac{1}{\sum_{k=1}^{|\mathcal{W}|} 1/k^s} \right)^n \prod_{i=1}^{n} \frac{1}{r_i^s} \tag{4.13}$$

Ensuring that n is sufficiently large to maintain high entropy despite the non-uniform word distribution.

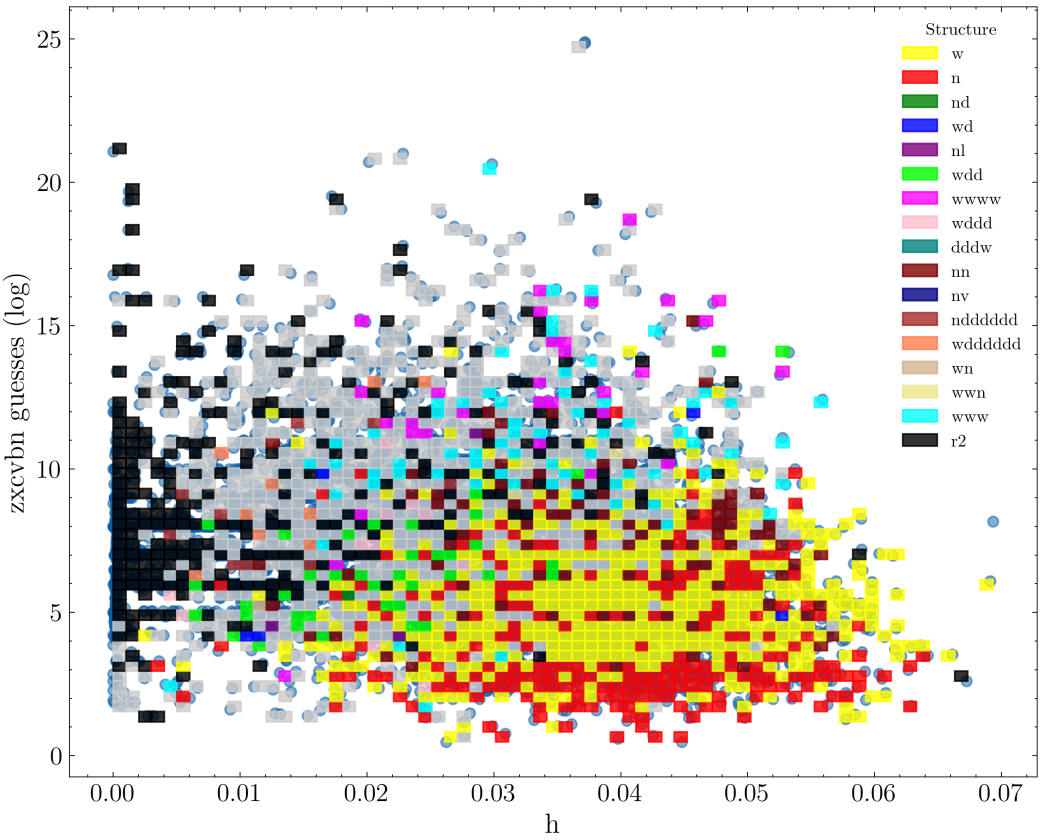


Figure 4.5: Chart showing the labelled hotmail passwords and their respective structure. Individual grids represent the most frequent structure. On the y axis we have the `zxcvbn` [261] `guesses_log10` metric and on the x axis we have the h = Markov 2-gram scores. As expected, the weakest passwords ($\boldsymbol{w}$: yellow and $\boldsymbol{n}$: red) have a high h score and are predicted to be weak by `zxcvbn`. Source: Own figure.

4.6 Experiment: Human and LLMs as Labellers

Data labelling, particularly for password strings, is a labour intensive task. To expedite this process, we explored the effectiveness and cost-effectiveness of human versus machine labelling. We recruited freelancers from Fiverr [74] to represent the human element. On the machine side, we employed several Large Language Models (LLMs), including GPT-3.5-turbo [181], LLaMA2-13B [240], and 70B, along with the Mistral-7B [107]. This approach allowed us to compare the precision, efficiency, and financial implications of utilising human labour against AI technologies in the task of labelling complex datasets like passwords.

Several researchers have explored the application of Large Language Models (LLMs) for data labelling [134], [161], [192] with positive results. Notably, this report [135] cites the accuracy

and significant speed up from the language models compared to human labellers.

4.6.1 Freelance Labelling

We tested the utility of freelancers from Fiverr [74] for password labelling. We took two samples of 1000 passwords from the phpbb dataset; we selected passwords that were of length 6 and greater, $L >= 6$, and non-numeric but contained at least one digit. We hired two separate freelancers for $50 each. The freelancers were proficient in the machine learning data labelling process but were non-native English speakers. They were selected randomly from the responses we received from a posted advert. We would categorise them as non-experts in this domain. We used Google Sheet to share the passwords and monitor the labelling progress. This method proved beneficial as we observed interesting behaviours. We took a hands on approach first by correcting the freelancers at the beginning by checking the first 50 labelled passwords. We had to do a lot of correction.

Interestingly, the second human agent enlisted additional helpers to expedite the labelling process, resulting in errors and inconsistency despite clear instructions. This was evident in the labelling document, where some labellers resorted to copying and pasting from search engine results, prioritising speed over accuracy. In contrast, the first agent focused on accuracy, seeking detailed feedback from us, but exceeded their five-day deadline and showed inconsistency in labelling. Accounting for the hands-on help, the accuracy of the extracted words was $50\% \pm 10\%$; this characteristic was the more accurate compared to chunking, deciphering the structure and tagging the passwords. Ultimately, the output from both agents proved unreliable.

The brief to the freelancers

You're given a table of 1 000 passwords on Google Sheet and your aim is to label them with the following features:

Chunks: Separate passwords into clear segments based on character-type changes and meaningful boundaries, e.g., `test4me` → *test, 4, me*.

Words: Extract meaningful words, including transformations back to standard form, e.g. `test4me` → *test, for, me* or `g8ors` → *gators*. Include names, locations, organisations, and recognisable terms.

Structure: Map chunks using the following codes: **w** (word), **n** (name), **d** (digit), **s** (special character), **l** (letter). Examples: `test4me` = *wdw*; `manterola84` = *ndd*; `elingeniero20` = *wwdd*.

Tags: Classify the passwords into the following categories.

- **word** (standard dictionary word)
- **name** (given/fictional names)
- **phrase** (multiple words)
- **location** (places, cities, countries)
- **date** (majority date content)
- **org** (organisation names)
- **object** (specific items like car models)
- **email/website** (email-like or URL patterns)
- **kp** (keyboard patterns like qwerty)
- **R1** (random/machine-generated)
- **R2** (human-generated but ambiguous structure)
- **R3** (repetitive patterns like abcd123)
- **Transformations:** Document any modifications like leetspeak (*4*→*for, 3*→*e*), phonetic changes (*luv*→*love*), reversals (*drowssap*→*password*), elongations (*psssss*), or character substitutions (*g8ors*→*gators*).

You can label the first 50 passwords and then review them and give feedback. You already have c.30 diverse, labelled examples to get a good sense of our intent here.

4.6.2 Language Models

Can language models automate the labelling process? Language models, such as the GPT family of LLMs from OpenAI, have generalised well across various tasks, including coding. Given their large vocabularies, we believe they could be useful for automating the labelling process. In this section, we describe how we fine-tuned closed- and open-source models using supervised fine-tuning, with successful results in labelling password datasets.

We fine-tuned a closed-source model (GPT-3.5-turbo) and open-source models that were downloaded from huggingface.co. For GPT-3.5-turbo, we used OpenAI's standard fine-tuning API by uploading a training file containing password-label pairs in JSON format (input: raw password; output: structured decomposition with chunks, words, structure, tags, transformations), while for the open-source models (LLaMA 2 and Mistral) we applied QLoRA (Quantized Low-Rank Adaptation), which freezes some of the model's weights and trains only small adapter layers to reduce computational requirements. The open-source models were fine-tuned using transformers[2] and PEFT[3]. The evaluation process involved two experimental phases. First, three models (GPT-3.5-turbo, LLaMA2-13B, and Mistral-7B) were fine-tuned on 825 manually labelled passwords from the phpbb dataset and tested on a separate 300-password subset from the same dataset to measure the accuracy of chunking passwords into components and extracting meaningful words. In the second phase, the training set was expanded to 1 725 passwords, combining both the phpbb and hotmail datasets, with the models tested on 3 062 passwords from the hotmail dataset to evaluate cross-dataset generalisation.

Part 1. We started by fine-tuning the **GPT-3.5-turbo-0613** model with 825 manually labelled passwords, aiming for JSON-formatted outputs (System prompt: *Given a password string, provide the following fields in a JSON dict: 'chunks', 'words', 'structure', 'tags', 'transformation' (if applicable).*) This fine-tuning was carried out using OpenAI's Python API. The fine-tuning cost was ∼$5 and was completed in a few hours. Testing involved evaluating this model

[2]https://github.com/huggingface/transformers
[3]https://github.com/huggingface/peft

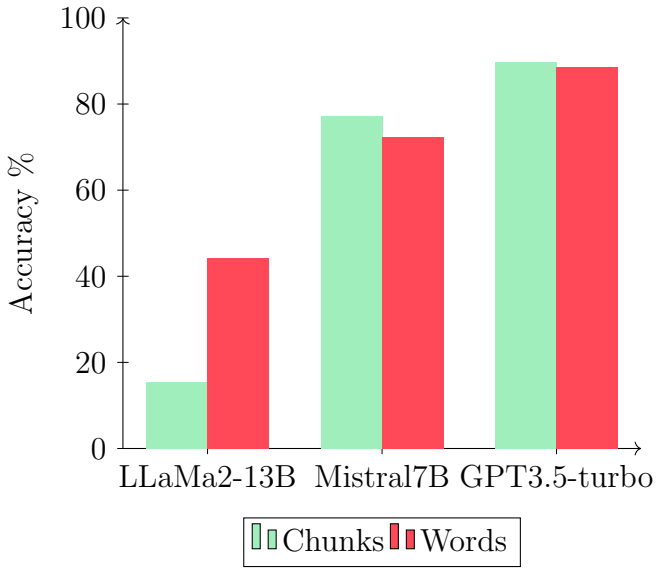


Figure 4.6: Accuracy of three models in chunking and extracting words from a password string. The models are fine-tuned using 825 passwords from the phpbb dataset, and are tested on a separate 300 passwords from the same dataset.

against 300 passwords from the same phpbb distribution, distinct from the training set. This initial fine-tuning, despite using limited data, showed success; however, 41 of the 300 (13.7%) passwords required relabelling. The model struggled with transformations, especially elongated passwords such as `sssssue`, `6etter`, and `vic20oria`; the latter we assume to be `"victoria"`. Tokenisation is an important aspect of language models. We believe mislabelling could stem from tokenisation as well. Conducting further tests on the tokenisation method was beyond the scope of this chapter.

Encouraged by promising initial results, we conducted further experiments with additional open-source models. We started with smaller models, LLaMA2-13B and Mistral7B, and fine-tuned them using LoRa [100] and QLoRa [59] techniques. The outcomes of the initial experiment, using an 825-labelled training set and a 300-item test set from phpbb, are presented in Figure 4.6.

Part 2. In the second phase of our experiment, we augmented our training set by adding 900 passwords from the hotmail dataset. Building on the initial work with GPT-3.5 and Mistral, we expanded our test set to include 3 062 labelled hotmail passwords. These passwords consisted primarily of characters (without digits) and were not found in dictionaries, indicating they were either phrases or obscure words. We replaced the LLaMa-13B model with a larger LLaMa2

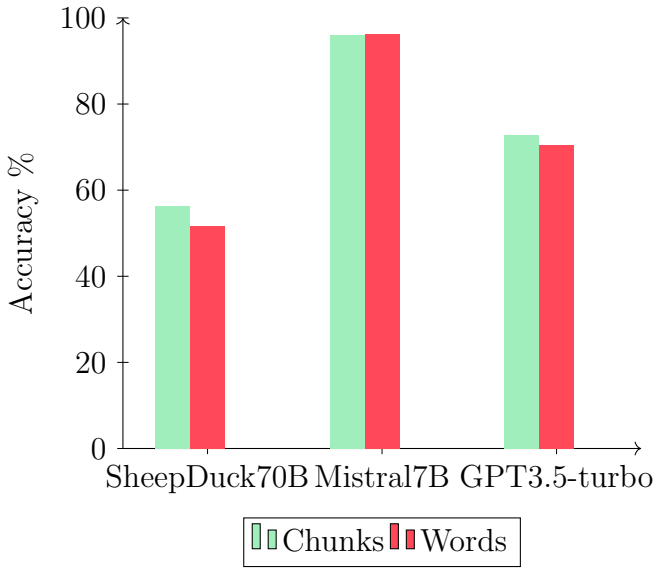


Figure 4.7: Chart showing the accuracy of three models in chunking and extracting words from a password string. The models are fine-tuned using 1 725 manually labelled passwords from the phpbb and hotmail dataset. Test data is 3062 passwords from the hotmail dataset.

variant, specifically using the *Riiid/sheep-duck-llama-2*[4] [66], [128], [165] model, which benefits from training on a more extensive dataset compared to the standard 70B parameter model.

4.6.3 Evaluation

The models do not generalise well with different structures from ones trained on, e.g. `rosa38` and `38rosa` have the reverse grammatical structures. The labelling of the *structure* were predominantly correct except for the digit counts. The models also found reversing the structure difficult. This $A \rightarrow B$ and $B \rightarrow A$ phenomena in LLM is well documented [20].

The model does not generalise well against languages. We tested the hotmail passwords that contained Spanish words with models that were trained on the phpbb data and often the models attempted to translate words back into English.

All the models produced well structured JSON data regardless of the amount of data it was fine-tuned with.

The results, as shown in Figures 4.6 and 4.7, were obtained using limited data; therefore,

[4] `https://huggingface.co/Riiid/sheep-duck-llama-2-70b-v1.1`

their applicability to a larger or an entirely different dataset may be limited. Our testing and experimentation were limited, and it is possible that the models have overfitted to a limited number of password structures. Lastly, we did not look deeply into why a 70B-parameter model performed worse than a 7B-parameter model (Figure 4.7)—these are good questions that can be answered in future work.

Our initial experiment on using language models to label passwords were successful enough to warrant future research in our opinion. Further experimentation of language model is beyond the scope of this chapter and there is significant research avenues to explore. Conducting more experiments with different prompting techniques, synthetic data, more human labelled datasets, and different models can prove beneficial. Moreover, using agentic or reasoning methods with the LLM might increase accuracy.

4.7 Discussion

In this chapter, we presented a large-scale, human-labelled password dataset, which, to the best of our knowledge, is the first of its kind. We then demonstrated its utility by using it to empirically derive new facts and insights regarding password composition and strength. Additionally, we introduced novel techniques for expediting the labelling process and demonstrated how Large Language Models (LLMs) can be employed to label passwords. The rest of this section discusses our key insights, the significance of the dataset, and the limitations of our study.

The method used to capture a dataset introduces a bias. Often, leaked datasets consist of hashes that have been cracked by various entities and subsequently redistributed. Passwords that remain uncracked are either omitted or included only in their hashed form. This omission, along with the presence of uncracked passwords, can distort the analysis. The value of the hotmail dataset lies in its acquisition through a phishing attack, allowing us to assume that it includes passwords of all complexities. In contrast, datasets derived solely from obtained hashes are likely to lack structures considered difficult to crack, such as sequences like *www*. The hotmail dataset does not exhibit signs of strict password policies, such as requirements for

special characters, digits, or mixed character cases.

4.7.1 Impact

Password Modelling: One of the key implications of this dataset is its potential to improve password modelling. By analysing the detailed breakdown of password structures, words, tags, and transformations, researchers can develop more accurate model passwords. This can help in assessing the strength of existing password policies and identifying potential vulnerabilities in authentication systems. Moreover, the insights gained from the dataset can guide the development of new password guessing tools and techniques. Password guessing strategies are useful because they allow us to discover new vulnerabilities in our passwords and thus use that knowledge to develop better password composition policies and strength meters.

Supervised Learning: This labelled dataset has the potential to be used for supervised learning techniques. It can also be used for evaluation techniques of other ML techniques.

Approximate String Matching: Design and evaluation of new string matching algorithms. The problem of password similarity is closely related to string matching. However, measuring password similarity is more difficult than measuring normal strings. The text transformations, phonetic changes, and concatenation of other words and symbols increase the difficulty. Password similarity algorithms are important and have numerous use cases. In password strength meters, they help determine how closely a user's password resembles existing passwords that may be deemed weak. If we are developing an algorithm to generate passwords, a similarity algorithm may be required to measure the difference between the generated password and the true password. A password similarity algorithm operates under the assumption that we have a model of the password.

Password Strength Meters: The labelled dataset can serve as a benchmark for evaluating the performance of password strength meters and other tools that assess password quality. By comparing the predictions of these tools against the actual composition patterns and structures present in the dataset, researchers can identify limitations and areas for improvement in existing

password strength estimation techniques.

Dictionary-based PSMs, such as the `zxcvbn` [261] PSM, rely on finding patterns in a password in order to determine the strength of the password. This means it needs to decompose the password and understand the transformations if there are any. Missed patterns could mean that a password's strength is incorrectly measured. In fact, `zxcvbn` [261] does miss some patterns as we shall in more detail below. Patterns such as `asdf1fdsa1` [197] get a high strength score of 3/4 from `zxcvbn` [261]. To answer the question, the transformations discovered in Section 4.4 can be implemented into PSMs such as `zxcvbn` [261] to give a more accurate measure of strength.

Synthetic Data Generation: With the ethical issues raised against using real datasets, one can attempt to generate synthetic data that closely match real passwords. You can also create synthetic passwords based on other languages but preserving the structures.

Natural Language Processing (NLP): The dataset contains slang words and other transformations that can be used for natural language modelling. At least it can inform the modelling process. It can also be used for specific tasks such as Named Entity Recognition (NER) in datasets where there are spelling mistakes or accidental concatenation of words.

Education: By understanding common password composition patterns and the prevalence of certain structures and transformations, we can design targeted educational campaigns and materials that address specific weaknesses and promote best practices for creating strong passwords. Insights from the dataset help craft more effective risk communication strategies and guide users toward adopting more secure password habits.

4.7.2 Ethical Considerations

We have used real passwords that belong to individuals that were phished and consequently tricked into revealing their passwords. This raises few ethical issues: whether this in depth analysis will hurt those users? will it reveal any other secrets? will it identify any individuals? This dataset dates back to 2009 and therefore it is highly unlikely that the same users would have kept their passwords the same even if the identity of the individual could be revealed.

Identities are highly unlikely to be revealed through our data. We have not used any PII or any other information that could link back to individuals. This dataset has also been previously used in other research papers.

This new dataset and repository of the data have been created solely for academic research purposes. The data, sourced from publicly accessible repositories like SecLists [159], does not reveal any novel information. Our analysis also does not expose any new details about the original users associated with these passwords. Importantly, this dataset is devoid of any personally identifiable information (PII).

We acknowledge that the users whose passwords were compromised never consented to their passwords being used for academic research. This creates a power imbalance. Whilst we cannot obtain consent from the affected individuals (which would be impractical), we aim to use this research to improve cyber security practices that protect future users.

The use of breached and leaked password datasets is well established in published research, with some studies even incorporating PII such as email addresses and dates of birth. As such, this work aligns with existing practices in the field and does not necessitate additional scrutiny. Analysing leaked datasets is crucial for advancing our understanding of how human-chosen secrets are employed, ultimately enabling us to enhance their resilience against malicious actors. Lastly, we believe the benefits outweigh the risks given that malicious actors actively use leaked password datasets. This almost creates an arm race between researchers and malicious actors.

4.7.3 Limitations

There is a portion of the dataset that is labelled as R2 that could be studied further. Some of these passwords were difficult to label. The presence of human errors and inconsistencies in the labelling process is a notable limitation of our work. It is reasonable to assume that human-labelled datasets will contain some inaccuracies. Nevertheless, these errors are generally negligible. To mitigate these issues, we propose the development of an online dashboard that allows users to continuously identify and correct any errors.

Labelling passwords in non-native languages presents additional inconsistencies for human agents,

primarily due to limited vocabulary and unfamiliarity with linguistic nuances. Implementing an online portal where native speakers can rectify these errors would be advantageous over time. Such a system could also facilitate the creation of an extensive phonetic dictionary, encompassing words and names that are typically challenging to locate in centralised repositories.

A notable limitation of this study is its applicability to datasets in languages other than English or Spanish, such as Chinese. While Chinese datasets and literature are extensively researched, direct model transfer is not feasible. However, the methodology and insights into password patterns may be adaptable. Investigating how cultural differences manifest in password choices, including variations in structural patterns, vocabulary categories, and word types, could yield significant insights. Given that literature and art reflect societal and cultural contexts, passwords may similarly serve as indicators within the same domain.

Some passwords are constructed using abbreviations, codes, or acronyms that are comprehensible only to their creators. Decomposing these into meaningful components is often impractical, warranting their categorisation under the "R2" label. However, these passwords are typically short and susceptible to basic brute-force attacks, diminishing the security of their cryptic elements.

We did not include numeric only passwords. Numeric only passwords are also guessable and have clear structures; however, in this Chapter we wanted to focus on the larger portions of passwords that are composed with words and letters. This limited the complexity of our experiment given it was the first ever kind of experiment that attempted to label passwords at such a scale.

4.8 Summary

In this chapter, we present the first large-scale, human-labelled password dataset, comprising over 8 000 passwords from the complete hotmail dataset, a selected subset of the phpbb dataset, and machine-labelled entries from the LizardSquad [157] (2015) dataset. These datasets encompass a range of password complexity elements, including chunks, words, structures, tags, and transformations. The applications of this dataset are numerous, spanning supervised learning, education, and password modelling.

We have developed a rigorous model that incorporates password structure and empirical user behaviour patterns, filling a critical gap in our understanding of password security. Through analysis of real-world password datasets, we demonstrate that structure-based password policy such as three-words (w, w, w) empirically and theoretically offer security advantages. However, the NCSC's previously recommended three-word password policy, while theoretically sound, fail to account for users' non-uniform word selection which follows a Zipf-like distribution. Our framework provides both theoretical foundations and practical guidelines for developing more robust authentication policies that reflect actual user behaviour rather than theoretical ideals.

We recommend extending the NCSC's password composition policy to require five words, which our models show provides comparable security to the theoretical three-word ideal even under real-world usage patterns. This recommendation maintains usability while providing demonstrably stronger protection.

Additionally, we investigated methods to streamline the time-consuming and costly process of password labelling. We employed freelancers and compared their performance with that of language models such as GPT-3.5-turbo, Mistral7B, LLaMa2, and their variations. Freelancers took approximately seven days to label 1000 passwords each, with results that were sometimes inconsistent. In contrast, language models proved to be fast, adaptable, and accurate.

Chapter 5

Password Guessing Algorithms

In the previous chapter, we performed an analysis of password composition, which reinforced *why* passwords are guessable. We also demonstrated how secure, yet memorable, passwords can be created. This chapter will focus on *how* passwords are guessed by examining the techniques and algorithms used. We aim to determine whether the guessing techniques described in the literature are practical and whether they add any value to password security. We provide the first systematic overview of techniques and guessing algorithms used in both industry and academia.

5.1 Introduction

Although significant research has focused on the composition and creation of passwords, the techniques used to guess or crack these passwords are equally important for understanding password security. However, we rarely see academic password guessing algorithms deployed and used in practice[1], suggesting problems with existing approaches. So, we hypothesise that academic password guessing methods might be misaligned with practical requirements and therefore require a new form of benchmark. We investigate this in Section 5.3, where we combine the quality of the candidate passwords generated with the generation speed.

[1]We have only encountered `Markov` n`-gram` [171] and `PCFG` [260] used in the wild.

Preimage Attack

Password guessing can be formalized as a preimage search problem: given a hash value y, find an input x such that $h(x) = y$, where h is a cryptographic hash function and x is the target password.

Problems with Existing Approaches. A trivial solution for password guessing is an exhaustive search (brute force), but this is computationally infeasible for even moderate password lengths. More sophisticated approaches leverage patterns in human-chosen passwords to reduce the search space. However, existing techniques suffer from several drawbacks:

- Academic password guessing algorithms often prioritise candidate quality over generation speed, making them impractical for real-world applications where speed is critical.
- Many proposed techniques lack reproducibility and proper experimental controls, making it difficult to fairly compare different approaches.
- There is a significant disconnect between academic research and industry practice, with theoretical advances rarely translating to practical improvements.

Our aim is to systematically analyse password guessing algorithms in order to: 1) Provide a comprehensive overview and system for existing techniques and algorithms. 2) Establish a model to benchmark methods and algorithms that takes into account the quality and speed of candidate generation.

In this chapter, we provide a systematisation of knowledge (SoK) of password guessing algorithms that bridges the gap between academic research and practical application. Our work provides:

- A unified framework for understanding and categorising password guessing algorithms.
- A framework that combines generation quality with generation speed to better measure the effective performance of password guessing algorithms.
- A detailed overview of *Mask Attack* and their performance using the passwordNinja data set created in Chapter 4.

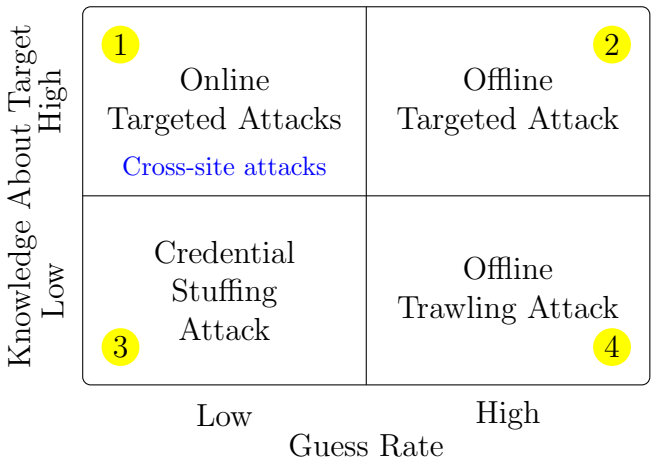


Figure 5.1: Categories of attacks based on the attack rate (available to the attacker) on the horizontal axis and the amount of knowledge the attacker has about the target on the vertical axis. High attack rate signifies that the attacker has access to the target's hash. A low attack rate means that the attacker is trying to attack a hash through a rate limited service.

5.2 Core Components

5.2.1 Taxonomy of Password Guessing Strategies

At a high level, we categorise password guessing attack strategies based on the guess rate and knowledge of the target. The guess rate is determined by whether the attacker has the hash value or not, which also determines whether the attack is offline or online. *An offline attack* assumes unlimited guessing attempts, while an online attack is typically rate-limited by the target application (we assume we do not possess the hash value). The second factor is whether the attack is targeted, assuming knowledge about the target (e.g., personal information and previous passwords). Figure 5.1 illustrates these factors and their relationships.

- **Offline Attacks.** In situations where adversaries have access to password hash values, they are able to make unlimited attempts to guess the passwords without restriction.
- **Online Attacks.** Conversely, guessing attempts are restricted by security protocols implemented within the target applications.

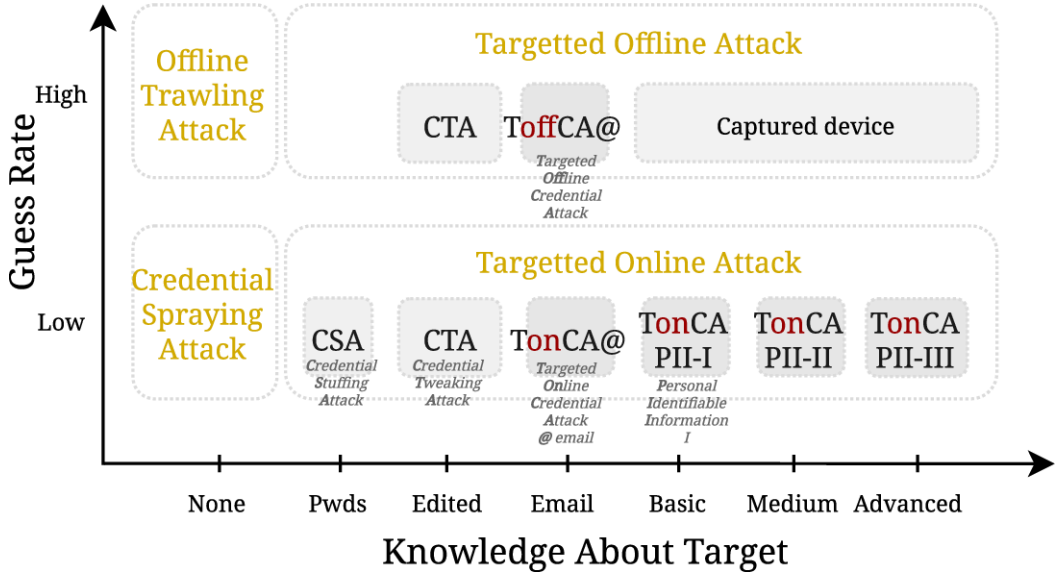


Figure 5.2: Detailed representation of algorithms conditioned on guess rate and the type of information is used for targeting.

The contextual information used for targeting can be further broken down into categories that have been inspired by previous studies [191], [255], [256], [257]. These studies have shown that using contextual information (such as previous passwords or PII) improves the likelihood of guessing human-chosen and memorable secrets. Figure 5.2 visualises the detailed breakdown of contextual information with respect to the guess rate. This is summarised in Conjecture 5.2.1—Our analysis of the content of the password, especially the location information, showed how users embed information about themselves and environment into their secrets.

Conjecture 5.2.1 (PII and Knowledge Embedding in Secrets) *When a person creates a secret (such as a password), they naturally embed information from their personal context into it. This embedding follows predictable patterns based on different categories of contextual information, including personal, environmental, temporal, social, and behavioural information. The likelihood of finding patterns derived from any particular type of contextual information in the secret increases with both the psychological proximity that the information has to the person and the ease with which it makes the secret memorable. This embedding occurs at a rate significantly higher than what would be expected by chance, with the strength of embedding varying based on how cognitively accessible and personally relevant the contextual information is to the creator.*

Research Opportunity

Given a limited set of strategically chosen questions about a user and their truthful answers, it is possible to generate a relatively small set of candidate passwords that contains the user's actual password with probability significantly higher than random guessing. This is achievable if and only if the questions are specifically designed to reveal information about how the user incorporates personal information into their passwords and their password creation habits. The resulting set of candidate passwords will be substantially smaller than the set of all possible passwords of the same length and character composition.

Conditioning a guess on assumptions about the content or structure of the password also affects guessing performance. Techniques like masking have proven highly useful in practice.

Some literature may categorise their method as a *Cross-site attack*; in the context of their use, this means that the attack is targeted and uses breached passwords from other websites (hence the name cross-site). However, this strategy is often known as *Credential Stuffing Attack*. If further targeting occurs, i.e., editing the password, then we call this *Credential Tweaking Attack*.

Beyond credential tweaking attack, one may use more Personally Identifiable Information (PII) such as email address, date of birth, family information, and etc. As shown in Figure 5.2, we refer to these as **T**argeted **on**line **CA** (Credential Attack).

These high level strategies aim to capture the environmental limitations. There are further limitations imposed within each category, e.g. a memory-hard hash function might demand a different strategy to a simpler hash function. Also, these strategies thus far have not captured the *mode* of the attack; this is the aim of the next section.

5.2.2 Generation Modes

Password guessing algorithms are essentially generative functions. These algorithms can be classified into three main categories: probabilistic, deep learning-based, and deterministic (or rule-based) approaches [242], [274]. However, this classification is overly simplistic for practical

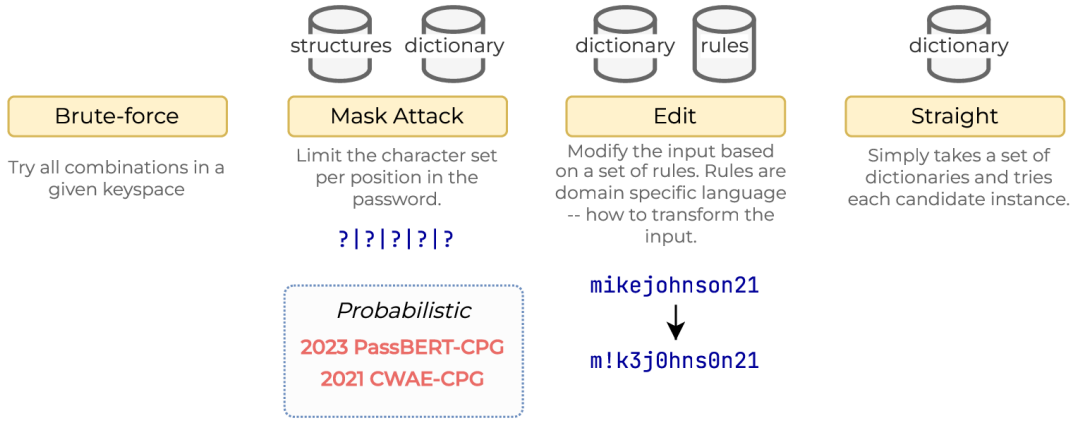


Figure 5.3: Examples of mode of attack.

applications. Instead, we propose categorizing password guessing algorithms based on their **mode** of attack, which better reflects their real-world effectiveness and implementation. Some of these modes are illustrated in Figure 5.3.

In academic research, most password guessing methods operate as *dictionary generators*, where the primary focus is on generating potential password candidates. The actual password cracking process typically relies on established tools such as Hashcat [228] or John the Ripper (JtR) [225], which perform the hash comparison operations. These open-source tools are highly optimized to take advantage of many types of GPUs and architectures to speed up the hashing process.

> **Definition 5.2.1 (Dictionary/Wordlist)** *A collection of precomputed password candidates, typically containing:*
>
> - *Common passwords from data breaches*
> - *Words from natural languages*
> - *Known password patterns*
> - *Organization-specific terms*
> - *Common keyboard patterns and variations*

The concept of attack **modes** is largely influenced by Hashcat's implementation architecture. Hashcat, together with JtR, has established itself as the industry standard for offline password cracking, offering various attack strategies optimized for different scenarios and password patterns. This mode-based approach provides a more practical framework for understanding

and implementing password guessing algorithms.

- **Brute-force**: In this mode, we try all combinations in a character space. This mode is essentially obsolete for any useful password guessing.
- **Mask attack**: An attacker reduces the number of combinations to try by limiting which characters can appear in each position of the password. This approach works well when the attacker knows part of the password structure. Section 5.4 explains mask attacks in detail and conducts a systemisation of the mode.
- **Edit**: An attacker modifies existing password candidates using specific rules. In Hashcat, these rules tell the programme exactly how to alter each candidate password.
- **Straight**: An attacker attempts passwords using a set list. Hashcat [228] refers to this method as a *Straight* attack. When attackers merge multiple dictionaries, it is termed a *Combinator* attack.
- **Generative**: Attackers can generate new password candidates using several methods:
  - **Markov n-gram models**: These analyse how frequently certain character sequences appear and use those patterns to generate likely passwords. Password crackers widely use this effective approach.
  - **Probabilistic Context-Free Grammar (PCFG)**: This method creates rules on the password structure and assigns probabilities to each rule. PCFGs excel at generating passwords that look like those created by humans.
  - **Deep Learning**: Networks like RNNs and GANs learn patterns from leaked password databases to generate similar passwords. As more password databases become public, researchers increasingly use deep learning to crack passwords.
  - **Machine Learning**: Algorithms like decision trees, random forests, and support vector machines help attackers predict which passwords to try first.
- **Hybrid**: An attacker combines multiple methods above to create more sophisticated attacks. For example, combine dictionary attack with mask attack.

Although these modes and password guessing attacks are presented as distinct categories, they are better approached as overlapping characteristics. For example, a generative model such as `CWAE-CPG` [196] using deep learning incorporates mask-like constraints. Therefore the modes and categorisations aren't mutually exclusive.

Ideally we should be able to chain these functions together and condition them on available information. The output of one function can serve as the input to another, creating a more sophisticated attack pipeline.

5.2.3 Existing Algorithms

Year	Model	Data	Comparison	Method
2023	`RFGuess` [257]	Rockyou 000webhost	FLA PCFG Markov	Random Forest Trees
2022	`GNPassGAN` [273]	Rockyou phpbb	PassGAN	GAN
2022	`OMECDN` [108]	Rockyou Yahoo 12306 CSDN	OMEN PassGAN	Markov n-gram
2021	`AdaMs` [195]	4iQ	Hashcat Markov	ANN
2019	`PassGAN` [97]	Rockyou LinkedIn	Hashcat PCFG FLA JtR	GAN
2016	`FLA` [149]	PGS PGS++	PCFG JtR Hashcat	LSTM
2015	`OMEN` [63]	Rockyou MySpace Facebook	Markov PCFG JtR	Markov n-gram
2009	`PCFG` [260]	MySpace Finnish List SilentWhisper	JtR	PCFG
2005	`Markov` n`-gram` [171]	142 Passware	Rainbow Table	Markov n-gram

Table 5.1: Examples of trawling and non-targetted password guessing methods and algorithms.

PRINCE Algorithm

The PRINCE algorithm [227] (**PR**obability **IN**finite **C**hained **E**lements"), is a password guessing algorithm when given an input wordlist W and a target length n, the algorithm generates password candidates through permuted combinations. The algorithm was introduced by Jens Steube in 2014.

Probabilistic Algorithms

Markov *n*-gram [171]. Markov n-gram models have been widely used for password guessing, with numerous iterations and improvements over time. The basic idea behind Markov n-gram models is to capture the statistical properties of passwords by analysing the frequency and probability of character sequences.

One notable example is `OMEN` [63] which outputs password guesses in descending order of their likelihood–prioritizing more probable passwords. This approach allows for more efficient password guessing by focusing on the most promising candidates first.

Building upon `OMEN` [63], `OMECDN` [108] incorporates a neural network-based discriminator to rank and filter the generated passwords. The discriminator, denoted as $f_{\text{GAN-D}}$, assigns a score S to each password:

$$S = f_{\text{GAN-D}}(X) \tag{5.1}$$

`OMECDN` then selects a subset of passwords D_{OMECDN} from the OMEN-generated passwords D_{OMEN} based on a threshold γ:

$$D_{\text{OMECDN}} = X \in D_{\text{OMEN}} : f_{\text{GAN-D}}(X) \geq \gamma \tag{5.2}$$

While `OMECDN` provides a marginal improvement over `OMEN` (4.07%), it likely introduces a significant overhead in terms of generation speed compared to `OMEN`.

The performance of Markov n-gram models can be further enhanced by incorporating additional

techniques, such as smoothing and interpolation, to handle unseen or rare character sequences. Smoothing methods, such as Laplace smoothing or Kneser-Ney smoothing, assign non-zero probabilities to unseen n-grams, preventing the model from assigning zero probability to potentially valid passwords.

Interpolation techniques, such as linear interpolation or backoff, combine the probabilities of different order n-grams to improve the model's ability to handle novel sequences. For example, linear interpolation computes the probability of an n-gram as a weighted sum of the probabilities of lower-order n-grams:

Probabilistic Context-Free Grammar. `PCFG` [260] models represent password structures as a set of grammar rules with associated probabilities. By learning the grammar from a password dataset, `PCFG` models can generate password guesses that adhere to the learned patterns and structures. The model works by dividing passwords into chunks based on letters (L), digits D, and special symbols S. The model then learns the rules and probabilities for these chunks from a training dataset. For example, this is how a password is represented in the `PCFG` grammar:

$$\texttt{password22!} \rightarrow L_8 D_2 S_1$$

Subsequent variants of the original `PCFG` [260] have introduced key improvements. `WordPCFG` [38] introduces word extraction techniques to identify semantic segments, including dictionary words, keyboard patterns, and other combinations, rather than only relying on basic character classes. `Personal-PCFG` [130] expanded the symbol sets to include semantic categories like birthdates, names, and personal information, enabling more targeted password guessing. Probability assignment has been refined by using frequency-based probabilities and non-zero probabilities for unseen strings, with some models also using cross-validation to improve generalisation. Finally, hybrid models, such as `PassTCN-PPLL` [272], have been developed that integrate `PCFG` with other methods, like Markov models and Recurrent Neural Networks.

Research Opportunity

A 2014 comprehensive review of probabilistic models by Ma et al. [138] concludes that whole string Markov models generally outperform PCFG models in password guessing tasks. This finding suggests that capturing the statistical properties of entire passwords is more effective than modelling the underlying grammar structure. With the improvements of the original PCFG [260] model, and perhaps using different algorithms and architectures, it would be interesting to see whether that conclusion remains valid.

Deep Learning

FLA [149] was one of the first methods to use an LSTM network to model passwords and show that they can be competent and at times superior to the likes of Markov n-gram and PCFG-based models; particularly for non-traditional password policies and when making more than 10^{10} guesses. Interestingly, the model did not have a material benefit from natural language dictionaries in the training set. However, an extended experiment might attempt this with different natural languages. For example, how would the results differ if the training set is in English but the test set is in Spanish?

PassGAN [97] uses a Generative Adversarial Network (GAN) to learn the distribution of real passwords from actual password leaks. Results show that it is competitive with state-of-the-art password guessing algorithms, and it can augment password generation rules by matching passwords that were not generated by other rules-based methods. There have been incremental improvements over the original PassGAN model. PassGAN used an Improved Wasserstein GAN (IWGAN) with gradient penalty. Subsequent models like GNPassGAN [273] addressed these issues by adopting gradient normalisation on the discriminator, which imposes a gradient norm restriction, and using binary entropy loss with a sigmoid layer instead of the Wasserstein loss. This approach enhanced the discriminator's capacity, reduced mode collapse, and led to better guessing capabilities and reduced duplicate password generation.

Other improvements include changes to the network architectures. REDPACK [169] replaced the ResNet blocks in PassGAN with Recurrent Neural Networks (RNNs), introducing a new

cost function for greater stability. G-Pass [275] uses LSTM in the generator for better sample quality, and multiple convolutional layers with varying filter sizes in the discriminator. G-Pass also incorporates Gumbel-Softmax with temperature control, allowing for a trade-off between sample diversity and quality by adjusting the temperature during training. CWAE-GAN [196] introduces stochastic smoothing to the password representation, which allows for more training iterations and greater model stability. The model also employs a deeper network architecture with residual bottleneck blocks and batch normalisation, which improves its ability to learn complex password patterns, leading to a higher matching rate compared to PassGAN.

The Adaptive Dynamic Mangling rules attack (AdaMs [195]) is an adaptation of Hashcat's mangling/edit mode. AdaMs aims to adapt to the target password distribution at runtime by learning best mangling rules rather than relying on static, pre-configured rules. AdaMs seems more efficient, i.e. less guesses required for a successful crack. AdaMs outperforms the standard mangling mode. Though, here the quality of the rule-set used is highly important and would have a material effect on the outcome of the experiment.

5.2.4 Limitations of Existing Methods

Current methods do not incorporate the speed of generation, which is an important factor that most open-source tools and industry have focused on. We believe a practical method would factor in both generation speed and quality of output, i.e. the number of guesses. The next station aims to model the password guessing process and incorporate these two factors to determine the true practical performance.

5.3 Process

When attackers attempt to guess passwords, they must overcome hash functions that are designed to be computationally infeasible to invert (one-way function property). Therefore, we should naturally be interested in the rate at which they attempt this guessing. Whilst

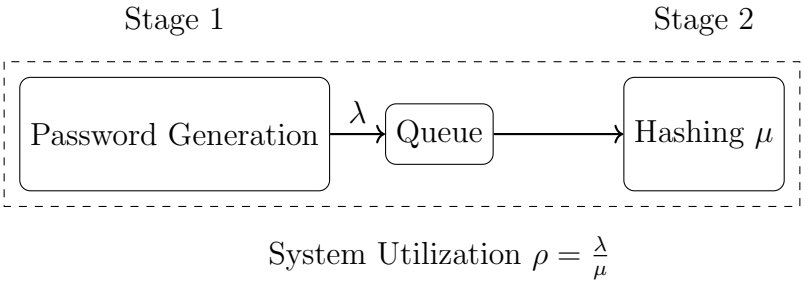


Figure 5.4: Visual representation of the password guessing process. Assuming an offline scenario where the attacker posses the hash value. λ is password generation rate and arrival rate; μ is the hash rate or service rate; and ρ is the utilization rate of the system.

industry and the open-source community have focused on optimising hash rates to traverse the search space more quickly, academic literature and models have conversely neglected such optimisation. The question for us then is whether the newly proposed models perform better than even brute-force methods.

Systematising the guessing process provides a framework for optimising and allocating resources efficiently. By identifying bottlenecks, such as a slow dictionary generator, we can compare and select algorithms based on specific scenarios. Each stage in the process requires different resources, such as GPUs; therefore, modelling this process enables us to allocate these resources effectively. This systematic approach allows us to determine precisely how and when to scale or parallelise resources. Moreover, such a model helps to accurately determine both the cost and the time requirements for the entire process.

We model the password guessing process in two distinct steps: 1) Generation: This step produces a potential candidate for a given hash, and 2) Hashing: in this step, we compute the hash of the candidate password produced in the previous step. Whilst we assume that we are carrying out an offline attack (where we possess the hash value), this model can be generalised for other modes of attack. This process is visualised in Figure 5.4.

To optimise a password guessing algorithm, we need to balance two key factors:

1. How well the algorithm guesses—measured by how many guesses it needs before succeeding. We can express this as $\mathbb{E}[X]$, where X represents the number of guesses.
2. How quickly the algorithm generates its guesses. We measure this speed using $N(t)$, which

represents the total guesses the algorithm has made by time t.

5.3.1 Password Guessing Probability

We will now incorporate the probability of successful guesses after a x number of guesses guesses. Let $F(x)$ be the cumulative probability distribution function (CDF) of successful password guesses after x number of guesses. $F(x)$ is essentially an assumed distribution by a guessing algorithm and we are interested in comparing these assumed distributions against each other with the ultimate aim of incorporating time (more on this in Section 5.3.3.)

Brute-force: The base case is a brute-force method that can be modelled using a uniform distribution. Let us assume the length of character space is C and the length of a password is ℓ. Our search space would therefore be C^ℓ.

Uniform distribution for the brute-force method becomes:

$$F_\ell(x) = \min\Big\{\frac{x}{C^\ell},\, 1\Big\} \tag{5.3}$$

Let $\mathcal{L} : \Omega \to \{1, \ldots, n\}$ be the random variable length. The composite distribution function becomes:

$$F_B(x) = \sum_{\ell=1}^{n} F_\ell(x)\, \mu_{\mathcal{L}}(\ell) \tag{5.4}$$

where $\mu_{\mathcal{L}}$ is the probability mass function of password lengths. This then resolves to:

$$F_B(x) = \sum_{\ell=1}^{n} \min\Big\{\frac{x}{C^\ell},\, 1\Big\}\, \mu_{\mathcal{L}}(\ell) \tag{5.5}$$

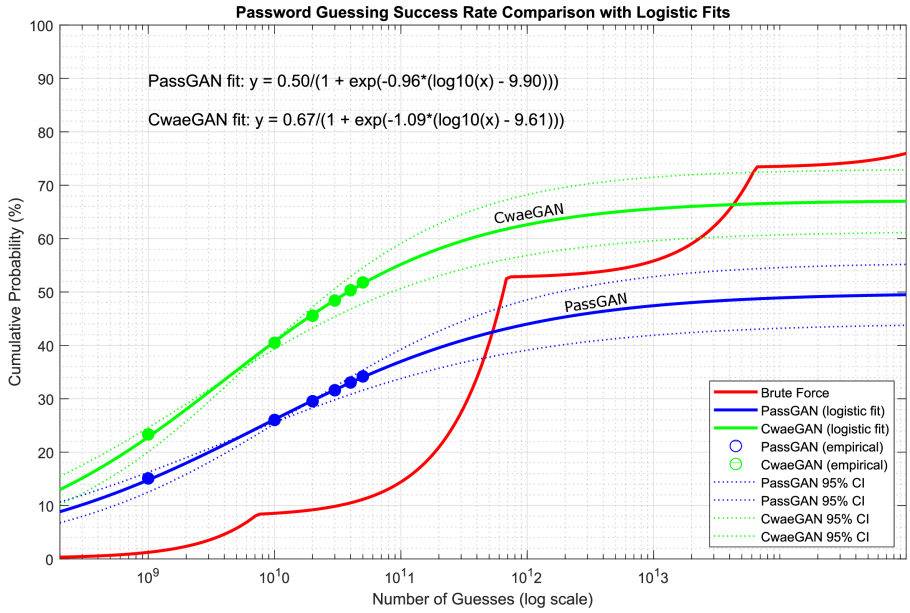


Figure 5.5: Chart showing the cumulative probability distribution of `PassGAN` [97], a similar GAN model (`CWAE-GAN` [196]) and a brute-force method based on the Rockyou length distribution. The data fro the GAN models are taken from the same experiment and fitted with a logistic function. Source: Own figure.

Logistic function: Based on the empirical results (as we shall see in Section 5.3.2), the deep learning models fit a logistic function well.

$$F_L(x) = \frac{M}{1 + e^{-k(x-x_0)}} \tag{5.6}$$

Where:

M : maximum asymptotic value

k : logistic growth rate coefficient

x_0 : horizontal shift parameter (inflection point)

5.3.2 Number of Guesses Empirical Evaluation

Putting our theoretical work into practice to this point, we want to ask how well the algorithms perform compared to the brute-force method. For this, we select the trawling attacks described in the CWAE-GAN [196] and OMECDN [108] papers.

For the length distribution $\mu_{\mathcal{L}}(\ell)$ (in brute-force method) we will use the empirical distribution from the Rockyou dataset as both papers used the RockYou password leak for training and testing their models, though they reported dataset details quite differently. The OMECDN [108] paper provided comprehensive statistics, stating they used the Skullsecurity RockYou leak containing 32.6M passwords, which reduced to 14.3M after deduplication, and was further processed by removing passwords over 10 characters and those with illegal characters before being split 80:20 for training and testing. In contrast, the CWAE-GAN [196] paper only briefly mentioned using the RockYou corpus with passwords up to 10 characters and filtered uncommon characters, but did not provide specific dataset sizes or train/test split details. This difference in dataset transparency makes it difficult to directly compare model performance.

We assume that models stop producing meaningful password candidates at some point and therefore the logistic function fits the empirical data well–as shown in Figure 5.5. The brute-force method overtakes the two GAN models in CWAE-GAN [196] at $\sim 4.7 \times 10^{13}$ guesses (68% successful guesses).

5.3.3 Expected Time to Crack

Let's assume the cumulative number of guesses made up to time t is $N(t)$, therefore the cumulative probability of a successful guess by time t is $F(N(t))$. With this we can proceed to modelling the expected time to crack a password.

$$N(t) = \min(\lambda,\, \mu)\, t \tag{5.7}$$

The system's performance metric combines both the queueing dynamics and success probability. The probability that the password is found in the interval $[t, t+dt]$ is $dF(N(t))$, so the expected time is:

$$\mathbb{E}[T_c] = \int_0^\infty t \, \frac{d}{dt} F(N(t)) \, \mathrm{d}t = \int_0^\infty t \, f(N(t)) \, N'(t) \, \mathrm{d}t \tag{5.8}$$

To compute the expected time to successfully guess a password, we consider the time until a successful guess occurs, which depends on the number of processed guesses and their success probabilities. $N'(t) = \frac{dN(t)}{dt}$ is the rate at which the guesses are processed.

Uniform/Brute-force Function:

Starting with our CDF for brute-force:

$$F_B(x) = \sum_{\ell=1}^{n} \min\Big\{ \frac{x}{C^\ell}, \, 1 \Big\} \, \mu_{\mathcal{L}}(\ell)$$

Evaluating the function with $N(t) = \min(\lambda, \mu)t$:

$$F_B(\min(\lambda, \mu)t) = \sum_{\ell=1}^{n} \min\Big\{ \frac{\min(\lambda, \mu)t}{C^\ell}, \, 1 \Big\} \, \mu_{\mathcal{L}}(\ell) \tag{5.9}$$

This equation identifies the critical point where the ratio equals unity, representing the threshold condition for level ℓ.

$$\frac{\min(\lambda, \mu)t}{C^\ell} = 1 \tag{5.10}$$

We define T_ℓ as the ratio of capacity at level ℓ to the minimum rate. This represents the time required to reach capacity at each level.

$$T_\ell = \frac{C^\ell}{\min(\lambda, \mu)} \tag{5.11}$$

Substituting back:

$$\mathbb{E}[T_c] = \sum_{\ell=1}^{n} \mu_{\mathcal{L}}(\ell) \cdot T_\ell \tag{5.12}$$

Finally:

$$\mathbb{E}[T_c] = \frac{1}{\min(\lambda, \mu)} \sum_{\ell=1}^{n} C^\ell \mu_{\mathcal{L}}(\ell) \tag{5.13}$$

Logistic Function: Calculating the probability density function for our logistic CDF:

$$F_L(x) = \frac{M}{1 + e^{-k(x-x_0)}}$$

$$\text{Let } u = -k(x - x_0)$$

Substituting to simplify differentiation and using the chain rule:

$$F(x) = M(1 + e^u)^{-1}$$

$$\frac{dF}{du} = M \cdot (-1)(1 + e^u)^{-2} \cdot e^u$$

$$\frac{du}{dx} = -k$$

Multiplying the derivatives:

$$\frac{dF}{dx} = M \cdot (-1)(1 + e^u)^{-2} \cdot e^u \cdot (-k)$$

Simplifying:

$$\frac{dF}{dx} = Mk \cdot \frac{e^u}{(1 + e^u)^2}$$

Final probability density function:

$$f_L(x) = Mk \cdot \frac{e^{-k(x-x_0)}}{(1 + e^{-k(x-x_0)})^2} \tag{5.14}$$

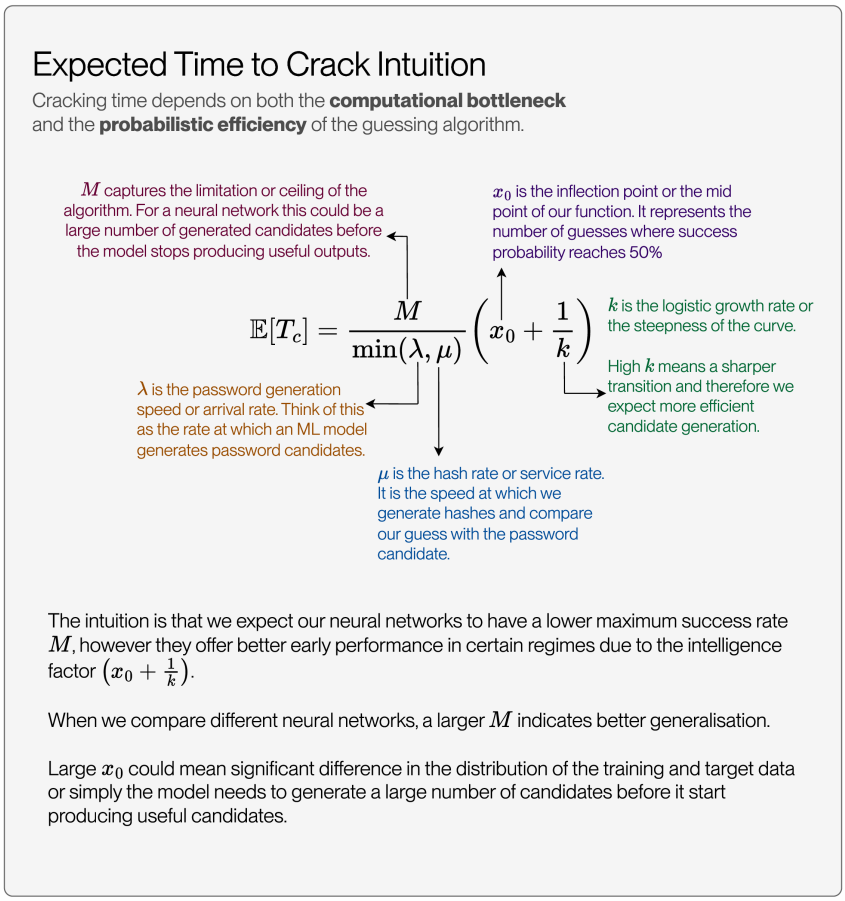


Figure 5.6: Visualisation of Equation 5.15. This figure explains the variables and provides some intuition behind the equation.

Now we can calculate the expected Time to Crack for our logistic function:

$$\begin{aligned}
\mathbb{E}[T_c] &= \int_0^\infty t \cdot \frac{kM\alpha e^{-k(\alpha t - x_0)}}{(1 + e^{-k(\alpha t - x_0)})^2} \, dt \\
&= \int_{-x_0}^\infty \frac{u + x_0}{\alpha} \cdot \frac{kM\alpha e^{-ku}}{(1 + e^{-ku})^2} \cdot \frac{du}{\alpha} \\
&= M \int_{-x_0}^\infty \frac{(u + x_0) k e^{-ku}}{(1 + e^{-ku})^2} \, du \\
&= M \left[\frac{x_0}{\alpha} + \frac{1}{k\alpha} \right]
\end{aligned}$$

Our final equation then becomes:

$$\mathbb{E}[T_c] = \frac{M}{\min(\lambda, \mu)} \left(x_0 + \frac{1}{k} \right) \tag{5.15}$$

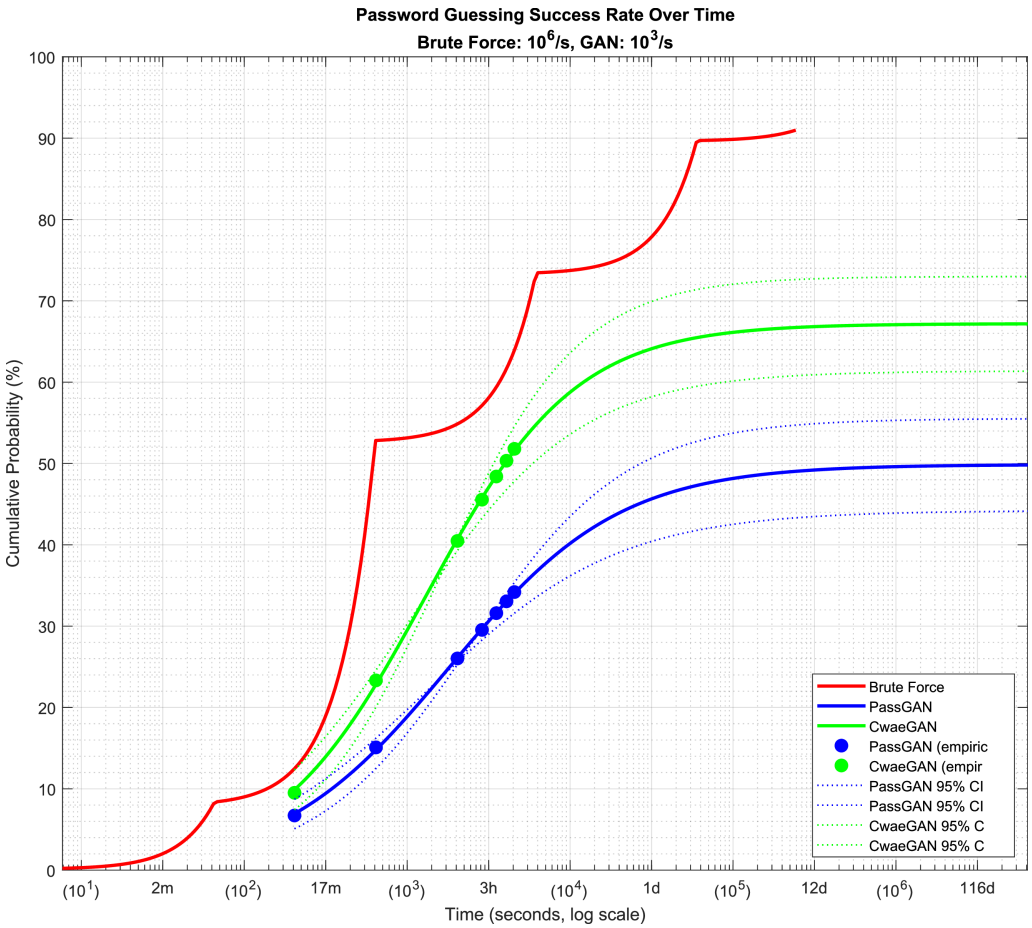


Figure 5.7: Calculating $F(N(t))$, incorporating both probability of guessing and the rate at which it happens gives us a more accurate view of the performance of password guessing algorithms. This framework demonstrates the utility of considering both generation quality and speed. Source: Own figure.

Equation 5.15 shows the expected time to crack a password. λ represents our arrival rate (or the number of password guesses we generate using a model), whilst μ is the hashing rate (or how many password hashes we can generate per unit of time). So, $\min(\lambda, \mu)$ tells us by what our system is throttled and is the *effective* password guessing rate. Together, the expression $\left(x_0 + \frac{1}{k}\right)$ scales the basic time estimate $\frac{M}{\min(\lambda,\mu)}$ to account for both initial search conditions and the probabilistic nature of password discovery. This gives us a more realistic expected completion time that reflects both the computational constraints and the stochastic aspects of the guessing process. Figure 5.6 provides a detailed breakdown of the equation in visual form.

5.3.4 Time to Crack Empirical Evaluation

Figure 5.7 shows our result using the same models as the number of guesses section. Here we assume the process is throttled by the number of candidates the GAN models are generating, i.e. $N(t) = \lambda t$, where λ is the arrival rate and therefore the generation speed.

For our empirical analysis, we used the same dataset configurations as in the previous guessing probability evaluation - the RockYou password leak processed to contain passwords up to 10 characters. We measured the actual time taken to generate and process password guesses using both brute-force and GAN approaches across different hardware configurations.

The hashing rate μ is based on Hashcat [228] benchmarks on consumer-level hardware. The hash rate is dependent on the hash function we are inverting and the hardware being used. Higher hash rate favours the brute-force method.

This empirical analysis supports our theoretical predictions about the relationship between generation rate, hash rate, and cracking success. It also highlights an important practical consideration: while sophisticated guessing strategies may be more efficient for common passwords, hardware-optimized can remain competitive and optimisation of the implementations should be considered a novel piece of work.

5.3.5 Other Strategies and Parallelisation

In the two-stage model considered in this chapter (candidate generation $\rightarrow$ hash verification), the effective cracking throughput is determined by the bottleneck between the candidate generation rate λ and the hash-processing capability μ. Here, the application of more advanced queueing theory would be beneficial for understanding how scaling the number of workers would affect guessing rates. Moreover, future work could also incorporate cost and the economics of password guessing at scale. In Figures 5.7 and 5.5, we assumed the attacker is not parallelising either λ or μ. The attacks are conducted on a consumer level hardware (internally the hashing might be parallelised at CPU or GPU level).

Increasing the number of generator or hashing workers changes the arrival process $N(t)$ and consequently the expected time-to-crack derived earlier. Framing parallelism in this way allows formal comparison between algorithmic improvements (better candidate ranking) and systems improvements (higher λ or μ).

For neural-network-based attacks, the system becomes constrained by the model's candidate-generation rate rather than by the computational hashing throughput. Generating intelligent password candidates requires significantly more computational resources per guess than verifying them. In this scenario, we can scale the number of generators, optimise generation speed through model compression or quantisation techniques, or precompute candidates. Ultimately, the strategy to choose depends on the attacker's intent, the type of neural network, and the target. There are leaked password dictionaries with billions of records already, so unless the neural network has better or more novel candidates, precomputing may not be useful.

When scaling the candidate-generation workers for neural networks, we can theoretically match or exceed the hashing rate μ, but this approach faces practical limitations due to memory-bandwidth constraints, inter-processor communication overheads, and the inherently sequential nature of some neural network operations. However, parallelisation enables neural networks to reach their maximum potential, M, more quickly, without fundamentally altering their learning characteristics. Therefore, we would merely reach that asymptotic ceiling more quickly while preserving the underlying intelligence limitations of the model architecture.

For naive brute-force strategies, the hashing rate can be parallelised almost linearly, significantly improving μ. This near-perfect parallel scaling gives brute-force approaches a significant advantage in highly parallel environments, potentially offsetting some of the intelligence-related benefits of neural approaches when sufficient computational resources are available.

Our work in this section remains valid when applying parallelisation techniques, and the approach we have taken enables us to reason effectively about parallelisation. Moreover, it prepares us to carry out future experiments using queueing theory, and even to consider the economics of password-guessing strategies.

5.4 Mask Attack

Mask attack is a method of guessing a password string when you have a partial representation of it, e.g. `p***w0r*`, or when individual characters are brute-forced using a chosen character set. Mask attack is a crucial password cracking technique as it allows us to make assumptions about the structure of the password, and therefore reduce the search space. Such assumptions about the structures are valid and practical. Moreover, it can aid in password recovery when the user has forgotten part of the password. Mask attacks can be used with the shoulder surfing attack, especially if the attacker has only captured small segments of the password. Despite the promising nature of it, the literature on this method is sparse. Hashcat [228] provides a rich deterministic mask attack mode, and in the academic literature, a variant of this attack is described as *Conditional Password Guessing (CPG)* [196], [270]. We will refer to it as *Mask Attack* because of its prominent use in practice, especially since Hashcat [228] refers to it as mask attack. The aim is to correctly guess the missing characters (denoted by "*").

The purpose of this section is to review existing methods and to systemise the mask attack. The methods described in [196], [270] follow a similar method; however, they have three drawbacks:

1. Limited comparison to Hashcat [228] and to the point that the mask attack method in Hashcat [228] is ignored.
2. Practicality of random masking of characters.
3. A flow and model of the entire guessing process is not provided and thus it avoids the most crucial aspect of password guessing–speed and candidate generation.

The brute-force method enumerates through masked characters in order to find a correct match. This method is predominantly used in practice using Hashcat [228].

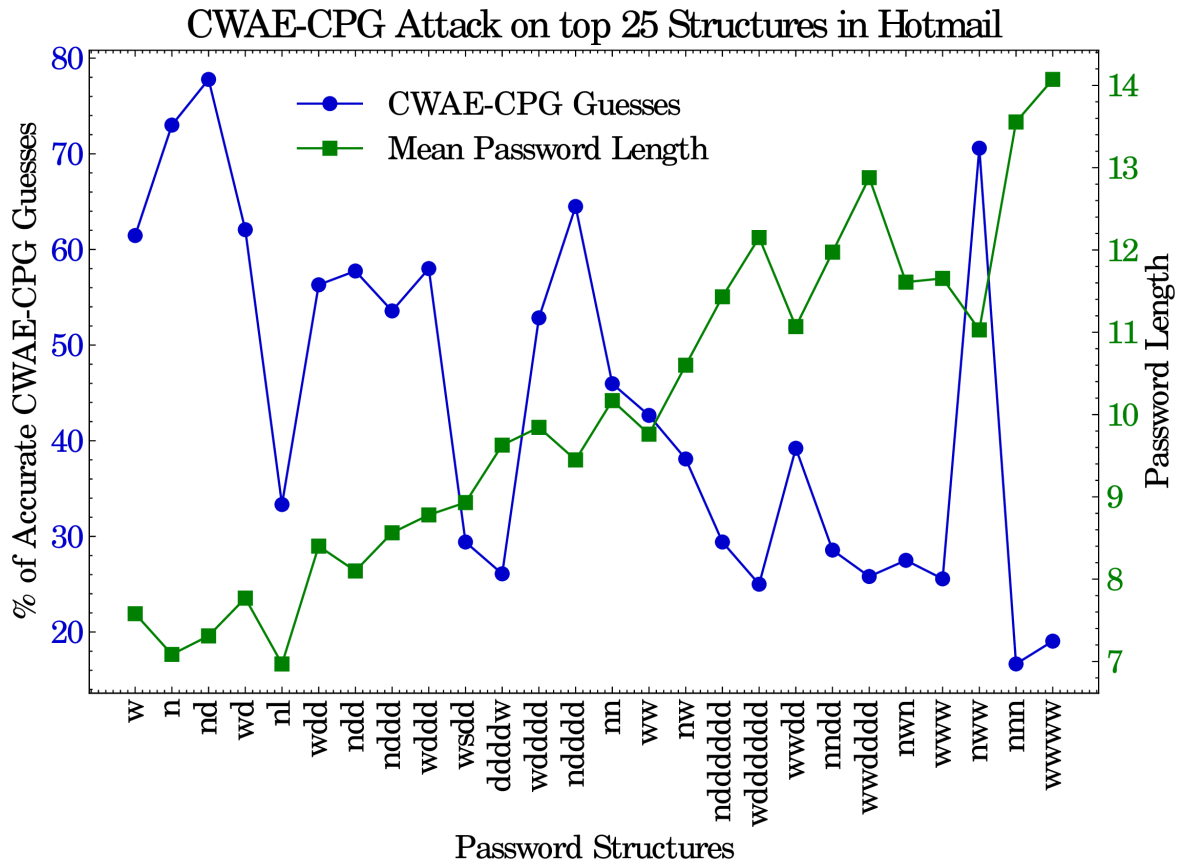


Figure 5.8: Graph showing the percentage of guesses (left y-axis) with respect to the password structures in the hotmail dataset. Right y-axis shows the mean password length with respect to the password structures in the x-axis. Source: Own figure.

Table 5.2: Deep Learning and Probabilistic Masking Attacks literature.

Exp	Train	Validation	Length	n
CWAE-CPG [196]	Rockyou	Linkedin	$9 \leq L \leq 16$	10^7
PassBERT [270]	Rockyou	NeoPets/Cit0day	$9 \leq L \leq ?$	10^7

5.4.1 Probabilistic Mask Attack

There are two notable models to be aware of at the time of writing: `CWAE-CPG` [196] uses a deep learning model based on the Wasserstein Auto-Encoder [239]. `PassBERT` [270] uses the BERT [60] architecture to model the password using the masking paradigm. These methods will often learn representation at word level, or structural patterns due to common passwords. These models condition the guess on some known characters in the password. The following

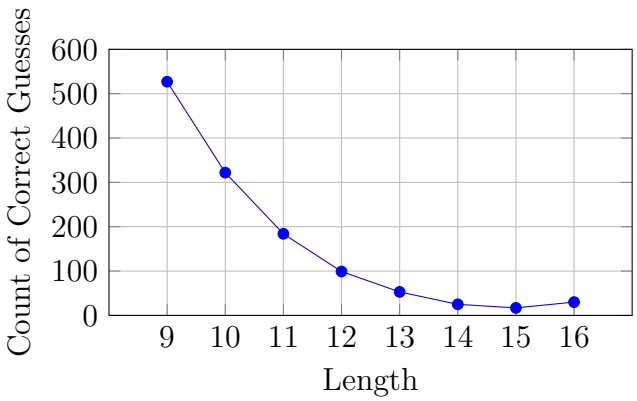


Figure 5.9: Graph showing the performance of CWAE-CPG [196] against length of passwords. Using the hotmail dataset for analysis. We see a significant drop off as the length of passwords increase.

shows the original password on the left and the *template*[2] or masked password on the right.

`passw0rd123` → `p***w0r*1*3`

The models then use deep learning techniques to learn to guess the masked characters or *wild cards* (denoted by "*".)

We hypothesise that these methods will not do well against passphrases, or uncommon structures. For example, `TheTrooper` → `*h*****per` has too many masked characters to identify it as two words. Due to the 16 character limit as well, the model's outcome will be biased towards passphrases–which as we discussed are the strongest password structures.

We evaluated CWAE-CPG [196] against our labelled hotmail dataset to better understand the models performance against different password structures, the results are shown in Figure 5.8. For each password in the labelled hotmail dataset, we generated a template as per the CWAE-CPG [196] masking algorithm; then we evaluated each template (i.e. masked password) using the implementation and checkpoints released by the authors; finally for each password structure we calculated the percentage of passwords that were guessed correctly within 10^7 guesses. Simpler structures (left side) tend to be shorter and more guessable; More complex structures (right side) tend to be longer and less guessable. Though the performance of CWAE-CPG

[2]The reader may use the following website to try the masking algorithm on their own inputs: `https://sirvan3tr.github.io/tools/pwd/pwd_masker.html`

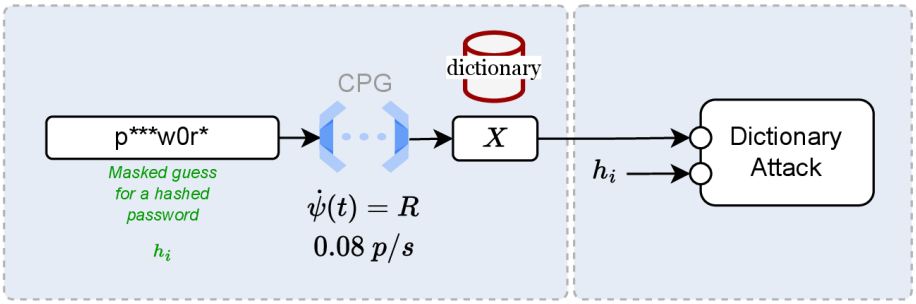


Figure 5.10: Data flow and model of a probabilistic mask attack or CPG. These methods thus far have randomly masked a password string and then generated a set of potential candidates. These candidates are then used to perform a dictionary attack. $\dot{\psi}(t)$ represents the rate of password generation. Note, an offline attack scenario is considered here.

seems correlated with the length of the password, we think the complexity is not necessarily the length but the fact that longer passwords in the dataset are composed using multiple words, dates, and names. This is further proof for NCSC's recommendation of composing these secrets using at least three random words.

Features of the training data, such as language, password structures, and date of breach, could effect the performance of the model. For example: Given a password dataset D_t released at time t, we hypothesise that passwords containing temporal features (like years and dates) follow a time-dependent distribution $p_t(y)$ that heavily favours years closer to and preceding the release date t. This creates a fundamental challenge for password guessing models: when a model trained on an older dataset (e.g., D_{2009}) is evaluated on a newer dataset (e.g., D_{2020}), it encounters significant temporal distribution shift, as the training data lacks examples of passwords containing more recent years. For instance, while passwords like `password2018` might be common in D_{2020}, they would be entirely absent from D_{2009}, leading to poor generalization performance for passwords containing post-2009 temporal features.

5.5 Discussion

Going beyond surveys, systematising the knowledge of password guessing algorithms can be highly beneficial to future researchers–our reasoning is as follows.

Our continued understanding of passwords remains critical, as passwords remain the most

adopted method of user authentication. The adoption of password-less authentication has remained slow, and we foresee that password will remain the leading authentication system over the next decade. Our continued understanding of it is vital for national security, too. The growth in password attacks and data breaches provides more reasoning on why we should continue research on password security.

The work in this chapter can guide future research efforts. We have identified limitations in previous approaches and various future research efforts. Insights from this analysis can inform the design and development of new authentication systems.

Password security encompasses various issues that might be relevant to other disciplines. It touches on human-computer interaction (HCI), information security, human psychology, user experience design (UX), and cryptography. Having a common understanding can foster collaboration. This literature can also serve as a source of education.

The days of passwords are not yet over as most systems and web applications still use passwords. Passwords will continue to be used for offline systems, secure and privately networked systems, and for ad hoc usage such as password-protecting files or folders. Password-less schemes have had a slow adoption rate. The adoption of the Web Authentication API (WebAuthn) has been slow, though Passkeys have shown promise because of their usability. For these reasons, we must continue to invest in password security through further research. The purpose of this chapter is to increase the pace of password-security research.

5.6 Summary

This chapter has provided a comprehensive systematisation of knowledge (SoK) of password guessing algorithms, bridging the gap between academic research and practical applications.

First, we address the gap between academic approaches and industry implementations. While practical tools like Hashcat focus on generation speed and hardware optimisation, academic research tends to focus on maximising successful guesses while minimising wasted attempts. We

developed a novel framework for evaluating password guessing algorithms that combines both generation quality and speed. Our initial experiments revealed that sophisticated academic methods do not necessarily outperform simpler methods when both factors are considered.

We then provided a detailed overview and potential of Mask Attacks and variants of them. Although mask attacks can significantly reduce the search space and improve guessing efficiency, their effectiveness varies considerably depending on the complexity and structure of the password. The empirical evaluation demonstrated that current deep learning-based mask attack methods struggle with longer passwords and complex structures, particularly passphrases.

Chapter 6

Password-less Authentication and Identification

This chapter designs and implements a passwordless system—deeID—for identification to mitigate impersonation attacks. deeID is a blockchain-based system that enables passwordless authentication and identification by extending the Fiat–Shamir protocol to support multiple identity issuers. Blockchain is used as a decentralised public-key infrastructure, providing identity verification without relying on passwords or centralised authorities while maintaining compatibility with existing web technologies.

The source code for the implementation can be found at `https://github.com/deeid`.

6.1 Introduction

Many organisations and digital identity systems rely on a fundamentally flawed premise for identification: using shared secrets for verification. Every time you enter a password, provide your date of birth, or answer security questions, you share reusable information that can be stolen, intercepted, or replayed by attackers. Most often, these supposed *secrets* are publicly available information, such as your date of birth. This approach creates systemic vulnerabilities

throughout the digital ecosystem that lead to identity theft and impersonation attacks.

Well-resourced and motivated attackers exploit these shared secret systems: in one incident, an attacker used deepfake technology to impersonate a company executive, successfully convincing an employee to transfer US$25 million [160]. In another case, an attacker compromised the SEC's Twitter/X account using a SIM-swapping attack facilitated by a fake ID and impersonation [112]. These cases demonstrate that we should seldom rely on readily available information for authentication and identification.

Our idea is to replace all shared-secret systems with interactive challenge-response protocols in which identity verification reveals nothing reusable, enabled by dynamic trust networks that allow any organisation to issue identities that work across national borders.

We call this system deeID and this is how it works: It extends the Fiat-Shamir identification protocol to work with multiple identity issuers, using the blockchain as a decentralised public key infrastructure. When you need to prove your identity, the system generates a cryptographic challenge to which you respond using private keys stored on your mobile device. This response proves that you possess the correct credentials without revealing any secret information that could be reused. Identity issuers (banks, governments, universities) can create and verify these credentials using a shared blockchain ledger, but no single authority controls the system—hence the dynamic trust network.

The novelty of deeID is in adapting the classical Fiat-Shamir scheme, originally designed for single-issuer scenarios, to work seamlessly with multiple independent identity providers. Users can obtain *many* digital identities on the blockchain provided by different organisations. Verifiers can check these credentials through interactive proofs rather than through shared-secret verification. Having many sources of identities is positive as it mitigates a single point of failure and verifiers can use this to verify consistency across identities.

Unlike password systems which rely on shared secrets, deeID reveals nothing reusable during verification. Unlike federated identity systems which create vendor lock-in, deeID enables any organisation to issue credentials that work with any other organisation's verification systems.

Unlike traditional PKI systems which require centralised certificate authorities, deeID uses blockchain to create a decentralised trust infrastructure. WebAuthn is primarily designed for authentication, deeID is designed for identification. deeID predates Passkeys (which is an implementation of WebAuthn and the relevant standards).

The benefits of this approach are numerous. It eliminates the security weakness of shared secrets—there is nothing for attackers to steal or replay; it preserves privacy through the zero-knowledge proof of identity. It enables interoperability where identities issued by one organisation can be verified by any other organisation in the network. Finally, it creates a foundation for dynamic trust networks where identity verification can adapt to real-world relationships rather than being constrained by centralised authorities.

6.2 Threat Scenario

Attackers exploit identity weaknesses not only through sophisticated social engineering campaigns but also through insider threats and compromised services. While traditional security models assume services can be trusted, we must consider that the very systems we authenticate with may be adversarial. Insiders with privileged access can bypass security controls, exfiltrate identity data, or manipulate authentication processes. Service providers themselves may be compromised, coerced, or simply negligent in protecting identity information.

By combining publicly available information with AI-generated content and insider knowledge, attackers can build highly convincing attacks that exploit both technical vulnerabilities and human trust relationships. This interconnected nature of identity systems, coupled with the necessity to treat authentication services as potentially hostile actors, means that a compromise in one area can cascade into unauthorised access across multiple domains.

6.3 Design

Our aim is to create an identity system that: (1) preserves privacy through zero-knowledge proofs, (2) supports multiple identity issuers in a decentralised way, (3) maintains compatibility with existing infrastructure, and (4) provides stronger security guarantees than current systems. While WebAuthn solves the authentication problem well, it does not address the broader identity management and verification challenges that deeID attempts to solve. The novel contribution would be in creating a comprehensive identity framework that extends beyond simple authentication to handle complex real-world identity verification scenarios.

The deeID is designed to function with mobile devices. Identity holders require a mobile phone capable of securely storing private keys using device-level encryption or secure enclave technology. The deeID application manages these keys, performs cryptographic operations, scans QR codes, and maintains WebSocket communication with verifiers. On the service side, integration is lightweight: Web applications employ JavaScript libraries to generate QR codes that contain session identifiers and challenges. When a user scans such a code, their mobile device initiates a WebSocket session with the service and executes the interactive authentication protocol. Importantly, no browser extensions or modifications are required. Browsers are used only to display QR codes and to confirm completed authentication, while all cryptographic operations remain confined to the user's device.

The deeID system involves four primary actors:

- Identity Holders are individuals who own and control their digital identities. Each identity holder possesses a unique blockchain-based smart contract that serves as their identity anchor, along with private keys stored on their mobile device for cryptographic operations.
- Identity Issuers are organisations (banks, universities, governments, employers) that can create and issue credentials to identity holders. Each issuer maintains their own smart contract containing cryptographic parameters used in the Fiat-Shamir protocol.

- Verifiers are services or organisations that need to authenticate identity holders. They may maintain lists of trusted issuers and can verify credentials through interactive cryptographic challenges without accessing any shared secrets.
- The Blockchain Network serves as the decentralised public key infrastructure, storing smart contracts, public keys, and issuer parameters while ensuring that no single authority controls the system.

Other requirements: deelD is designed with the following requirements:

Compatibility. Compatible with existing browsers and existing technology that use the email/password combination for authentication. The user must not download new browsers. Developers should be able to use existing JavaScript or Python web stacks to implement deelD as an authentication scheme.

Key Store Management. Shared and equal access to public keys that link users to their identities, ensuring transparency and verifiability within the system. This assumes that a user can possess multiple keys (e.g., for different devices and keys for encryption), allowing flexibility and enhanced security across various use cases.

Dynamic Identity. Ability for anyone to issue identities to individuals. Utilise and implement the Fiat–Shamir scheme for multiple identity issuers.

Privacy by Design. Cannot record any personal details on the public chain.

Unlinkable Authentication Colluding actors cannot determine if our anonymous identity is the same across their servers and systems.

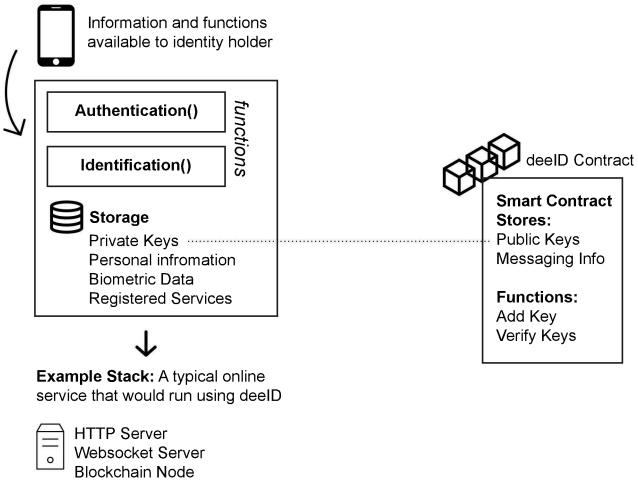


Figure 6.1: deeID architecture overview. Source: Own figure.

Requirements

Steps required before an entity can issue an identity:

Person **A**: The individual wishing to receive a new identity from another person or organisation.

Person **B**: Identity issuer.

1. **A**: Must create a deeID representation on the blockchain, e.g. on Ethereum, create a unique contract as per the common standards.

2. **B**: Must also create a deeID representation on the network.

3. **B**: Create a random modulus which is a product of two large prime numbers, $n = pq$, and n should be at least 512 bits.

4. **B**: Store n on the network. In our implementation it is stored in a data structure on the deeID contract.

6.4 Implementation

6.4.1 Overview

The proposed system, deeID, is a combination of technologies that enables a dynamic and decentralised identity system with functions compatible with existing services. Although the proposed system can be built on various devices, even a unique independent device solely for this application, we have assumed the implementation on a typical mobile device running Android or iOS. deeID uses the Fiat–Shamir identification cryptosystem with a blockchain that acts as a PKI.

Figure 6.1 provides an overview of the system. The core components are:

- **Distributed Ledger:** Our implementation uses Ethereum and we have developed several smart-contracts that act as portals and identity managers.
- **Mobile Phone:** A device such as a mobile phone that can store private keys and interact with services and servers over the Internet (in part to ensure compatibility with existing web applications). We built a mobile phone application that allows the user to authenticate and identify (Fiat–Shamir scheme) themselves. The application uses QR codes to interact with web applications and then talk to servers (using WebSocket connection) to carry out the necessary functions (e.g. interactive Fiat–Shamir scheme).
- **Libraries:** Required libraries that developers can install so that their users can use deeID to authenticate themselves on the developer's application.

Analogy

To effectively showcase the potential of deeID, one could employ the analogy of an identity card. Figure 6.2 illustrates this analogy, highlighting the three primary elements of an identity card:

Photograph for verification: We typically use the photograph of the individual on the identity card to identify the user. That is, the assumed valid ID card belongs to the individual trying to prove their identity to the challenger. In deeID, this is done by the ownership of a private key associated with an identity. Via the Fiat–Shamir scheme, one would be able to prove their ownership of the private-key and, therefore, identity.

Human and machine readable strings: We require a unique string that would uniquely identify the card. This string can also be used for record keeping with respect to the identity holder. Along with this unique string, we also have the full name of the user. Although it is typically not unique. However, it is what we use to identify with one another at the human level. In deeID, the smart contract on the network with its unique address represents the unique machine-readable identity.

Identity issuer's integrity: The integrity of the identity card comes through trusting the physical card itself and its physical properties. This trust also assumes that the verifier trusts the organisation that has issued the card. We find organisations that require different proofs of identity, e.g. passport and driving license in the UK. Therefore, we are concerned with different issuers and their reputation. Reputation in itself would require a full research experiment; in the current state of deeID we have used a simple link on the identity issuer's smart contract, which links to their web address if they have one proving that the owner has access to both.

6.4.2 Decentralised Public Key Infrastructure

A fundamental challenge in extending a scheme, such as the Fiat–Shamir scheme, to multiple identity issuers is a trustworthy and robust public-key management infrastructure. We leverage the decentralised public key infrastructure (DPKI) to enable deeID to handle multiple identity issuers at the same time. Numerous studies have contributed to decentralised PKI (DPKI) [64], [124], [198], [271] and here we represent a simple solution built on Ethereum for deeID. With this approach, the user can associate other public-keys to their identity, e.g. RSA keys for encryption and public-keys used on other devices that links back to the identity.

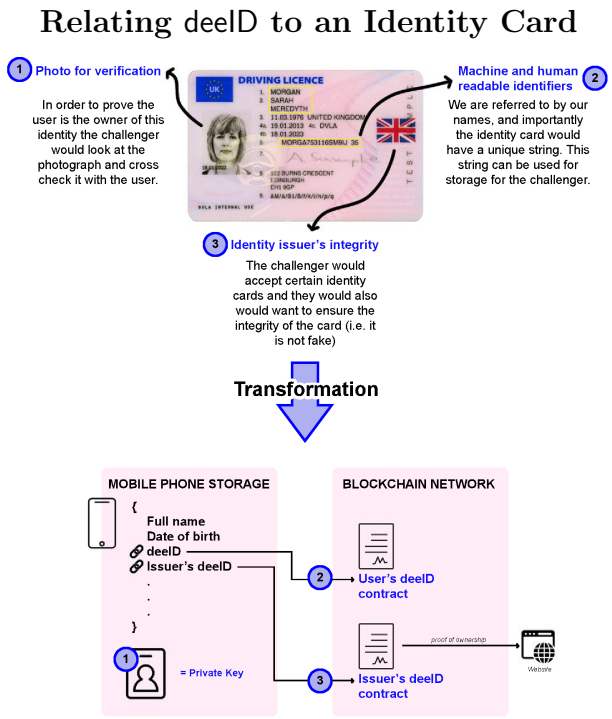


Figure 6.2: At the top we have the main components of an identity card and at the bottom we have the equivalent functions in deeID. Source: Own figure.

Our DPKI implementation diverges from traditional certificate-based systems by anchoring key authenticity directly to verifiable identities. This represents a paradigm shift: instead of relying on certificate chains, we establish trust through the identity verification process itself. While existing DPKI solutions [64], [124], [198], [271] focus on certificate management, our approach integrates identity verification with key management.

We assume that the user has ownership control of their deeID smart contract, and thus any changes within the contract are not malicious. Furthermore, the integrity of the public keys falls under the actual deeID, and individual public keys are not issued certificates as such. So, if a verifier has no knowledge of the user's deeID, then they must first identify the deeID user (through the Fiat–Shamir scheme).

Adding new public keys to the smart contract deeID is a simple process of invoking the `addKey()` function with the following arguments: `title`, `key` and `comment`. The criterion for adding the key is that the user must be the owner of deeID—which in our implementation has been cross-checking the sender's address with the address of the owner (stored on the smart contract). A real-world application could use any of the keys stored on the deeID to verify the integrity of

the sender. Lastly, verifying a specific key would involve querying the blockchain with respect to the public key and deeID, in our implementation we had a function `getKey()`, which would return all the details related to that key.

Other blockchain-based PKIs are more traditional in that they use certifications, e.g. CertLedger [124] and CertCoin [82]. In deeID authenticity and security come from the identification of the deeID itself, which can depend on the identity issuer and the verifier's willingness to trust the issuer or not. Although in the current implementation of deeID this verifier and issuer trust is not quantified, we envision that one would be able to add a quantifiable reputation layer.

6.4.3 Authentication

The deeID authentication system comprises two primary components: a mobile client application and a server-side blockchain node implementation with associated libraries for Web3 integration. The architecture enables direct communication between the web application and mobile client during authentication flows.

deeID offers users a simpler way to authenticate themselves compared to passwords, while allowing developers to implement it using current technologies with minimal effort. The system accomplishes this through two interconnected components: a mobile phone running the deeID app serves as the first component, while a server-side application operating a blockchain node with deeID libraries installed serves as the second component. These libraries enable the web application to interact directly with the mobile phone during the authentication process.

One can authenticate via four different methods:

- **Explicit Link.** With this method, we are using the deeID, i.e. identity smart-contract, to authenticate the user. Basically, using the PKI method. Once the user has a public-key stored on their deeID contract then they can use that to authenticate themselves using their deeID. The user and the verifier have to query the chain for the existence of the specific public-key used to sign the authentication message. Lastly, because only deeID is

used to authenticate, the user can store multiple public keys on the contract and thus use multiple devices with different keys for authentication. At the end the typical session creation occurs, we have simply shown "Login successful" to re-iterate the unique points related to our protocol.

- **Generate a Public Key.** One can also avoid the use of the blockchain all together and simply generate a unique public-private key once they register with a website and use this public-key for authentication. The benefit of this is that the user can remain anonymous to the application. This also provides *unlinkable authentication*. This means that two or more colluding servers are unable to determine if the user is using their platform. Although the colluding servers could use other means such as using IP address to guess if they have the same users.

- **Fiat–Shamir Identification.** The user can simply use the Fiat–Shamir scheme to also authenticate themselves. With this method you are also sharing your identity and its proof; this is therefore not useful if the user wishes to remain anonymous.

- **Strong Password.** Just like with the second method, one can use their mobile phone like a password manager and generate strong passwords to authenticate themselves with a given service.

Table 6.1: Methods of authentication with deelD.

Auth Method	**Chain Storage**	**Query Chain**	**Interactive**	**Multi Device**
Explicit link to deelD	✓	✓	✗	✓
Generate a public-key	✗	✗	✗	✗
Fiat–Shamir identity	✓	✓	✓	✗
Strong password	✗	✗	✗	✗

Table 6.1 summarises the above authentication methods. Although the choice of which authentication method to use is not entirely dependent on the user, we believe, ultimately, online behaviour will dictate which method is best suited to a given application. The differences

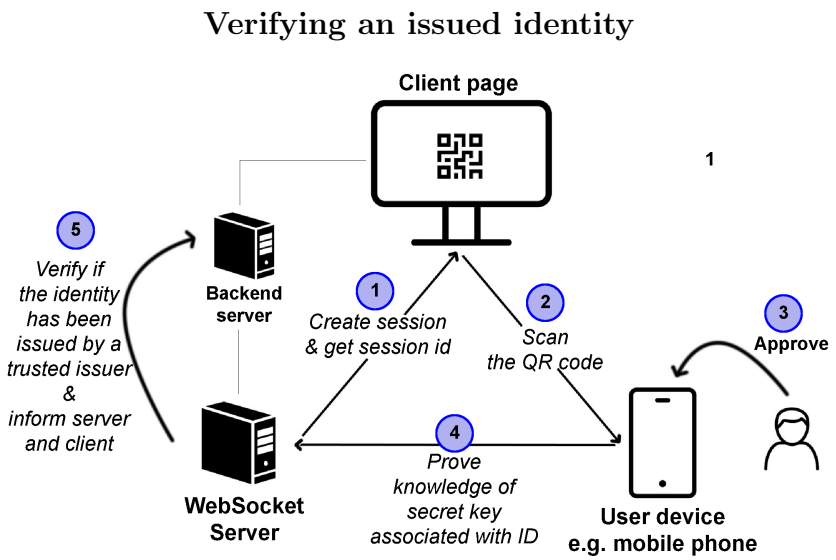


Figure 6.3: Visualisation of how a traditional web page can authenticate a user's identity. Source: Own figure.

between the authentication methods are primarily the use of blockchain and ease of use (e.g. multi-device and requirement of a mobile phone).

6.4.4 Issuing an Identity

We have described the system as dynamic and self-serving. This means that anyone and anything can issue an identity. This is done through the Fiat–Shamir scheme. Essentially, each issuer would have two large primes numbers that are kept secret, the product of these two are kept on the issuer's blockchain identity, i.e. smart-contract, so, when an identity holder claims to have been issued an identity, the verifier can check the existence of the product of the two large prime numbers within the issuers smart-contract or $\mathsf{deeID}_{\text{contract}}$.

Below we go through an example of how an identity is issued by any other participant on the network. The first box states the requirements and the necessary steps required before two participants can create identities for each other. In our implementation we created a flask (Python based) app that allowed a user to create an identity. Therefore, most organisations and current systems are able to do so without much development overhead.

In our implementation we created a Python function (described as f throughout the chapter)

that any developer can use to transform an identity string to a Fiat–Shamir public-key given the string, indices and n.

Example of Issuing an Identity

1. **A:** Hand over the required identity bits, $I_1 = \{$Fullname, DoB, deeID, City of Birth$\}$
2. **B:** Add a random string to I_1, and let that be I hereafter.
3. **B:** Verify the identity of A (e.g. face to face by looking at passport etc.)
4. **B:** If successful, create the credentials as follows:
   (a) Create the public-keys: $v_j = f(I, j)$ where v_j is the public-key, f is some pseudo-random function (*Construction of such a function is explained in [89]*) and j are small random values; to create the public keys we pick k distinct j values for which v_j is a quadratic residue $\mod n$, i.e. solution exists for $x^2 = v_j \mod (n)$
   (b) Calculate the secret keys for A by taking the smallest $s_j = \sqrt{v_j^{-1}} \mod n$.
   (c) So the set of k public keys and associated j values would give us two sets of values: $V = \{v_{j(1)}, v_{j(2)}, \cdots, v_{j(k)}\}$ and $J = \{j_1, j_2, \cdots, j_k\}$
5. Now B can hand over $\{I, J, S, V\}$ to A. V is technically not required because they can re-calculate it themselves if they have I and J.

When an identity is issued, the state of the blockchain would not change; therefore there is no blockchain related costs associated with issuing new identities. The issuer would have the product of p and q, n, within their smart-contract already, if not then the issuer would have to create a transaction and place n within their `keys` array on the smart-contract, in this scenario there is an associated cost with this transaction.

6.4.5 Verifying a Legal Identity

By verifying a true identity, we mean that we are concerned with the actual real-life identity of the person. You can think of this as checking one's passport to get the real identity of the person we are concerned with. So, we have covered how you can issue an identity to someone, now we will go over how you can verify that identity. To give context, this would typically happen when one is applying for a credit card, insurance policy, or car finance online. Figure 6.3 provides an overview of how we will proceed. We assume that the verifier (the backend and WebSocket servers) will have a set of trusted identity issuers deelD addresses. Therefore, when they ask for an identity, they will ensure that the provided identity is provided by one of the trusted issuers. The actual authentication happens with the web server and the user's device (mobile phone); thus, for this to happen, the user has to first interact with the client, capture the required information, and then carry out the identification protocol with the WebSocket server. Upon the completion of the verification the server can update the client and carry out with the intended functions (e.g. registration and creating a session for the user).

6.5 Evaluation

This section evaluates the deelD system using the framework proposed by Bonneau *et al.* in *The Quest to Replace Passwords* [30]. The framework assesses authentication schemes across three key dimensions: usability, deployability, and security. Figure 6.4 visually compares deelD with other authentication schemes, and here, we systematically analyse how deelD meets or falls short of each evaluation criterion, acknowledging its strengths and limitations.

6.5.1 Usability Benefits

1. **Memorywise-Effortless**: There is no need for users to memorise passwords or other secrets; deelD relies on a mobile phone device to store cryptographic secrets. This reduces cognitive load compared to traditional password systems. This benefit is similar to

WebAuthn/Passkeys or password managers. However, the users may need to use a PIN to unlock their device.

2. **Scalable-for-Users**: deelD supports unlimited credentials and users can have multiple identities (issued from different organisations) without additional complexity. With the current system, there is a cost associated with transactions with the blockchain. Whilst this is a limitation of our method it is a technical challenge to overcome in future work. This benefit is similar to WebAuthn/Passkeys in regards to authentication.

3. **Physically-Effortless**: Authentication and identification with deelD requires scanning a QR code and pressing a button to confirm, a process that is physically straightforward and comparable to tapping a contactless card. There is a reliance on a mobile phone device and having it with you.

4. **Easy-to-Learn**: deelD is designed to be intuitive with minimal cognitive overhead. The actions and processes are familiar, such as scanning QR codes. The blockchain element is not easy to learn and most users are not required to learn or understand it; it can be abstracted away. Identity providers can provide onboarding and initial training if required.

5. **Efficient-to-Use**: Authentication and identification is streamlined, requiring only a QR code scan and a single button press, making it faster than typing passwords or answering security questions. Compared to multi-factor authentication methods that involve multiple steps, deelD offers a more efficient user experience. Network latency during blockchain queries may introduce minor delays.

6. **Infrequent-Errors**: deelD is reliable and deterministic. However, users would require their mobile phone device with them. Blockchain network issues might introduce errors or inconvenience.

7. **Easy-Recovery-from-Loss**: This is a significant limitation of deelD; there is no robust recovery mechanism. If a user loses their mobile device then they would not be able to recover their identity without backups. Users would need to get their identity re-issued from their providers.

6.5.2 Deployability Benefits

1. **Accessible**: The aim is to match or go beyond password's accessibility. Mobile phone devices are ubiquitous and most users have access to these devices with capability of securing cryptographic secrets on the hardware. Accessibility is therefore high.

2. **Negligible-Cost-per-User**: Blockchain transactions can incur charges, such as for issuing identities or updating contracts. The user requires a mobile phone which can be costly. Beyond these costs, all identity verifications and authentications have no costs associated with them.

3. **Server-Compatible**: Compatibility with existing tech stacks were a key requirement in the design stage. deeID requires no specialised server-side infrastructure unless a blockchain needs to be installed. Most organisations would be able to implement the password-less authentication feature easily.

4. **Browser-Compatible**: Do users need client change or additional software? It would be quasi-compatible if it reliance on common plugins like Flash. deeID is a compatible with most modern browsers.

5. **Mature-Technology**: deeID uses mature technologies as the building block, however in itself it is not a mature technology or widely adopted. Maturity and adoption of this deeID requires recognition amongst large technology organisations that control distribution.

6. **Non-Proprietary**: This is where anyone can implement or use the scheme without royalties (not protected by patent or trade secrets). deeID is designed and developed on open technologies such as Ethereum, the Fiat–Shamir cryptosystem, and other widely available client-server technologies.

6.5.3 Security Benefits

1. **Resilient-to-Physical-Observation**: No secrets are displayed or entered during authentication. The QR code displayed can be timed or randomised at a rapid rate. All

operations occur on the user's device. The zero-knowledge properties of the protocols used means no secret information is revealed.

2. **Resilient-to-Targeted-Impersonation**: Skilled or well-resourced malicious actors would not be able to impersonate the user unless the user's mobile phone device is stolen and the attacker can access it. deeID is resilient against phishing attacks and other social engineering attacks.

3. **Resilient-to-Throttled-Guessing**: Secrets are not stored on centralised servers, they are held on the user's device. It is computationally infeasible to attempt to guess the users's Fiat–Shamir secrets, especially with large key sizes.

4. **Resilient-to-Unthrottled-Guessing**: Similar to throttled guessing, this is also unfeasible due to the nature of the cryptosystems used and lack of credential centralisation.

5. **Resilient-to-Internal-Observation**: If a user interacts with a service on the client then they are secure against attacks, such as the Man-in-the-Browser malware. The users do not interact with the cryptographic secrets on the mobile phone, if stored on the secure enclaves then they are unobservable.

6. **Resilient-to-Phishing**: The zero-knowledge nature of deeID 's authentication ensures that no secret information is shared with verifiers. The operations on the mobile phone, where a service is authenticated, ensures that the user is unlikely to be phisehd.

7. **Resilient-to-Theft**: The system is resilient-to-theft if a physical object cannot be used for authentication or identification by another user. It would be quasi-resilient-to-theft if protected by a PIN without rate control. deeID is quasi-resilient-to-theft as the mobile phone device is likely to have a PIN or biometric. Future work can focus on making this aspect more secure.

8. **No-Trusted-Third-Party**: deeID uses a trusted third party to create an identity on the blockchain. Beyond this initial setup, deeID does not use any other trusted third parties.

9. **Requiring-Explicit-Consent**: Authentication and identification requires an explicit

action from the user, such as scanning a QR code or pressing a button. This ensures authorised access and accidental or drive-by identification.

10. **Unlinkable**: If the user is carrying out identification, then they are linkable due to their unique identity. However, if they carry out authentication (without the use of the blockchain), then it will be unlinkable, unless the services track the users by other means.

6.6 Discussion

One of the primary strengths of deeID is in its extension of the Fiat–Shamir identification protocol to accommodate multiple identity issuers. Unlike traditional systems that rely on a single trusted authority or on centralised certificate authorities, deeID's use of blockchain as a DPKI enables a dynamic trust network where any organisation—be it a bank, government, or university—can issue verifiable credentials. This decentralisation eliminates single points of failure and reduces the risk of systemic compromise, as demonstrated by real-world incidents such as the SEC Twitter/X account hack via SIM swapping [112] or the deepfake-enabled US$25 million fraud [160]. By replacing shared secrets with zero-knowledge proofs, deeID ensures that no reusable information is exposed during verification, significantly reducing the attack surface for impersonation and replay attacks.

deeID has taken a privacy-by-design approach. By employing zero-knowledge proofs, the system ensures that verifiers receive only the information necessary for authentication/identification, protecting users from oversharing sensitive data. This aligns with growing regulatory demands for data minimisation, such as those outlined in the GDPR and similar frameworks. Additionally, the ability to support multiple devices and keys enhances usability, allowing users to manage their identities across various contexts without being locked into a single vendor or ecosystem, unlike federated identity systems.

When compared to existing authentication systems like WebAuthn and Passkeys, deeID stands out for its focus on identification rather than solely authentication. While WebAuthn effectively addresses password-less authentication through public-key cryptography, it does not inherently

support the complex, multi-issuer identity verification scenarios that deeID targets. For instance, WebAuthn is primarily designed for single-site authentication, whereas deeID enables cross-organisational identity verification through its blockchain-based trust network. This makes deeID particularly suited for applications requiring robust identity verification, such as financial services, healthcare, or government services, where multiple trusted issuers are involved.

Traditional public key infrastructures (PKIs) rely on centralised certificate authorities, which introduce vulnerabilities such as single points of failure and potential coercion. In contrast, deeID's blockchain-based DPKI distributes trust across a decentralised network, reducing reliance on any single entity. Compared to other blockchain-based identity systems like Namecoin [170] or Certcoin [81], deeID's integration of the Fiat–Shamir protocol with a multi-issuer framework provides a more flexible and scalable solution for real-world identity management. Additionally, deeID's emphasis on zero-knowledge proofs distinguishes it from systems like Blockstack [27], which may not prioritise privacy to the same extent.

Preventing SIM swap attack is another use case of deeID. SIM swap attack is where an attacker uses social-engineering (operator error and weakness) to convince the user's mobile carrier to swap the SIM to the attacker's SIM. The consequence of this is that the attacker can hijack accounts that use two-factor authentication (2FA). Notable hacks include the hijacking of celebrity Instagram accounts [18] and individuals losing millions of pounds of cryptocurrency as their exchange accounts were linked to their mobile phone number using 2FA. This is a growing threat that is leading to more organised crime [79]. Preventive techniques include using a PIN or better 2FA [99]. However, these are patchy and preventive measures; a standardised solution through the use of decentralised identity like that of deeID across mobile network carriers would be a better solution. Explicitly linking the user's deeID to their SIM and having a hardware linked crypto keys - meaning that it is physical and malicious attackers cannot carry out their attack without access to the physical elements. When the SIM is linked to the user's deeID (do this when the user acquires the SIM), the attacker would have to identity themselves which is far more difficult that knowing the user's name, address and date of birth.

Mitigating identity theft presents a compelling use case for deeID, as current credit verification

systems enable attackers to obtain credit cards using only a victim's name, address, and date of birth. This vulnerability stems from credit agencies operating on a "pull" model, where third parties can access credit reports by providing basic personal information, rather than a "push" model requiring explicit authorization from the identity owner. Under a deelD-based framework, credit agencies such as Experian [68], Equifax [65], and TransUnion [243] would link customer data to individual deelD credentials, transforming credit inquiries into authorization requests that require explicit consent from the identity owner before releasing information. While the economic implications of such systemic changes present significant implementation challenges, establishing this intermediary authentication layer would provide technically sophisticated users with substantially greater control over their personal credit information and enhanced protection against unauthorized access.

6.7 Limitations

We have not completely eliminated trusted third parties (identity issuers) from our system. Can we ever fully remove them? We believe not. However, our system requires trusted third parties only when identities are first created. Future work could consider reputation systems for the identity issuers.

The identity issuers are currently unable to revoke an identity. If an identity is compromised or the user has lost their secrets related to that identity then the identity issuer should be able to revoke that identity so no other party can use it.

There is no feature to *recover an identity*. If a user loses their mobile phone device with their secrets then they won't be able to get prove their identities any more unless they have backups. Currently, there is no equivalent of "Forgot my password".

Mobile device dependence creates usability barriers and single points of failure. We do not run a blockchain node on the device and would therefore need to have a trusted node to communicate with. Future work could focus on dedicated hardware to carry out the main deelD functions.

When an identity is proved, the user reveals all *identity bits*. This limitation means that the user is oversharing information and leads to privacy concerns. Future work could consider selective disclosure of identity bits (such as date of birth or just birth year if they only need to prove their age).

Future Work: Selective Disclosure

Treating services as adversarial dictates that we must minimise shared information bits. While Fiat–Shamir zero-knowledge cryptosystems ensure that secrets themselves are not disclosed, they still require revealing various attributes of our identity. However, in real-world scenarios, a prover often needs to disclose only some attributes rather than all. For example, someone proving that they are German does not need to reveal other identity attributes. This is where *Selective Disclosure* of identity attributes becomes crucial. We can combine the Fiat–Shamir [72] cryptosystem with a Merkle tree [151] construction to selectively reveal certain attributes while keeping others hidden.

Well-resourced malicious actors and nation-states threaten blockchain networks. These attackers can destabilise networks or disable them entirely by launching coordinated attacks such as 51% attacks, eclipse attacks, or distributed denial-of-service campaigns.

We developed our implementation on Ethereum, which relies on a Proof-of-Work consensus mechanism. However, Proof-of-Work creates two significant problems: it consumes an excessive amount of energy and requires us to trust miners whom we cannot verify. For identity management networks, we need better alternatives. Proof-of-Stake offers one solution, while Proof-of-Reputation [276] provides a more experimental approach. Both schemes would serve identity management networks more effectively than Proof-of-Work, as they address its core limitations while supporting the specific requirements of identity systems.

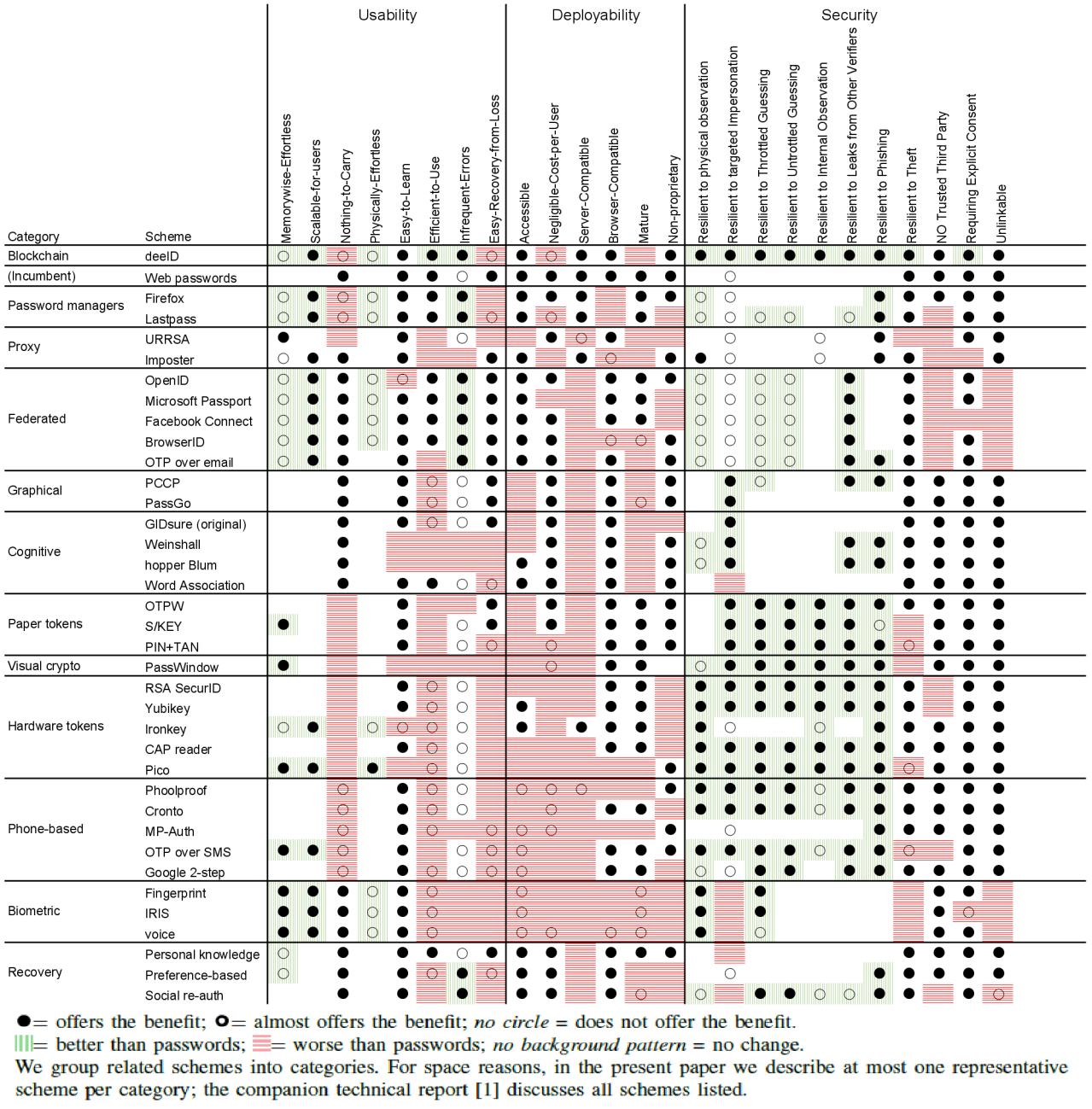
6.8 Summary

In this chapter we have broken down the idea of identity, its representation and definition and explored how one can one uniquely identify themselves. We have compared our blockchain-based

identity system with existing systems and typical password systems. We have argued that deeID provides better security than federated, mobile phone-based, and hardware-based systems which in turn provide better security than passwords. Deployability is where most schemes fall back on compared to passwords. However, this is something that cannot be beaten due to 'knowing something' is more deployable than all other systems. Given that our system can inherently be a password manager too, it therefore strikes a better balance between the three major comparison factors of usability, deployability and security.

deeID can also be built on top of other blockchain technologies. Our novel contribution is improving upon identity representation through smart-contracts, allowing the blockchain as a cryptographic key management and a trusted-source of identities and providing a mechanism for people on the network to provide identities to one another. This was achieved by allowing the Fiat–Shamir scheme to have multiple identity issuers through the blockchain. This is most useful in the real world where multiple organisations can issue identities to the same individual.

Future work can be separated into four different pillars, (1) General algorithm optimisation of the Fiat–Shamir cryptosystem. (2) We believe that an identity system such as the one proposed in this chapter could become an integral part of future blockchain systems that utilise Proof-of-Authority consensus schemes. A deeID based Proof-of-Authority that is Byzantine Fault Tolerant with some quantifiable method for reputation to manage the network, (3) We have skewed our approach towards human identity, the definition can be debated, however one can extend the scheme to go beyond humans and make it a universal identity system for entities that include IoT devices, animals, fictional entities, and even cyborgs with vision of a decentralised galactic identity network. (4) Lastly, our design has been blockchain implementation agnostic, therefore one can think about interoperability of the scheme and how identities across networks can interact with one another.



●= offers the benefit; ⭘= almost offers the benefit; *no circle* = does not offer the benefit.
||||= better than passwords; ≡= worse than passwords; *no background pattern* = no change.
We group related schemes into categories. For space reasons, in the present paper we describe at most one representative scheme per category; the companion technical report [1] discusses all schemes listed.

Figure 6.4: Visual evaluation of various schemes and our new blockchain-based scheme (deeID).

Chapter 7

FormL3SS:

Protecting User Secrets on the Client

In Chapter 3, we found that Man-in-the-Browser (MitB) malware (also known as *form grabbers*, *skimmers*, or *magecart-style attacks*) can silently exfiltrate sensitive user data from web forms. As users interact increasingly with browsers and websites, the risk of sensitive data such as passwords and credit card details being stolen by MitB malware and form grabbers increases. These threats can cause serious financial and information loss. To counter such risks, we propose an alternative security measure: an out-of-band mobile phone system that protects against compromised browsers and malicious form manipulation. This chapter presents our system FormL3SS, which includes a standardised protocol for secure information requests and integrates blockchain technology for identity verification. We showcase the effectiveness of the proposed system in securely transmitting data to a trusted server, demonstrating a fusion of traditional security techniques with blockchain systems.

7.1 Introduction

Form grabbers and Man-in-the-Browser (MitB) malware attacks are an effective method of stealing sensitive information and web traffic data. They can be used to steal credit card details,

personal information and passwords, which are then auctioned off [62] or directly used for fraud and the financial benefit of the attacker. Once the user's machine is infected, there is little modern browsers can do. The Trojans and form grabbers act silently as the user interacts with the web page. One method of circumventing compromised browsers and operating systems is by using another device to submit the data (an out-of-band system).

In this chapter, we are concerned with a secure method of submitting data (typically done via a web form) by the user to the web server (which is hosting the website). We present a design, called FormL3SS, and a proof-of-concept implementation of an out-of-band form submission technique that can securely transfer required data (e.g., payment information) to the trusted server hosting the web application. We design and implement the proposed system on top of a blockchain-based identification and authentication system, deeID. deeID already offers passwordless and out-of-band authentication. Moreover, deeID offers the ability to send messages to the user via a secure communication channel.

Form grabbers and MitB attacks are channelled through various methods. Here, we focus on two: Trojans and malicious third-party JavaScripts injected into the page.

MitB Trojans or financial malware silently infect a device and monitor outgoing communication before it is encrypted. Targeted Trojans attack online banking and ecommerce websites. ZeuS and SpyEye are two famous examples of such Trojans. Recent examples such as BackSwap [114] tailor their code to target specific banks, in this case Spanish banks.

Trojans are inherently form grabbers, but in this context we will refer to form grabbers to scripts that infect third-party libraries. It is common for most web applications to use third-party libraries, mostly JavaScript, to enhance the functionality of the web page. Compromised libraries once loaded into a web page can scrape forms and steal information as they are entered into HTML forms. The most prominent example of such form-grabbing attack is the British Airways (BA) 2018 breach [98] where 380 000 customer payment details (including CVV numbers) were stolen as they were typed into the page between Aug 21 and Sep 5. The attackers did not compromise the BA's servers, but managed to compromise a third party library called `modernizr.js`. The attackers injected 22 lines of JavaScript code [106] (as shown in Listing

```
window.onload = function() {
    jQuery("#submitButton").bind("mouseup touchend", function(a) {
        var n = {};
        jQuery("#paymentForm").serializeArray().map(function(a) {
            n[a.name] = a.value
        });
        var e = document.getElementById("personPaying").innerHTML;
        n.person = e;
        var t = JSON.stringify(n);

        setTimeout(function() {
            jQuery.ajax({
                type: "POST",
                async: !0,
                url: "https://baways.com/gateway/app/dataprocessing/api/",
                data: t,
                datatype: "application/json"
            })
        }, 500)
    })
};
```

Listing 2: Javascript code that was injected into the British Airways website.

2) that sent form data as they were submitted to a rogue phishing server. The financial and economic cost of these attacks should be taken seriously. Not only do victims face financial losses, but services providers such as British Airways (BA) can also be fined for putting their customers in danger. BA faces a £183 million fine by the UK regulators (ICO) [32].

With an increasing number of individuals using the Internet for online banking, shopping, etc. The pool of potential victims is only increasing in size. The importance of protecting users whilst enhancing their experience is evident, and thus we shall propose a method to circumvent MitB type attacks and allow the user to transact securely even if the browser or the operating system is untrustworthy compromised.

7.2 Design

The goal of the proposed system, FormL3SS, is to circumvent compromised operating systems, browsers and malicious third party JavaScript libraries (form grabbers), thus creating a secure link between the user and the trusted server (the server hosting the website that the user is interacting with) in order to send sensitive data (e.g. payment information). We are countering

two types of attacks: 1) Man-in-the-Browser Trojans such as *ZeuS* and *SpyEye* (also known as financial malware). 2) Form grabbers: Malicious third party libraries that scrape the web page and send the data to a controlled server. Malicious browser extensions fall between those two forms of attack. Therefore, we assume that all interactions and data on the web page displayed on the infected client can be manipulated and altered. The proposed system, FormL3SS, is shown in Figure 7.2. The communication flow between the different components of FormL3SS is show in Figure 7.1.

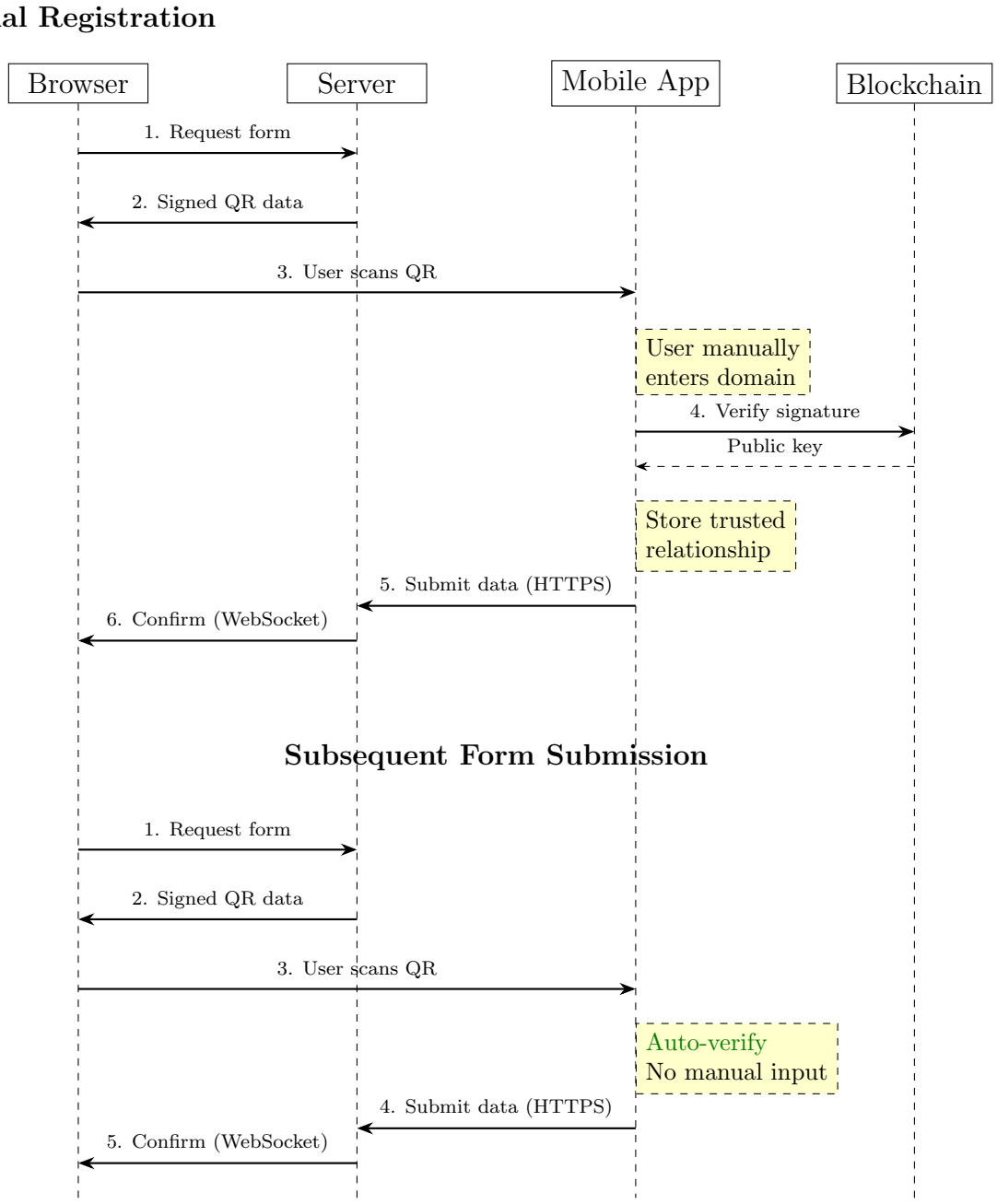


Figure 7.1: FormL3SS communication flow. Here we see how the different components interact with each other and the two scenarios if the user has interacted with the service before.

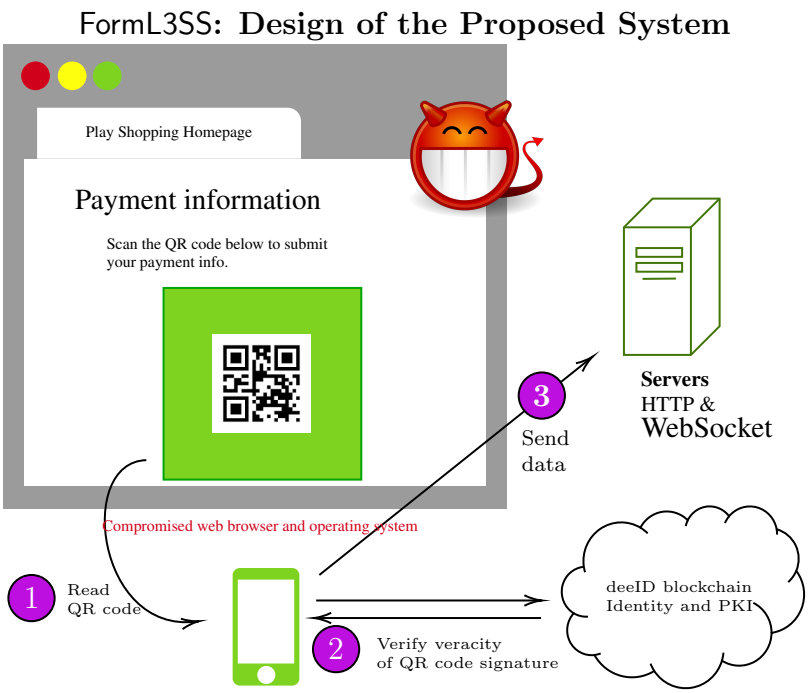


Figure 7.2: Visualisation of the proposed system. The web server communicates the required information (to be sent to the server by the user) to the client which is then displayed as a QR code. The mobile app scans the QR code, verifies the veracity of the signature. If verified, the mobile app sends the required data directly to the server.

The server is assumed to be secure. Therefore, the data sent to the client or the mobile phone device is assumed to be valid and verified via a signature scheme. The user's device (OS and browser) is assumed to be compromised and insecure. So, we are using an out-of-band system to send sensitive information to the trusted web server. A mobile phone is a common device that has the ability to perform the necessary cryptographic and networking operations. Thus, a mobile phone is used to send the data securely to the web server. The data from the client (web browser) is transferred to the mobile phone via a QR code. We have built on top of the deeID system because it already provides password-less and out-of-band authentication and thus this extends deeID for out-of-band transfer of other data.

We ensure that the QR code is not manipulated by the compromised browser and client-side code. We do this as follows: a) the trusted server (hosting the website) signs the data, b) the user scans the QR code, c) the user then types in the website's domain into the mobile phone app upon being asked for it, d) the domain names are cross-checked with the scanned one from the QR code, e) the mobile phone app can now connect to the WebSocket server (given via the QR code) and ask for the authenticity of the signature, f) upon successful verification, the user

will send the data. Asking the user to manually enter the domain name ensures there are no phishing attacks in play.

If the user has already registered with the website using deeID then the *link* created upon registration, we assume, has already persisted on the mobile phone. The mobile phone application will use this to protect against phishing attacks. The website's deeID blockchain contract should also contain the domains associated with the website's deeID which the mobile phone app can cross-check with.

Just like humans, fictional entities like websites can also have verifiable identities. This is most important if the website is supposed to have a legal identity. If the user has come across a website, then they can attempt to get proof of identity of the website if another person or organisation has verified it. Then it is up to the user to trust the person or organisation that has verified the website's identity. This means that an attacker has to fake a web server, an identity contract and an identity proof to carry out a successful attack. These are strong barriers that the attackers have to overcome.

FormL3SS consists of the following components:

- **Frontend**: JavaScript code is used to transform the server data into a QR image. The client is treated as insecure and therefore the manipulation of the data is verified by the mobile phone application.
- **Server**: A backend is developed in Python to receive client requests, handle communication with the mobile app, and a WebSocket connection is used so that the client can be updated upon confirmation (from the server) of receiving the correct data from the mobile phone. The server is treated as secure.
- **Mobile app**: We build on top of the deeID application to add out-of-band form transactions. The mobile app is treated as secure.
- **Blockchain (deeID)**: In Section 7.4, we will see alternative designs and solutions that will utilise the blockchain and the deeID functionality. We use the messaging function, the proof of identity, and its PKI functionality.

Form Types. For payment and personal information, the server does not have to send a full HTML form to be filled in by the user. For payment and personal information, we have standardised it so that the server only has to identify a `form_type` in the QR code (or any medium of message). For payment the `form_type` is `card_info` and for personal information the `form_type` is `pers_info`.

For `card_info` the user will return an array containing the card type, number, expiry date and the CVV number. For `pers_info` the user will return an array containing the user's first name, surname, address (line 1, line 2, city, country and postcode), and date of birth.

Other forms will require a `form_type` being `custom`.

Custom Form Type. The custom form type indicated by `custom` currently supports the following input types: `text, number, email` and `date`. The advantage of this is that the added functionalities on deeID will do some form validation before being sent over to the trusted server. This will minimise the communication steps between the mobile phone and the trusted server.

7.3 Implementation

We created a proof-of-concept that demonstrates the capabilities of the idea. The acting server was created using the `flask` [75] web framework. The front end of the dummy website used HTML and JavaScript. The frontend is connected to another WebSocket library built on Python. The mobile phone functions extended those of deeID, with our proof of concept the mobile phone application can read a QR code that has a `type` called `deeIDForm`.

We will now go into further detail for the implementation and some of the data structures that are passed around. Figure 7.3 shows the interface of the acting payment website and the mobile phone after scanning the QR image.

Backend Functionalities. The acting server (trusted web server) in our implementation picks up a user journey where the input of a user is required. In our context, we want the user

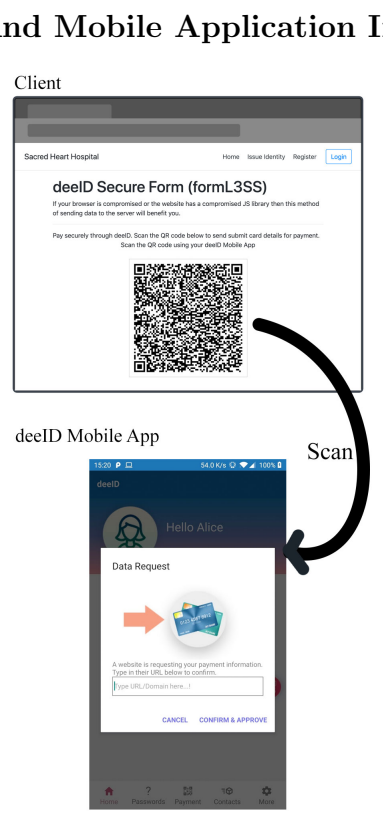


Figure 7.3: Interface design of the web page and the mobile phone app. Upon scanning the image the mobile app will ensure the QR code's data is not manipulated by verifying the domain and its cryptographic signature.

to send a debit or credit card detail to the trusted server. Listing 3 shows the data structure sent from the server, it is requesting payment information from the user.

```
qr_msg = json.dumps({
        'type': 'deeIDForm',
        'domain': '',
        'uID': '', #Unique client & WS connection ID
        'form_type': 'card_details',
        'y': y, #Variable to link to a user session
        'exp_time' : '',
        'deeID': deeID, #Websites deeID address
        'sig' : str(sig), #Signature
        'ws_url': ws_url, #WebSocket URL
    })
```

Listing 3: Data structure and example of data behind the QR code. These data are signed and sent from the trusted server hosting the application the user is interacting with.

Frontend Functionalities. The frontend functionality is simply a QR code encoder. The client receives the data (signed) from the server and the JavaScript will create a QR image out of that data.

7.3.1 Mobile Application Functionalities

Upon reading the QR code from the compromised computer, the mobile application will attempt to verify the content and its signature. At this point the threat is that malicious actors could have a phishing server setup and have completely changed the content of the QR image to point to the phishing server. We mitigate using the following methods.

- If this is our first interaction with the server then we ask the user to type in the domain of the site they have visited; then cross check it with the one in the QR image. The mobile app will then verify the public-key from the server.
- If this is not our first interaction with the server then we may verify the identity of the server that signed the QR code with the identity we have associated with the server.

Note, phishing threats are minimised through the deeID ecosystem if we have already established

a connection with a service. This relationship is stored and any incoming communication is always verified with the stored information.

7.3.2 WebSocket Functionalities

The purpose of the WebSocket server is to create a dynamic and user friendly application; that is to connect the client, server and the mobile phone. It is to allow each device to be updated upon state changes in other devices.

7.3.3 Custom Form

We implemented a simple custom form functionality. This functionality allows the developer to request information that is not in the standardised form requests (currently payment and personal information). In the request data we have a `custom form_type`. This also requires another field named `form`; which is an array of the form fields required. As you can see in the Listing 4, we have requested a secret question and answer from the user. This then allows the mobile application to display the right form inputs as seen in Figure 7.4. This figure shows the two form fields requested by the trusted web server.

```
{
    'type': 'deeIDForm',
    'form_type': 'custom',
    'form': [
        ['text', 'Secret question', 'sec_ques_1'],
        ['text', 'Answer', 'sec_ans_1'],
    ],
    ...
}
```

Listing 4: Snippet of the QR code's data when a custom form is implemented. An extra `form` field to insert an array of required form fields.

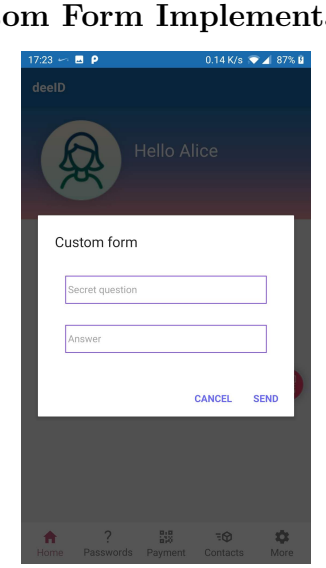


Figure 7.4: Custom form interface design. Implementation example showing a server requesting two pieces of information.

7.4 Alternative Designs

In Section 7.2 we described a simple design based on the objective of circumventing MitB threats and form grabbers. In this section we will explore different designs and architectures that have different benefits and can be more suited to a given web application. These alternative designs utilise more of the deeID functionalities: messaging server (being able to send messages to the user's mobile phone without using SMS), encryption and alternative keys stored on the user's deeID and the deeID mobile phone app itself.

The following factors determine the suitability of a design:

- Complexity of the form: QR code might not be suitable for a complex form.
- User and web server relationship: Is the user already registered with the trusted server?
- Whether the user has coupled a messaging server with their identity contract.

7.4.1 User is Registered with the App and has a Messaging Server

If a user is already registered with the website of interest and they have a messaging server associated with their deeID contract then we can use this method.

Remark: We assume the user is able to use the password-less and out-of-band authentication method with the server.

The server, on request from the client, can send a secure message to the user's mobile phone application via their messaging server requesting a form to be filled in and sent back. Given that the user is already registered with the service they can verify whether it is a phishing threat (cross-checking domain, public-keys used to sign a message, etc.) and its veracity.

7.4.2 User is Registered with the App and Has No Messaging Server

Continuing from previous design but assuming there is no messaging server on the user's deeID contract. The trusted server (hosting the website) can encrypt the QR code data, send it to the client where it can be read by the deeID mobile phone application and be decrypted. From hereon the process continues as per the first design.

7.4.3 Send WebSocket Server and Request Form from Server

If the form (assuming a custom type form) is complex and thus cannot be fully transferred via a QR code; or perhaps the developer is looking for a more dynamic approach to form filling (e.g. the form is split into sections where an answer to one section will determine the type and content of consequent sections) then this method can be used.

Instead of sending all of the form in the QR code, we only send the URL of the trusted WebSocket server (belonging to the website of interest). Upon being verified by the user on their deeID mobile phone application, the user will begin requesting the form from the WebSocket server. At this point the process is a simple back and forth conversation between the two trusted devices.

7.5 Usability Evaluation

We evaluate the usability of FormL3SS using the framework by Bonneau *et al.* in *The Quest to Replace Passwords* [30]. We used this method to evaluate deeID in the previous chapter. We utilise the the usability section of the framework for FormL3SS as well; the deployability and security aspects of the framework are more authentication focussed so they are not used here.

1. **Memorywise-Effortless**: FormL3SS is built around deeID from the previous chapter and therefore stores secrets and sensitive data on the mobile phone device. This eliminates the need to memorise information. Users may need to use a PIN to unlock their device or app, however, this is common operation for most users and with the advent of biometrics it is not a significant usability issue.

2. **Scalable-for-Users**: FormL3SS can store many sets of data on the mobile phone device. Scalability is not an issue for the user or the system itself.

3. **Physically-Effortless**: Authentication involves scanning a QR code and confirming submission, a straightforward process comparable to tapping a card, though it assumes consistent carrying of the mobile device.

4. **Easy-to-Learn**: The interface is simple and familiar; it involves scanning QR codes or following simple instructions on the device.

5. **Efficient-to-Use**: The process is quick. The network operations would often take few seconds on most LTE or broadband networks. The user does not need additional software on their client, only the mobile app on their mobile phone device.

6. **Infrequent-Errors**: Most of the functions are deterministic and environmental factors do not impact it. QR code scanning might introduce some source of errors (e.g. if the mobile phone camera is unable to read the QR code), however, with most modern hardware we do not foresee this being an issue.

7. **Easy-Recovery-from-Loss**: This is a limitation of the current design and implementation. FormL3SS in its current form has no recovery methods.

7.6 Discussion

Our objective has been to circumvent threats from Man-in-the-Browser malware and form grabbers. Specifically, to deny them from exfiltrating use secrets. These types of attack have been around for more than a decade, and they are here to stay. They are becoming much more sophisticated in targeting their victims and evading defences. Targeted attacks against banks and even cryptocurrency exchanges [164] using a mix of techniques are becoming more common now. MitB Trojans and form grabbers have primarily targeted web-based payment services and hence are commonly referred to as banking or financial malware. The primary goal of MitB and form grabbers is to essentially steal information that is being typed into a form or turn the user's actions against them (e.g. sending funds to the attacker's bank account if the user is attempting to transfer funds).

It is clear that the threat from MitB and form grabbers is significant. The British Airways case is an example of the scale of the threat. We can defend against these threats through preventive techniques that would guard the user's machine from being infected, and detection techniques that would detect and eliminate the threat. However, our method has been to circumvent the threats, most helpful to a paranoid user. Moreover, whilst it is hard for the prevention and detection techniques to guard against form grabbers, our proposed system can defend the user from such a threat.

The proposed system, FormL3SS, is an out-of-band system that uses a mobile phone device to securely send data to the trusted server that hosts the website.

Prevention and detection techniques have their own benefits. However, an out-of-band system can have superior user experience and security benefits. In the same form, data are always removed from the user experience of a website. With the proposed system that is combined with deeID the user can store information that is commonly required from websites, such as

payment and personal information, on the device (in this case a mobile phone). This stored information can be requested with the proposed standardisation of requests; e.g. payment information is requested by a `form_type` being `card_info`. Standardising data requests is one of the contributions of this chapter. Targeted information such as payment information can be standardised so that all devices know what data to send in order to satisfy the server without the server explicitly stating the input fields required. This reduces communication overhead.

Given that we have built on top of deeID, which in itself is a formless and password-less authentication scheme, the system can also defend against phishing attacks. Moreover, deeID plays an important role, as identification is crucial when dealing with sensitive information online. We consider commonly shared sensitive information to be personal information (name, date of birth, address, etc.) and payment information. These are then interlinked with the identity. Hence, the benefit of using deeID, an identity system with FormL3SS, is that it can increase security with respect to identity theft and financial fraud. Providing the information and also proving one's identity on the fly adds an extra layer of security.

Malware attacks from banking to ransomware attacks are becoming more sophisticated. Collaboration between criminal gangs [235] and targeted attacks [210] enables better penetration and quicker responses to the security guards put up by individuals and organisations. IBM predicts [104] that using cryptocurrency, criminal gangs are shifting to targeted ransomware attacks because they can evade security forces and launder their money more effectively.

FormL3SS shares conceptual similarities with systems such as Apple Pay and Google Pay. Both make it convenient to carry out a function securely. We can take inspiration from these systems to use NFC for example to transmit data at physical points of *sale*. FormL3SS extends the principle of out-of-band secure transmission to arbitrary web form data using a different technical approach suited to the web browsing context. Using QR codes has become a common method of payment in China using the likes of WeChat—clear evidence that this method of transacting PII or payment can be scalable and reach wide adoption.

The proposed system also has its own limitations. Mobile phones are also susceptible to malware attacks. As the use of mobile phone devices increases, they become more of an attractive target

for malicious actors [230]. The fact that the user has to use an additional device to submit the form is an additional burden. However, one can argue that because they are so common it is less of a burden. Also, because the user does not constantly type in the same information, the process of submitting sensitive information is easier and more user friendly. The proposed system used deeID, an experimental technology that generates additional complexities.

7.7 Related Work

As far as we know, there are no other similar works in which a general form submission protocol is used with a mobile phone and a blockchain PKI system. However, there are similar works with respect to circumventing Man-in-the-Browser and compromised browser threats.

Software-based solutions that circumvent MitB threats include the likes of TrustJS [91] and ShadowCrypt [96]. TrustJS enables trustworthy execution of JavaScript inside a browser by using trusted hardware (Intel SGX in their case) and a browser extension. This enables the validation of the input of the form on the client side. ShadowCrypt enables encrypted input/output and runs as a browser extension. Although it is successful in preventing the DOM from containing sensitive data and thus being accessible to other extensions and compromised libraries, it does not prevent keyloggers and other sophisticated Trojans. Privly [204] performs similar functions to ShadowCrypt but also uses a third-party server for encrypted data management.

Fidelius [67] uses additional hardware to completely secure the I/O. In Fidelius, the entire I/O path is protected by having a trusted device between the monitor/keyboard and the compromised computer. The drawbacks of Fidelius are page load times and display response latency, as additional hardware and operations of the keyboard and display affect the user experience. We think that our proposal enhances the user experience by eliminating the input of data that is already securely stored on the mobile phone. Also, since nothing is required on the browser, there is no impact on page load and response rates. Bumpy [146] is similar to Fidelius and can be considered a predecessor to Fidelius.

7.8 Summary

In this chapter, we have discussed the threats from Man-in-the-Browser malware and form grabbers, highlighting the importance of these threats with recent examples. We propose a design to protect the user from compromised browsers and client-side or frontend code.

We have designed an out-of-band solution using a mobile phone named FormL3SS. This system circumvents MitB and form grabbers safely while improving the user's experience whilst interacting with the website. We implemented this design on top of an out-of-band authentication and blockchain-based system, deeID. We believe that the combination of technologies used is a novel contribution. Moreover, a first in proposing a standard messaging format for out-of-band form submission systems. Developers can request payment and personal information without explicitly defining the form; this reduces communication overhead. Other data can be requested using a `custom` tag.

Future work include: website identification using deeID, to add an extra level of security; refining the messaging standard between the trusted server and the user's mobile phone; implementation of the alternative designs that were proposed; and implementing the encryption functions for the transmission of the data.

Chapter 8

Conclusion

8.1 Summary and Key Insights

This dissertation investigates cyber threats to user secrets on the web and proposes methods to defend against them. The work presented in this dissertation makes significant contributions to the field of web security, addressing the increasing threat of cyber crime that hinders the web's function as a key driver of modern communication and commerce. Our dissertation is: **Password-based authentication protocols should be made obsolete, even though strong, human-chosen, and memorable secrets can be created. This is due to the lack of consensus on password strength and policy, the broader spectrum of threats to such secrets that are not considered in current protocol designs, and usability hurdles like the integration of multi-factor authentication (MFA).** Our contribution is primarily through new data and the proposal (through designs and implementations) of novel schemes. The breadth of this dissertation is substantial and somewhat unusual. However, we believe this holistic approach is important in order to understand the complexity of the systems that guarantee the security of our secrets. To this end, the dissertation is characterised by the journey of user secrets, where each study represents a key point in this journey.

One of our objectives was to empirically analyse data breaches to understand how they occur in

the wild and to find opportunities for improvement. We analysed 60 reported data breaches in 2020 and used these insights to lay the foundation for future chapters. We also provide a comprehensive exposition and taxonomy of Man-in-the-Browser malware. This type of malware is fairly easy to develop and is often under-reported, posing a high threat to user secrets. As a result, we further investigate this issue in Chapter 7, where we provide a method that uses an out-of-band device to transmit user secrets with the added advantage of being phishing-resistant. Thus, if the client is compromised, the user's secret will remain safe. We find that the majority of breaches are either due to poor administration by organisations or the exploitation of vulnerabilities in their systems.

In our dissertation, we argue that secure, human-chosen, and memorable secrets can be composed under the right conditions. These conditions, often defined as Password Composition Policies (PCPs), are widely debated, and we showed that most web applications do not follow the standards. To understand what conditions can create such strong passwords, we produced the first human-labelled password dataset. Using this dataset, we demonstrate which empirical passwords can be secure against brute-force and dictionary attacks. PCPs, such as using three random words, are shown to be both secure and usable (i.e., they can be easily communicated and adopted). We also provide a method based on Large Language Models (LLMs) to expedite the labelling process. These models prove to be more accurate and cheaper than their human counterparts for password labelling and decomposition.

Our dataset is also useful for analysing password habits and patterns. It has been well studied that we embed predictable information into our chosen secrets, i.e., passwords. However, limitations in password strength meters, such as `zxcvbn` [261], suggest that we have not fully investigated all the patterns. In Chapter 4, we present a comprehensive set of transformations that can be adopted in password strength meters to better inform users about the security of their passwords. These transformations take into account various linguistic and cultural factors that influence password creation, providing a more accurate assessment of password strength. By incorporating these transformations, password strength meters can offer users more meaningful feedback and encourage the creation of stronger, more resilient passwords.

One way of understanding and improving password strength is through password guessing algorithms. We find that despite the rich body of literature on password guessing, the techniques and algorithms are seldom used in practice, signifying their lack of practicality. We provide a comprehensive exposition of password guessing algorithms in order to address their shortcomings and improve research repeatability.

In Chapter 6 we propose using identity-based cryptosystems that provide a strong form of entity authentication and are zero-knowledge proofs. Using the Fiat–Shamir cryptosystem, we design and implement deeID. We go on to address "identity" on the web. To achieve this, we improve our design and implementation to incorporate blockchain technology, which acts as a decentralised store of identity. We believe this form of identification can transcend human identity and be useful for fictional-entity identity (such as organisations) across the world, as well as machine identity.

We complete the journey on the client side, where we design and implement a system to protect users' secrets from client-side malware. To protect secrets from MitB malware on the client side, Chapter 7 introduced FormL3SS, an out-of-band system that uses mobile devices and QR codes for secure data transmission. FormL3SS mitigates risks from compromised browsers, enhancing usability while preventing phishing and form-grabbing, and is motivated by real-world incidents such as the British Airways breach.

Despite showing that human-chosen and memorable secrets can exist, we argue that they are not scalable, and additional security layers merely make password-based authentication systems less usable. These scalability and usability issues highlight the need for a more radical approach to protocol design. WebAuthn and Passkeys are steps in that direction, but the current implementations lack flexibility and raise privacy concerns.

8.2 Future Work

In the introduction, we addressed the acceleration of cybercrime driven by weak authentication, limitations of shared secrets, and specific attack vectors (e.g., MitB). The dissertation addressed

a wide breadth of issues successfully (password-less authentication, password cracking, client-side malware, and decentralised and zero-knowledge identification). However, significant work remains within each pillar, such as the password cracking algorithms, where more work could be done on mask and conditional guessing attacks. Moreover, accessible AI brings a new form of threat, such as deepfakes.

In the process of conducting this research, we experimented with various other ideas that we believe hold value for further exploration.

8.2.1 Low-cost EEG-based Authentication

Continuing the work on password-less authentication (Chapter 6), we experimented with various other password-less authentication methods, such as EEG-based authentication. Although work has been conducted in this area before, we believe there is a gap in the research—specifically in the EEG hardware itself. The lack of relatively cheap and accurate EEG devices makes this approach highly unreliable and unusable. To address this, we designed a low-cost EEG device based on the OpenBCI system, as shown in Figure 8.1. We used a low-cost and highly available microcontroller—the ESP32. The most important component, the analogue-to-digital converter (ADC), is also the most expensive. We believe that continuing this project would prove highly beneficial for both future researchers and users.

8.2.2 Credential Stuffing Attack (CSA)

CSA attacks are often the most silent form of credential theft and account takeovers. Unlike other breaches, most organisations do not suffer the same level of consequences from regulators. Often, organisations may not even be aware that they have suffered a CSA attack. Whilst commercial tools are available to prevent CSA attacks, we believe that further academic research into defending against these attacks would be highly beneficial.

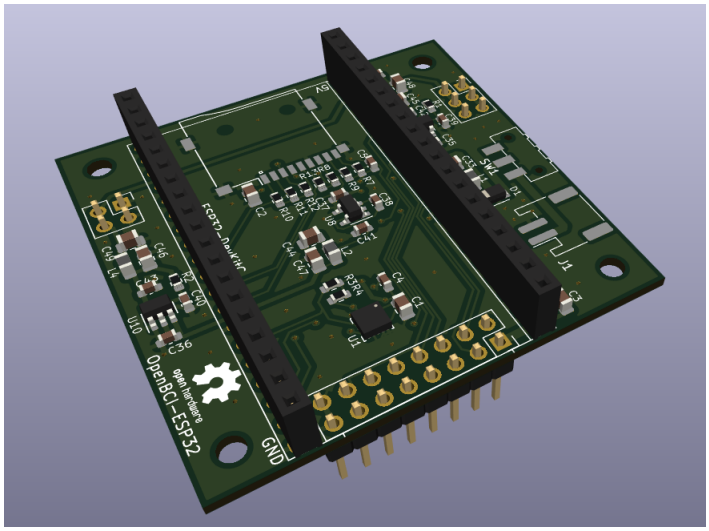


Figure 8.1: 3D render of the AXON EEG device. We designed and manufactured an EEG PCB that uses the ESP32 micro-controller.

8.2.3 Preventing MitB Malware

In Chapter 3, we empirically demonstrated the threat posed by Man-in-the-Browser malware (also known as form grabbers, skimmers, or Magecart-style attacks). Then, in Chapter 7, we designed and implemented a method to circumvent this type of malware. We believe there are ample opportunities for further research on this malware. Little work has been done in creating open-source tools to identify and classify these types of malware.

8.2.4 Security-First API Design

In Chapter 3, we found that poor design and human errors could contribute to user secrets being leaked in their millions. An interesting opportunity to mitigate this issue would be to develop necessary tooling that reduces the likelihood of human errors.

Bibliography

[1] *6,000 Patients Notified About Email Security Breach at Beaumont Health*, en-US, Jul. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.hipaajournal.com/6000-patients-notified-about-email-security-breach-at-beaumont-health/.

[2] Abhliash, *Breaking down the San Francisco airport hack*, en-US, Section: IT Security, Apr. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://blogs.manageengine.com/it-security/2020/04/22/breaking-down-the-san-francisco-airport-hack.html.

[3] S. Abramova and R. Böhme, "Anatomy of a {High-Profile} Data Breach: Dissecting the Aftermath of a {Crypto-Wallet} Case," en, Usenix, 2023, pp. 715–732, ISBN: 978-1-939133-37-3. Accessed: Jun. 8, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/abramova.

[4] L. Abrams, *UK Retailer Sweaty Betty Hacked to Steal Customer Payment Info*, en-us, Dec. 2019. Accessed: Jan. 9, 2021. [Online]. Available: https://www.bleepingcomputer.com/news/security/uk-retailer-sweaty-betty-hacked-to-steal-customer-payment-info/.

[5] L. Abrams, *Barnes & Noble hit by cyberattack that exposed customer data*, en-us, News, Oct. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.bleepingcomputer.com/news/security/barnes-and-noble-hit-by-cyberattack-that-exposed-customer-data/.

[6] L. Abrams, *Chowbus delivery service breached, hacker emails data to users*, en-us, Oct. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.bleepingcomputer.co

m/news/security/chowbus-delivery-service-breached-hacker-emails-data-to-users/.

[7] L. Abrams, *Minted discloses data breach after 5M user records sold online*, en-us, News, May 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.bleepingcomputer.com/news/security/minted-discloses-data-breach-after-5m-user-records-sold-online/.

[8] L. Abrams, *Over 500,000 Zoom accounts sold on hacker forums, the dark web*, en-us, News, Apr. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.bleepingcomputer.com/news/security/over-500-000-zoom-accounts-sold-on-hacker-forums-the-dark-web/.

[9] L. Abrams, *Promo.com discloses data breach after 22M user records leaked online*, en-us, News, Jul. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.bleepingcomputer.com/news/security/promocom-discloses-data-breach-after-22m-user-records-leaked-online/.

[10] S. Almasi, *Passwordninja: Human-labelled and real password datasets*, Dataset, Human-annotated password dataset with linguistic transformations, 2024. [Online]. Available: https://github.com/sirvan3tr/passwordninja.

[11] S. Alroomi and F. Li, "Measuring website password creation policies at scale," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23, Copenhagen, Denmark: Association for Computing Machinery, 2023, pp. 3108–3122. DOI: 10.1145/3576915.3623156. [Online]. Available: https://doi.org/10.1145/3576915.3623156.

[12] E. R. Andrade, M. A. Simplicio, P. S. Barreto, and P. C. D. Santos, "Lyra2: Efficient Password Hashing with High Security against Time-Memory Trade-Offs," *IEEE Transactions on Computers*, vol. 65, no. 10, pp. 3096–3108, Oct. 2016, lyra2, ISSN: 0018-9340. DOI: 10.1109/tc.2016.2516011. Accessed: May 26, 2024. [Online]. Available: http://ieeexplore.ieee.org/document/7377075/.

[13] Android Open Source Project, *Android keystore api*, Security documentation for Android Developers, Android Developers, 2025. [Online]. Available: https://developer.android.com/privacy-and-security/keystore.

[14] Android Open Source Project, *Trusty TEE*, https://source.android.com/docs/security/features/trusty, Accessed: 2025-09-20, 2025.

[15] Apple Inc., *Biometric security*, Apple Platform Security Guide, Apple Support, 2025. [Online]. Available: https://support.apple.com/en-gb/guide/security/sec067eb0c9e/web.

[16] G. Ateniese and B. de Medeiros, "Identity-Based Chameleon Hash and Applications," en, in *Financial Cryptography*, A. Juels, Ed., Berlin, Heidelberg: Springer, 2004, pp. 164–180, ISBN: 978-3-540-27809-2. DOI: 10.1007/978-3-540-27809-2_19.

[17] A. Belanger, *SIM-swapping ring stole $400M in crypto from a US company, officials allege*, en-us, Jan. 2024. Accessed: Jun. 6, 2024. [Online]. Available: https://arstechnica.com/tech-policy/2024/01/sim-swapping-ring-stole-400m-in-crypto-from-a-us-company-officials-allege/.

[18] K. Bell, *Instagram users are reporting the same bizarre hack*, en, https://mashable.com/article/instagram-hack-locked-out-of-account. Accessed: Nov. 29, 2019.

[19] M. Benedikt, Ed., *Cyberspace: first steps*, eng, 7th print. Cambridge, Mass.: MIT Press, 1994, ISBN: 978-0-262-52177-2.

[20] L. Berglund et al., "The reversal curse: Llms trained on "a is b" fail to learn "b is a"," no. arXiv:2309.12288, Sep. 2023, arXiv:2309.12288 [cs]. DOI: 10.48550/arXiv.2309.12288. [Online]. Available: http://arxiv.org/abs/2309.12288.

[21] T. Beth, "Efficient Zero-Knowledge Identification Scheme for Smart Cards," en, in *Advances in Cryptology – EUROCRYPT '88*, D. Barstow et al., Eds., 55 citations (Crossref) [2024-05-23], Berlin, Heidelberg: Springer, 1988, pp. 77–84, ISBN: 978-3-540-45961-3. DOI: 10.1007/3-540-45961-8_7.

[22] L. Bilge and T. Dumitraş, "Before we knew it: An empirical study of zero-day attacks in the real world," in *Proceedings of the 2012 ACM conference on Computer and communications security*, ser. Ccs '12, Ccs, New York, NY, USA: Association for Computing Machinery, Oct. 2012, pp. 833–844, ISBN: 978-1-4503-1651-4. DOI: 10.1145/2382196.2382284. Accessed: Jun. 8, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/2382196.2382284.

[23] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications," in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, argon2, Mar. 2016, pp. 292–302. DOI: 10.1109/EuroSP.2016.31. Accessed: May 26, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/7467361.

[24] A. Bizga, *Medical Information of 233,000 Individuals Exposed after Genetic Testing Lab Hack*, en-US, News, Apr. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://securityboulevard.com/2020/04/medical-information-of-233000-individuals-exposed-after-genetic-testing-lab-hack/.

[25] A. Bizga and 2020, *Walgreens Discloses Data Breach Impacting Personal Health Information of More Than 72,000 Customers*, en-US, Aug. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://securityboulevard.com/2020/08/walgreens-discloses-data-breach-impacting-personal-health-information-of-more-than-72000-customers/.

[26] J. Blessing, R. J. Anderson, and A. R. Beresford, *Keydroid: A large-scale analysis of secure key storage in android apps*, 2025. arXiv: 2507.07927. [Online]. Available: https://arxiv.org/abs/2507.07927.

[27] *Blockstack*, en, https://blockstack.org. Accessed: Dec. 1, 2019.

[28] D. Boneh, H. Corrigan-Gibbs, and S. Schechter, "Balloon Hashing: A Memory-Hard Function Providing Provable Protection Against Sequential Attacks," en, in *Advances in Cryptology – ASIACRYPT 2016*, J. H. Cheon and T. Takagi, Eds., balloon, Berlin,

Heidelberg: Springer, 2016, pp. 220–248, ISBN: 978-3-662-53887-6. DOI: 10.1007/978-3-662-53887-6_8.

[29] C. Bonk, Z. Parish, J. Thorpe, and A. Salehi-Abari, "Long passphrases: Potentials and limits," no. arXiv:2110.08971, Oct. 2021, arXiv:2110.08971 [cs]. [Online]. Available: http://arxiv.org/abs/2110.08971.

[30] J. Bonneau, C. Herley, P. C. v. Oorschot, and F. Stajano, "The Quest to Replace Passwords: A Framework for Comparative Evaluation of Web Authentication Schemes," in *2012 IEEE Symposium on Security and Privacy*, May 2012, pp. 553–567. DOI: 10.1109/sp.2012.44.

[31] J. Bonneau, "Guessing human-chosen secrets," en, password, Ph.D. dissertation, University of Cambridge, Jun. 2012. Accessed: Jun. 8, 2024. [Online]. Available: http://www.dspace.cam.ac.uk/handle/1810/243444.

[32] "British Airways faces record £183m fine for data breach," en-GB, *BBC News*, Jul. 2019. Accessed: Nov. 30, 2019.

[33] *Broadvoice, VoIP service provider, leaks 350 million call logs*, en, Oct. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://hacknews247.com/news/20201016/broadvoice-voip-service-provider-leaks-350-million-call-logs.html/.

[34] M. Broersma, *Warner Music Warns Of Three-Month Payment Card Hack — Silicon UK*, News, Sep. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.silicon.co.uk/workspace/warner-music-hack-347561.

[35] M. Burrows, M. Abadi, and R. Needham, "A logic of authentication," *ACM Transactions on Computer Systems*, vol. 8, no. 1, pp. 18–36, Feb. 1990, Ban, ISSN: 0734-2071. DOI: 10.1145/77648.77649. Accessed: Jun. 8, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/77648.77649.

[36] L. d. Castro, H. Lang, S. Liu, and C. Mata, "Modeling password guessing with neural networks," 2017. [Online]. Available: https://www.semanticscholar.org/paper/Modeling-Password-Guessing-with-Neural-Networks-Castro-Lang/8ef01b61ec9428556ba78465f4098baf7734f613.

[37] J. A. Cazier and B. D. Medlin, "Password security: An empirical investigation into e-commerce passwords and their crack times," *Information Systems Security*, vol. 15, no. 6, pp. 45–55, Dec. 2006, ISSN: 1065-898x. DOI: 10.1080/10658980601051318.

[38] H. Cheng, W. Li, P. Wang, and K. Liang, "Improved probabilistic context-free grammars for passwords using word extraction," in *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 2690–2694. DOI: 10.1109/icassp39728.2021.9414886.

[39] J. Chia and J.-J. Chin, "An Identity Based-Identification Scheme With Tight Security Against Active and Concurrent Adversaries," *IEEE Access*, vol. 8, pp. 61 711–61 725, 2020, Conference Name: IEEE Access, ISSN: 2169-3536. DOI: 10.1109/access.2020.2983750. Accessed: May 21, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/9049156.

[40] J.-J. Chin, S.-H. Heng, and B.-M. Goi, "Hierarchical Identity-Based Identification Schemes," vol. 58, Dec. 2009, pp. 93–99, ISBN: 978-3-642-10846-4. DOI: 10.1007/978-3-642-10847-1_12.

[41] J. C. Choon and J. Hee Cheon, "An Identity-Based Signature from Gap Diffie-Hellman Groups," en, in *Public Key Cryptography – PKC 2003*, Y. G. Desmedt, Ed., Berlin, Heidelberg: Springer, 2002, pp. 18–30, ISBN: 978-3-540-36288-3. DOI: 10.1007/3-540-36288-6_2.

[42] C. Cimpanu, *25 million user records leak online from popular math app Mathway*, en, May 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/25-million-user-records-leak-online-from-popular-math-app-mathway/.

[43] C. Cimpanu, *CouchSurfing investigates data breach after 17m user records appear on hacking forum*, en, Jul. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/couchsurfing-investigates-data-breach-after-17m-user-records-appear-on-hacking-forum/.

[44] C. Cimpanu, *Hacker leaks 40 million user records from popular Wishbone app*, en, News, May 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/hacker-selling-40-million-user-records-from-popular-wishbone-app/.

[45] C. Cimpanu, *Info of 27.7 million Texas drivers exposed in Vertafore data breach*, en, News, Nov. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/info-of-27-7-million-texas-drivers-exposed-in-vertafore-data-breach/.

[46] C. Cimpanu, *Tech unicorn Dave admits to security breach impacting 7.5 million users*, en, News, Jul. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/tech-unicorn-dave-admits-to-security-breach-impacting-7-5-million-users/.

[47] C. Cimpanu, *US Marshals Service exposed prisoner details in security breach*, en, News, May 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/us-marshals-service-exposed-prisoner-details-in-security-breach/.

[48] C. Cimpanu, *Virgin Media exposes data of 900,000 users via unprotected marketing database*, en, News, Mar. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/virgin-media-exposes-data-of-900000-users-via-unprotected-marketing-database/.

[49] Cisa, *Cyber Essentials by Cybersecurity & Infrastructure Security Agency*, Dec. 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://www.cisa.gov/cyber-essentials.

[50] *Client to Authenticator Protocol (CTAP)*, Jan. 2019. Accessed: Jun. 24, 2024. [Online]. Available: https://fidoalliance.org/specs/fido-v2.0-ps-20190130/fido-client-to-authenticator-protocol-v2.0-ps-20190130.html.

[51] S. Coble, *Key Ring App Data Leak Exposes 44 Million Images*, News, Apr. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.infosecurity-magazine.com:443/news/key-ring-app-data-leak-exposes-44m/.

[52] J. E. Cohen, "Cyberspace as/and Space," *Columbia Law Review*, vol. 107, p. 210, 2007. [Online]. Available: https://heinonline.org/HOL/Page?handle=hein.journals/clr107&id=246&div=&collection=.

[53] G. Corfield, *Ticketmaster cops £1.25m ICO fine for 2018 Magecart breach, blames someone else and vows to appeal*, en. Accessed: Jun. 12, 2024. [Online]. Available: https://www.theregister.com/2020/11/13/ticketmaster%5C%5Ffined%5C%5F1%5C%5F25m%5C%5Fmagecart%5C%5Fbreach/.

[54] E. D. Cristofaro, H. Du, J. Freudiger, and G. Norcie, *A Comparative Usability Study of Two-Factor Authentication*, arXiv:1309.5344 [cs], Jan. 2014. DOI: 10.48550/arXiv.1309.5344. Accessed: Dec. 17, 2024. [Online]. Available: http://arxiv.org/abs/1309.5344.

[55] A. Das, J. Bonneau, M. Caesar, N. Borisov, and X. Wang, "The tangled web of password reuse," en, in *Proceedings 2014 Network and Distributed System Security Symposium*, San Diego, CA: Internet Society, 2014, ISBN: 978-1-891562-35-8. DOI: 10.14722/ndss.2014.23357. [Online]. Available: https://www.ndss-symposium.org/ndss2014/programme/tangled-web-password-reuse/.

[56] J. Davis, *Ransomware Attack on Magellan Health Results in Data Exfiltration*, en-US, News, May 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://healthitsecurity.com/news/ransomware-attack-on-magellan-health-results-in-data-exfiltration.

[57] *Dental Care Alliance Data Breach Impacts More Than 1 Million Patients*, en-US, News, Dec. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.hipaajournal.com/dental-care-alliance-data-breach-impacts-more-than-1-million-patients/.

[58] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, *QLoRA: Efficient Finetuning of Quantized LLMs*, en, May 2023. Accessed: Feb. 8, 2024. [Online]. Available: https://arxiv.org/abs/2305.14314v1.

[59] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, *Qlora: Efficient finetuning of quantized llms*, en, https://arxiv.org/pdf/2305.14314.pdf, May 2023. [Online]. Available: https://arxiv.org/abs/2305.14314v1.

[60] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," en, in *Proceedings of the 2019 Conference of the North*, Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. Accessed: Mar. 11, 2024. [Online]. Available: http://aclweb.org/anthology/N19-1423.

[61] A. Drosou, "Activity related biometrics for person authentication," en-US-GB, biometric, Ph.D. dissertation, Apr. 2013. Accessed: Jun. 8, 2024. [Online]. Available: http://spiral.imperial.ac.uk/handle/10044/1/23659.

[62] B. Dupont, A.-M. Côté, J.-I. Boutin, and J. Fernandez, "Darkode: Recruitment Patterns and Transactional Features of "the Most Dangerous Cybercrime Forum in the World"," en, *American Behavioral Scientist*, vol. 61, no. 11, pp. 1219–1243, Oct. 2017, ISSN: 0002-7642. DOI: 10.1177/0002764217734263. Accessed: Dec. 14, 2019. [Online]. Available: https://doi.org/10.1177/0002764217734263.

[63] M. Dürmuth, F. Angelstorf, C. Castelluccia, D. Perito, and A. Chaabane, "OMEN: Faster Password Guessing Using an Ordered Markov Enumerator," en, in *Engineering Secure Software and Systems*, F. Piessens, J. Caballero, and N. Bielova, Eds., ser. Lecture Notes in Computer Science, Cham: Springer International Publishing, 2015, pp. 119–132, ISBN: 978-3-319-15618-7. DOI: 10.1007/978-3-319-15618-7_10.

[64] L. Dykcik, L. Chuat, P. Szalachowski, and A. Perrig, "BlockPKI: An Automated, Resilient, and Transparent Public-Key Infrastructure," in *2018 IEEE International Conference on Data Mining Workshops (ICDMW)*, Nov. 2018, pp. 105–114. DOI: 10.1109/icdmw.2018.00022.

[65] *Equifax UK - Credit Agency*, https://www.equifax.co.uk. Accessed: Nov. 29, 2019.

[66] S. Es, *Orca-best: A filtered version of orca gpt4 dataset.* https://huggingface.co/datasets/shahules786/orca-best, 2023.

[67] S. Eskandarian et al., "Fidelius: Protecting User Secrets from Compromised Browsers," in *2019 IEEE Symposium on Security and Privacy (SP)*, Issn: 1081-6011, May 2019, pp. 264–280. DOI: 10.1109/sp.2019.00036.

[68] *Experian plc - Credit Agency*, en, https://www.experianplc.com. Accessed: Nov. 29, 2019.

[69] F. M. Farke, L. Lorenz, T. Schnitzler, P. Markert, and M. Dürmuth, "You still use the password after all – Exploring FIDO2 Security Keys in a Small Company," en, 2020, pp. 19–35, ISBN: 978-1-939133-16-8. Accessed: Jun. 24, 2024. [Online]. Available: https://www.usenix.org/conference/soups2020/presentation/farke.

[70] U. Feige, A. Fiat, and A. Shamir, "Zero-knowledge proofs of identity," en, *Journal of Cryptology*, vol. 1, no. 2, pp. 77–94, Jun. 1988, ISSN: 1432-1378. DOI: 10.1007/bf02351717. Accessed: Dec. 1, 2019. [Online]. Available: https://doi.org/10.1007/BF02351717.

[71] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology – CRYPTO' 86*, ser. Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, Aug. 11, 1986, pp. 186–194, ISBN: 978-3-540-18047-0. DOI: 10.1007/3-540-47721-7_12. Accessed: Jul. 31, 2018.

[72] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Proceedings on Advances in cryptology—CRYPTO '86*, Santa Barbara, California, USA: Springer-Verlag, Jan. 1987, pp. 186–194, ISBN: 978-0-387-18047-2. Accessed: Feb. 18, 2020.

[73] FireEye, *Highly Evasive Attacker Leverages SolarWinds Supply Chain to Compromise Multiple Global Victims With SUNBURST Backdoor*, en, Business, Dec. 2020. Accessed: Jan. 4, 2021. [Online]. Available: https://www.fireeye.com/blog/threat-research/2020/12/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backdoor.html.

[74] *Fiverr*, https://www.fiverr.com, 2024.

[75] *Flask*. Accessed: Dec. 19, 2019. [Online]. Available: https://palletsprojects.com/p/flask/.

[76] C. Forler, S. Lucks, and J. Wenzel, "Catena: A Memory-Consuming Password Scrambler," *IACR Cryptol. ePrint Arch.*, 2013, catena. Accessed: May 26, 2024. [Online]. Available:

https://www.semanticscholar.org/paper/Catena%3A-A-Memory-Consuming-Password-Scrambler-Forler-Lucks/d02ecbe31041d38be4febad484994a2c04bd9014.

[77] C. Forler, S. Lucks, and J. Wenzel, "Memory-demanding password scrambling," in *Advances in Cryptology – ASIACRYPT 2014*, P. Sarkar and T. Iwata, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 289–305, ISBN: 978-3-662-45608-8.

[78] *Fragomen, a law firm used by Google, confirms data breach*, en-US, News, Oct. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://social.techcrunch.com/2020/10/26/fragomen-data-breach-google-employees/.

[79] L. Franceschi-Bicchierai, *The SIM Hijackers*, en, https://www.vice.com/en_us/article/vbqax3/hackers-sim-swapping-steal-phone-numbers-instagram-bitcoin, Jul. 2018. Accessed: Nov. 29, 2019.

[80] Freepik, *Statement on Security Incident at Freepik Company*, en-US, Business, Section: Freepik Company, Aug. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.freepik.com/blog/statement-on-security-incident-at-freepik-company/.

[81] C. Fromknecht and D. Velicanu, "CertCoin : A NameCoin Based Decentralized Authentication System 6 . 857 Class Project," 2014.

[82] C. Fromknecht, D. Velicanu, and S. Yakoubov, "A Decentralized Public Key Infrastructure with Identity Retention," *IACR Cryptology ePrint Archive*, vol. 2014, p. 803, 2014.

[83] T. Furdui, "502 Senior IT Professionals Survey in the UK - REDSEAL B2B Research," en, Atomik Research on behalf of RedSeal, Uk, Survey, Jun. 2019, p. 5. Accessed: Jan. 9, 2021. [Online]. Available: https://www.redseal.net/files/PDFs/RedSeal%20UK%20B2B%20Research%20SUMMARY%5C%5FJuly2019.pdf.

[84] S. Furnell, "Assessing password guidance and enforcement on leading websites," *Computer Fraud & Security*, vol. 2011, no. 12, pp. 10–18, Dec. 2011, real_world_eval, ISSN: 1361-3723. DOI: 10.1016/s1361-3723(11)70123-3. Accessed: May 27, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1361372311701233.

[85] E. Gerlitz, M. Häring, and M. Smith, "Please do not use !?_ or your License Plate Number: Analyzing Password Policies in German Companies," en, 2021, pp. 17–36, ISBN: 978-1-939133-25-0. Accessed: Feb. 10, 2024. [Online]. Available: https://www.usenix.org/conference/soups2021/presentation/gerlitz.

[86] S. Ghorbani Lyastani, M. Schilling, M. Neumayr, M. Backes, and S. Bugiel, "Is FIDO2 the Kingslayer of User Authentication? A Comparative Usability Study of FIDO2 Passwordless Authentication," in *2020 IEEE Symposium on Security and Privacy (SP)*, Issn: 2375-1207, May 2020, pp. 268–285. DOI: 10.1109/sp40000.2020.00047. Accessed: Jun. 24, 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9152694.

[87] W. Gibson, *Neuromancer*, eng. New York: Ace Books, 1984, ISBN: 0-441-56959-5.

[88] M. Girault, "An identity-based identification scheme based on discrete logarithms modulo a composite number," en, in *Advances in Cryptology – EUROCRYPT '90*, I. B. Damgård, Ed., Berlin, Heidelberg: Springer, 1991, pp. 481–486, ISBN: 978-3-540-46877-6. DOI: 10.1007/3-540-46877-3_44.

[89] O. Goldreich, S. Goldwasser, and S. Micali, "How to Construct Random Functions," *J. Acm*, vol. 33, no. 4, pp. 792–807, Aug. 1986, ISSN: 0004-5411. DOI: 10.1145/6490.6503. Accessed: Nov. 27, 2019. [Online]. Available: http://doi.acm.org/10.1145/6490.6503.

[90] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof-systems," in *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, ser. Stoc '85, New York, NY, USA: Acm, 1985, pp. 291–304, ISBN: 978-0-89791-151-1. DOI: 10.1145/22145.22178. Accessed: Aug. 19, 2018. [Online]. Available: http://doi.acm.org/10.1145/22145.22178.

[91] D. Goltzsche, C. Wulf, D. Muthukumaran, K. Rieck, P. Pietzuch, and R. Kapitza, "TrustJS: Trusted Client-side Execution of JavaScript," in *Proceedings of the 10th European Workshop on Systems Security*, ser. EuroSec'17, event-place: Belgrade, Serbia, New York, NY, USA: Acm, 2017, 7:1–7:6, ISBN: 978-1-4503-4935-2. DOI: 10.1145/3065913.3

065917. Accessed: Dec. 18, 2019. [Online]. Available: http://doi.acm.org/10.1145/3065913.3065917.

[92] P. Grassi, M. Garcia, and J. Fenton, *Digital identity guidelines*, en, Mar. 2020. DOI: 10.6028/nist.sp.800-63-3. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/63/3/upd2/final.

[93] L. K. Grover, *A fast quantum mechanical algorithm for database search*, arXiv:quant-ph/9605043, Nov. 1996. DOI: 10.48550/arXiv.quant-ph/9605043. Accessed: Dec. 19, 2024. [Online]. Available: http://arxiv.org/abs/quant-ph/9605043.

[94] L. C. Guillou and J.-J. Quisquater, "A Practical Zero-knowledge Protocol Fitted to Security Microprocessor Minimizing Both Transmission and Memory," in *Lecture Notes in Computer Science on Advances in Cryptology-EUROCRYPT'88*, event-place: Davos, Switzerland, New York, NY, USA: Springer-Verlag New York, Inc., 1988, pp. 123–128, ISBN: 978-0-387-50251-9. Accessed: Dec. 1, 2019. [Online]. Available: http://dl.acm.org/citation.cfm?id=55554.55565.

[95] *Hackers steal 8.3 million user records from 123RF*, en, News, Nov. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.itpro.com/security/357770/hackers-steal-83m-user-records-from-123rf.

[96] W. He, D. Akhawe, S. Jain, E. Shi, and D. Song, "ShadowCrypt: Encrypted Web Applications for Everyone," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, ser. Ccs '14, event-place: Scottsdale, Arizona, USA, New York, NY, USA: Acm, 2014, pp. 1028–1039, ISBN: 978-1-4503-2957-6. DOI: 10.1145/2660267.2660326. Accessed: Dec. 18, 2019. [Online]. Available: http://doi.acm.org/10.1145/2660267.2660326.

[97] B. Hitaj, P. Gasti, G. Ateniese, and F. Perez-Cruz, "PassGAN: A Deep Learning Approach for Password Guessing," en, in *Applied Cryptography and Network Security*, vol. 11464, Springer, 2019, pp. 217–237, ISBN: 978-3-030-21568-2. DOI: 10.1007/978-3-030-21568-2_11. [Online]. Available: https://arxiv.org/abs/1709.00440.

[98] "How much is your personal data worth?" Accessed: Aug. 15, 2018.

[99] *How to Protect Your Phone Against a SIM Swap Attack. wired*, en, https://www.wired.com/story/sim-swap-attack-defend-phone. Accessed: Nov. 29, 2019.

[100] E. J. Hu et al., *Lora: Low-rank adaptation of large language models*, en, https://arxiv.org/pdf/2106.09685.pdf, Jun. 2021. [Online]. Available: https://arxiv.org/abs/2106.09685v2.

[101] T. Hunt, *Have i been pwned? – check if your email address has been exposed in a data breach*, Accessed: 2025-09-26, Sep. 2025. [Online]. Available: https://haveibeenpwned.com.

[102] Ibm, *IBM-2023 Cost of a Data Breach Report*, en-us, 2023. Accessed: May 13, 2024. [Online]. Available: https://www.ibm.com/reports/data-breach.

[103] IBM Security, "IBM-2020 Cost of a Data Breach Report," en, 2020.

[104] *IBM X-Force Security Predictions for 2020*, en, Dec. 2019. Accessed: Dec. 19, 2019. [Online]. Available: https://securityintelligence.com/posts/ibm-x-force-security-predictions-for-2020/.

[105] Ico, *Historical Cyber Incident Reports by the UK's ICO*, en, Publisher: ICO, Sep. 2020. Accessed: Jan. 9, 2021. [Online]. Available: https://ico.org.uk/action-weve-taken/data-security-incident-trends/previous-reports/.

[106] M. J. Schwartz, *RiskIQ: British Airways Breach Ties to Cybercrime Group*, en, Blog. Accessed: Jan. 12, 2025. [Online]. Available: https://www.bankinfosecurity.com/riskiq-british-airways-breach-ties-to-cybercrime-group-a-11481.

[107] A. Q. Jiang et al., "**Mistral 7B**," no. arXiv:2310.06825, Oct. 2023, https://arxiv.org/pdf/2310.06825.pdf. DOI: 10.48550/arXiv.2310.06825. [Online]. Available: http://arxiv.org/abs/2310.06825.

[108] J. Jiang, A. Zhou, L. Liu, and L. Zhang, "OMECDN: A Password-Generation Model Based on an Ordered Markov Enumerator and Critic Discriminant Network," en, *Applied Sciences*, vol. 12, no. 23, p. 12 379, Jan. 2022, Number: 23 Publisher: Multidisciplinary Digital Publishing Institute, ISSN: 2076-3417. DOI: 10.3390/app122312379. Accessed: Oct. 30, 2023. [Online]. Available: https://www.mdpi.com/2076-3417/12/23/12379.

[109] B. Johnson and S. Francisco, "Hotmail password breach blamed on phishing attack," en-GB, *The Guardian*, Oct. 2009, ISSN: 0261-3077. Accessed: Jan. 9, 2024. [Online]. Available: https://www.theguardian.com/technology/2009/oct/06/hotmail-phishing.

[110] H. Journal, *PHI of Almost 140,000 Individuals Potentially Compromised in Imperium Health Phishing Attack*, en-US, Sep. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.hipaajournal.com/phi-of-almost-140000-individuals-potentially-compromised-in-imperium-health-phishing-attack.

[111] B. Kaliski, "PKCS #5: Password-Based Cryptography Specification Version 2.0," Internet Engineering Task Force, Request for Comments Rfc 2898, Sep. 2000, Pkdf2. Accessed: May 24, 2024. [Online]. Available: https://datatracker.ietf.org/doc/rfc2898.

[112] M. Kan, *To Hijack the SEC's Twitter Account, a Hacker Impersonated the FBI*, en-gb, Section: Security, Oct. 2024. Accessed: Dec. 21, 2024. [Online]. Available: https://uk.pcmag.com/security/154936/to-hijack-the-secs-twitter-account-a-hacker-impersonated-the-fbi.

[113] R. Kate, *The logic behind three random words*, en, https://www.ncsc.gov.uk/blog-post/the-logic-behind-three-random-words, Aug. 2021. [Online]. Available: https://www.ncsc.gov.uk/blog-post/the-logic-behind-three-random-words.

[114] L. Kessem, *BackSwap Malware Now Targets Six Banks in Spain*, en, Aug. 2018. Accessed: Dec. 13, 2019. [Online]. Available: https://securityintelligence.com/backswap-malware-now-targets-six-banks-in-spain/.

[115] Y. Klijnsma, *The British Airways Breach: How Magecart Claimed 380,000 Victims*, Sep. 2018. Accessed: Dec. 16, 2019. [Online]. Available: https://www.riskiq.com/blog/labs/magecart-british-airways-breach/.

[116] H. Kobayashi, B. L. Mark, and W. Turin, *Probability, Random Processes, and Statistical Analysis: Applications to Communications, Signal Processing, Queueing Theory and Mathematical Finance*. Cambridge: Cambridge University Press, 2011, ISBN: 978-0-521-89544-6. DOI: 10.1017/cbo9780511977770. [Online]. Available: https://www.cambri

dge.org/core/books/probability-random-processes-and-statistical-analysis/1909C657E4758038B54C4235B3AD0FDF.

[117] *KoreLogic - Home*. Accessed: Feb. 13, 2024. [Online]. Available: https://korelogic.com.

[118] E. Kovacs, *Gold Dealer JM Bullion Discloses Months-Long Payment Card Breach — SecurityWeek.Com*, News, Nov. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.securityweek.com/gold-dealer-jm-bullion-discloses-months-long-payment-card-breach.

[119] E. Kovacs, *Hypr Raises $25 Million for Passwordless Authentication Platform*, en-US, Dec. 2022. Accessed: Jun. 4, 2024. [Online]. Available: https://www.securityweek.com/hypr-raises-25-million-passwordless-authentication-platform/.

[120] B. Krebs, *'BlueLeaks' Exposes Files from Hundreds of Police Departments – Krebs on Security*, en-US, Blog, Jun. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://krebsonsecurity.com/2020/06/blueleaks-exposes-files-from-hundreds-of-police-departments/.

[121] B. Krebs, *Breach at Dickey's BBQ Smokes 3M Cards – Krebs on Security*, en-US, Blog, Oct. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://krebsonsecurity.com/2020/10/breach-at-dickeys-bbq-smokes-3m-cards/.

[122] B. Krebs, *Europe's Largest Private Hospital Operator Fresenius Hit by Ransomware – Krebs on Security*, en-US, Blog, May 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://krebsonsecurity.com/2020/05/europes-largest-private-hospital-operator-fresenius-hit-by-ransomware/.

[123] B. Krebs, *Arrests in $400M SIM-Swap Tied to Heist at FTX? – Krebs on Security*, en-US, Feb. 2024. Accessed: Jun. 6, 2024. [Online]. Available: https://krebsonsecurity.com/2024/02/arrests-in-400m-sim-swap-tied-to-heist-at-ftx/.

[124] M. Y. Kubilay, M. S. Kiraz, and H. A. Mantar, "CertLedger: A new PKI model with Certificate Transparency based on blockchain," *Computers & Security*, vol. 85, pp. 333–352, Aug. 2019, ISSN: 0167-4048. DOI: 10.1016/j.cose.2019.05.013. Accessed: Nov. 25,

2019. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0167404818313014.

[125] K. Kurosawa and S.-H. Heng, "Identity-Based Identification Without Random Oracles," en, in *Computational Science and Its Applications – ICCSA 2005*, O. Gervasi et al., Eds., Berlin, Heidelberg: Springer, 2005, pp. 603–613, ISBN: 978-3-540-32044-9. DOI: 10.1007/11424826_64.

[126] L. F. de La Torre, *The Hanna 2019 Breach: Consumers get $38.00 for a breach of the "fullz." — by Lydia F de la Torre — Golden Data — Dec, 2020 — Medium*, Blog, Dec. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://medium.com/golden-data/the-hanna-andersson-2019-breach-96f5c9415bf2.

[127] L. Leal, *Skimmer Loaded Via Image On MemberPress Checkout Form & Magento :: Luke's Research*, en, Dec. 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://lukeleal.com/research/posts/0x165a3f-skimmer/.

[128] A. N. Lee, C. J. Hunter, and N. Ruiz, "Platypus: Quick, cheap, and powerful refinement of llms," 2023.

[129] K. Lee, S. Sjöberg, and A. Narayanan, "Password policies of most top websites fail to follow best practices," in *Proceedings of the Eighteenth USENIX Conference on Usable Privacy and Security*, ser. Soups'22, Usa: USENIX Association, Aug. 2022, pp. 561–580, ISBN: 978-1-939133-30-4. Accessed: Feb. 11, 2024.

[130] Y. Li, H. Wang, and K. Sun, "A study of personal information in human-chosen passwords and its security implications," in *IEEE INFOCOM 2016 - The 35th Annual IEEE International Conference on Computer Communications*, Apr. 2016, pp. 1–9. DOI: 10.1109/infocom.2016.7524583. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7524583.

[131] Z. Li, W. Han, and W. Xu, "A large-scale empirical analysis of chinese web passwords," in *USENIX Security Symposium*, 2014. [Online]. Available: https://api.semanticscholar.org/CorpusID:14461062.

[132] S. Liao, *Nintendo said that a total of 300,000 accounts have been hacked - CNN*, News, Jun. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://edition.cnn.com/2020/06/09/tech/nintendo-300000-accounts-hacked/index.html.

[133] K. Lim, J. H. Kang, M. Dixson, H. Koo, and D. Kim, "Evaluating Password Composition Policy and Password Meters of Popular Websites," in *2023 IEEE Security and Privacy Workshops (SPW)*, Issn: 2770-8411, May 2023, pp. 12–20. DOI: 10.1109/spw59333.2023.00006. Accessed: Feb. 11, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10188654.

[134] Y. Liu, "The importance of human-labeled data in the era of llms," no. arXiv:2306.14910, Jun. 2023, arXiv:2306.14910 [cs]. DOI: 10.48550/arXiv.2306.14910. [Online]. Available: http://arxiv.org/abs/2306.14910.

[135] *Llms can label data as well as humans, but 100x faster*, en, url={https://refuel.ai/blog-posts/llm-labeling-technical-report}, 2024. [Online]. Available: https://refuel.ai/blog-posts/llm-labeling-technical-report.

[136] J. Lomchan, R. Wiangsripanawan, and S. F. Shahandashti, "The Comparison of Password Composition Policies Among US, German, and Thailand Samples," in *2023 20th International Joint Conference on Computer Science and Software Engineering (JCSSE)*, real_world_eval, Jun. 2023, pp. 213–218. DOI: 10.1109/jcsse58229.2023.10202141. Accessed: May 27, 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10202141.

[137] Lynx0x00, *I hacked SlickWraps. This is how. - Lynx0x00 - Medium*, Blog, Feb. 2020. Accessed: Jan. 7, 2021. [Online]. Available: http://archive.li/yEIJT.

[138] J. Ma, W. Yang, M. Luo, and N. Li, "A Study of Probabilistic Password Models," in *2014 IEEE Symposium on Security and Privacy*, May 2014, pp. 689–704. DOI: 10.1109/sp.2014.50. [Online]. Available: https://ieeexplore.ieee.org/document/6956595.

[139] D. Malone and K. Maher, "Investigating the distribution of password choices," in *Proceedings of the 21st international conference on World Wide Web*, ser. Www '12, New York, NY, USA: Association for Computing Machinery, Apr. 2012, pp. 301–310, ISBN:

978-1-4503-1229-5. DOI: 10.1145/2187836.2187878. [Online]. Available: https://doi.org/10.1145/2187836.2187878.

[140] P. B. Maoneke and S. Flowerday, "Password Policies Adopted by South African Organizations: Influential Factors and Weaknesses," en, in *Information Security*, H. Venter, M. Loock, M. Coetzee, M. Eloff, and J. Eloff, Eds., real_world_eval, Cham: Springer International Publishing, 2019, pp. 30–43, ISBN: 978-3-030-11407-7. DOI: 10.1007/978-3-030-11407-7_3.

[141] O. Mares, *Amtrak suffers data breach; passenger information leaked*, en-US, News, Jun. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.securitynewspaper.com/2020/06/02/amtrak-suffers-data-breach-passenger-information-leaked/.

[142] O. Mares, *Millions of images and videos of babies sold on deep web*, en-US, Jul. 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://www.securitynewspaper.com/2020/01/14/millions-of-images-and-videos-of-babies-sold-on-deep-web-data-gap-in-the-peekaboo-moments-app/.

[143] B. Martin. "Woocommerce skimmer uses fake fonts and favicon to steal cc details," Sucuri Blog, Accessed: Feb. 21, 2025. [Online]. Available: https://blog.sucuri.net/2022/02/woocommerce-skimmer-uses-fake-fonts-and-favicon-to-steal-cc-details.html.

[144] Mashable, *Notice of data security incident*, en, Business, Nov. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://mashable.com/article/mashable-data-security-issue/.

[145] I. McCormack, *Three random words or #thinkrandom*, en. [Online]. Available: https://www.ncsc.gov.uk/blog-post/three-random-words-or-thinkrandom-0.

[146] J. M. McCune, A. Perrig, and M. K. Reiter, "Safe Passage for Passwords and Other Sensitive Data," in *Ndss*, 2009.

[147] *Meet passkeys - WWDC22 - Videos*, en, 2022. Accessed: Jun. 24, 2024. [Online]. Available: https://developer.apple.com/videos/play/wwdc2022/10092/.

[148] W. Melicher et al., "Fast, lean, and accurate: Modeling password guessability using neural networks," en, 2016, pp. 175–191, ISBN: 978-1-931971-32-4. [Online]. Available: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/melicher.

[149] W. Melicher et al., "Fast, lean, and accurate: Modeling password guessability using neural networks," in *25th USENIX Security Symposium (USENIX Security 16)*, Austin, TX: USENIX Association, Aug. 2016, pp. 175–191, ISBN: 978-1-931971-32-4. [Online]. Available: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/melicher.

[150] A. J. Menezes, P. C. Van Oorschot, and S. A. Vanstone, *Handbook of applied cryptography* (CRC Press series on discrete mathematics and its applications). Boca Raton: CRC Press, 1997, 780 pp., ISBN: 978-0-8493-8523-0.

[151] R. C. Merkle, "A digital signature based on a conventional encryption function," *Advances in Cryptology — CRYPTO '87*, pp. 369–378, 1988. DOI: 10.1007/3-540-48184-2_32.

[152] "MGM hack exposes personal data of 10.6 million guests," en-GB, *BBC News*, Feb. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.bbc.com/news/technology-51568885.

[153] D. Miessler, *Hotmail 2009 Dataset SecLists*, 2009. Accessed: Apr. 30, 2024. [Online]. Available: https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/hotmail.txt.

[154] D. Miessler, *Phpbb 2009 Dataset SecLists*, en, 2009. Accessed: Mar. 30, 2024. [Online]. Available: https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/phpbb.txt.

[155] D. Miessler, *Rockyou 2009 Dataset SecLists*, en, 2009. Accessed: Apr. 30, 2024. [Online]. Available: https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/rockyou.txt.tar.gz.

[156] D. Miessler, *000Webhost Dataset SecLists*, en, 2015. Accessed: Apr. 30, 2024. [Online]. Available: https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/000webhost.txt.

[157] D. Miessler, *LizardSquad Dataset SecLists*, en, 2015. Accessed: Apr. 30, 2024. [Online]. Available: https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/Lizard-Squad.txt.

[158] D. Miessler, *Fortinet Dataset SecLists*, 2021. Accessed: Apr. 30, 2024. [Online]. Available: https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/fortinet-2021%5C%5Fpasswords.txt.

[159] D. Miessler, *Danielmiessler/SecLists*, original-date: 2012-02-19T01:30:18Z, Apr. 2024. Accessed: Apr. 30, 2024. [Online]. Available: https://github.com/danielmiessler/SecLists.

[160] D. Milmo, "UK engineering firm Arup falls victim to £20m deepfake scam," en-GB, *The Guardian*, May 2024, ISSN: 0261-3077. Accessed: Dec. 21, 2024. [Online]. Available: https://www.theguardian.com/technology/article/2024/may/17/uk-engineering-arup-deepfake-scam-hong-kong-ai-video.

[161] A. G. Møller, J. A. Dalsgaard, A. Pera, and L. M. Aiello, "The parrot dilemma: Human-labeled vs. llm-augmented data in classification tasks," no. arXiv:2304.13861, Feb. 2024, http://arxiv.org/abs/2304.13861. DOI: 10.48550/arXiv.2304.13861. [Online]. Available: http://arxiv.org/abs/2304.13861.

[162] E. Montalbano, *Leak Exposes Private Data of Genealogy Service Users*, en, news, Jul. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://threatpost.com/leak-exposes-private-data-of-genealogy-service-users/157612/.

[163] M. Moore and 2020, *Thousands of Spotify accounts hacked - here's what you need to know*, en, News, Nov. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.techradar.com/news/thousands-of-spotify-accounts-hacked-heres-what-you-need-to-know.

[164] R. C. Moshailov Roy, *Gozi Adds Evasion Techniques to its Growing Bag of Tricks*, en, Jan. 2019. Accessed: Dec. 27, 2019. [Online]. Available: https://www.f5.com/labs/articles/threat-intelligence/gozi-adds-evasion-techniques-to-its-growing-bag-of-tricks.html.

[165] S. Mukherjee, A. Mitra, G. Jawahar, S. Agarwal, H. Palangi, and A. Awadallah, *Orca: Progressive learning from complex explanation traces of gpt-4*, 2023. arXiv: 2306.02707.

[166] P. Muncaster, *Cosmetics Giant Avon Leaks 19 Million Records*, Jul. 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://www.infosecurity-magazine.com:443/news/cosmetics-giant-avon-leaks-19/.

[167] P. Muncaster, *PhotoSquared: App Leaks Data on Thousands of Users*, News, Feb. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.infosecurity-magazine.com:443/news/photosquared-app-leaks-data/.

[168] T. Nakamura, "Wearable in-ear electroencephalography for real world applications: Sleep monitoring and person authentication," en-US-GB, biometric, Ph.D. dissertation, May 2019. Accessed: Jun. 8, 2024. [Online]. Available: http://spiral.imperial.ac.uk/handle/10044/1/73907.

[169] S. Nam, S. Jeon, and J. Moon, "Generating optimized guessing candidates toward better password cracking from multi-dictionaries using relativistic gan," *Applied Sciences*, vol. 10, p. 7306, Oct. 2020. DOI: 10.3390/app10207306.

[170] *Namecoin*, https://www.namecoin.org. Accessed: Nov. 30, 2019.

[171] A. Narayanan and V. Shmatikov, "Fast Dictionary Attacks On Passwords Using Time-Space Tradeoff," in *Proceedings of the 12th ACM conference on Computer and communications security*, ser. CCS '05, New York, NY, USA: Association for Computing Machinery, Nov. 2005, pp. 364–372, ISBN: 978-1-59593-226-6. DOI: 10.1145/1102120.1102168. [Online]. Available: https://dl.acm.org/doi/10.1145/1102120.1102168.

[172] *National cyber security centre*, https://www.ncsc.gov.uk, 2024. [Online]. Available: https://www.ncsc.gov.uk.

[173] *National institute of standards and technology*, en, https://www.nist.gov, text, Last Modified: 2024-02-07T09:49-05:00, Feb. 2024. [Online]. Available: https://www.nist.gov.

[174] Ncsc, *Password policy: Updating your approach*, en, https://www.ncsc.gov.uk/collection/passwords/updating-your-approach, 2018. [Online]. Available: https://www.ncsc.gov.uk/collection/passwords/updating-your-approach.

[175] Ncsc, "Next steps in preparing for post-quantum cryptography," en, Ncsc, Tech. Rep. Accessed: Dec. 18, 2024. [Online]. Available: https://www.ncsc.gov.uk/whitepaper/next-steps-preparing-for-post-quantum-cryptography.

[176] *Nearly 4 Million Quidd User Credentials Stolen and Shared on Hacking Forum*, en-US, Section: Data Breaches, Apr. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.riskbasedsecurity.com/2020/04/10/nearly-4-million-quidd-user-credentials-stolen-and-shared-on-hacking-forum/.

[177] S. Neves, "Yescrypt - a Password Hashing Competition submission," yescrypt, 2015. Accessed: May 26, 2024. [Online]. Available: https://www.semanticscholar.org/paper/yescrypt-a-Password-Hashing-Competition-submission-Neves/51bae20aac18ac420527349a14e7e046ebf1be00.

[178] M. Oi and N. Sherman, "Netflix: Profits soar after password sharing crackdown," en-GB, *BBC News*, Apr. 2024. Accessed: May 20, 2024. [Online]. Available: https://www.bbc.com/news/business-68850766.

[179] T. Okamoto, "Provably Secure and Practical Identification Schemes and Corresponding Signature Schemes," en, in *Advances in Cryptology – CRYPTO' 92*, E. F. Brickell, Ed., Berlin, Heidelberg: Springer, 1993, pp. 31–53, ISBN: 978-3-540-48071-6. DOI: 10.1007/3-540-48071-4_3.

[180] H. Ong and C. P. Schnorr, "Fast Signature Generation with a Fiat Shamir – Like Scheme," en, in *Advances in Cryptology – EUROCRYPT '90*, I. B. Damgård, Ed., Berlin, Heidelberg: Springer, 1991, pp. 432–440, ISBN: 978-3-540-46877-6. DOI: 10.1007/3-540-46877-3_38.

[181] OpenAI, *Gpt-3.5 turbo fine-tuning and api updates*, en-US, https://openai.com/blog/gpt-3-5-turbo-fine-tuning-and-api-updates. [Online]. Available: https://openai.com/blog/gpt-3-5-turbo-fine-tuning-and-api-updates.

[182] G. Orwell and E. Fromm, *1984* (Signet classics), eng, Centennial ed.; [Nachdr. der Ausg.] 1. Signet Classics printing, July 1950, 125. [Nachdr.], New American Library, a division of Penguin Group (USA), New York, NY, 1977. St. Louis, Mo.: Turtleback Books, 2000, ISBN: 978-0-88103-036-5.

[183] C. Osborne, *Amtrak discloses data breach, potential leak of customer account data*, en, News, Jun. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/amtrak-discloses-data-breach-potential-leak-of-sensitive-customer-information/.

[184] C. Osborne, *Boom! Mobile falls prey to Magecart card-skimming attack*, en, News, Oct. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/boom-mobile-falls-prey-to-magecart-card-skimming-attack/.

[185] C. Osborne, *Health Share of Oregon discloses data breach, theft of member PII*, en, News, Feb. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/health-share-of-oregon-discloses-data-breach-theft-of-member-pii/.

[186] C. Osborne, *US actor casting company leaked private data of over 260,000 individuals*, en, News, Jul. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/us-actor-casting-company-leaked-private-data-of-over-260000-individuals/.

[187] C. Osborne, *Whisper, an anonymous secret-sharing app, failed to keep messages or profiles private*, en, News, Mar. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.zdnet.com/article/whisper-an-anonymous-secret-sharing-app-failed-to-keep-messages-profiles-private/.

[188] Owasp, *OWASP Password Composition Policy Recommendation*, English, pcp_owasp, 2024. Accessed: May 27, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication%5C%5FCheat%5C%5FSheet.html.

[189] Owasp, *Password Storage - OWASP Cheat Sheet Series*, 2024. Accessed: May 26, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password%5C%5FStorage%5C%5FCheat%5C%5FSheet.html.

[190] P. Paganini, *440M records found online in unprotected database belonging to Estée Lauder*, en-US, Feb. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://securityaffairs.co/wordpress/97675/data-breach/estee-lauder-data-leak.html.

[191] B. Pal, T. Daniel, R. Chatterjee, and T. Ristenpart, "Beyond Credential Stuffing: Password Similarity Models Using Neural Networks," in *2019 IEEE Symposium on Security and Privacy (SP)*, San Francisco, CA, USA: Ieee, May 2019, pp. 417–434, ISBN: 978-1-5386-6660-9. DOI: 10.1109/sp.2019.00056. Accessed: Jan. 3, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/8835247/.

[192] N. Pangakis, S. Wolken, and N. Fasching, "Automated annotation with generative ai requires validation," no. arXiv:2306.00176, May 2023, arXiv:2306.00176 [cs]. DOI: 10.48550/arXiv.2306.00176. [Online]. Available: http://arxiv.org/abs/2306.00176.

[193] D. Pasquini, "Enabling secure passwords via Deep Learning: Towards a new generation of attacks and defenses," password, Ph.D. dissertation, Sapienza University of Rome, 2021. Accessed: Jun. 8, 2024. [Online]. Available: https://iris.uniroma1.it/handle/11573/1561476.

[194] D. Pasquini, M. Cianfriglia, G. Ateniese, and M. Bernaschi, "Reducing bias in modeling real-world password strength via deep learning and dynamic dictionaries," en, https://www.usenix.org/conference/usenixsecurity21/presentation/pasquini, 2021, pp. 821–838, ISBN: 978-1-939133-24-3. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/pasquini.

[195] D. Pasquini, M. Cianfriglia, G. Ateniese, and M. Bernaschi, "Reducing bias in modeling real-world password strength via deep learning and dynamic dictionaries," in *30th USENIX Security Symposium (USENIX Security 21)*, USENIX Association, Aug. 2021, pp. 821–838, ISBN: 978-1-939133-24-3. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/pasquini.

[196] D. Pasquini, A. Gangwal, G. Ateniese, M. Bernaschi, and M. Conti, "Improving Password Guessing via Representation Learning," in *2021 IEEE Symposium on Security and Privacy (SP)*, Issn: 2375-1207, May 2021, pp. 1382–1399. DOI: 10.1109/sp40001.2021.00016.

[197] *Password policy: Updating your approach*, en. Accessed: Feb. 2, 2024. [Online]. Available: https://www.ncsc.gov.uk/collection/passwords/updating-your-approach.

[198] C. Patsonakis, K. Samari, A. Kiayias, and M. Roussopoulos, "On the Practicality of Smart Contract PKI," *2019 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPCON)*, pp. 109–118, Apr. 2019, arXiv: 1902.00878. DOI: 10.1109/dappcon.2019.00022. Accessed: Nov. 25, 2019. [Online]. Available: http://arxiv.org/abs/1902.00878.

[199] S. Pearman et al., "Let's go in for a closer look: Observing passwords in their natural habitat," en, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp. 295–310, Oct. 2017, https://dl.acm.org/doi/10.1145/3133956.3133973. DOI: 10.1145/3133956.3133973. [Online]. Available: https://dl.acm.org/doi/10.1145/3133956.3133973.

[200] C. Percival, "Stronger Key Derivation Via Sequential Memory-hard Functions," scrypt, 2009. Accessed: May 26, 2024. [Online]. Available: https://www.semanticscholar.org/paper/STRONGER-KEY-DERIVATION-VIA-SEQUENTIAL-MEMORY-HARD-Percival/7c74956d21f0466c9771bb583e2fdf854c2aedbf.

[201] "Pfizer/BioNTech vaccine docs hacked from European Medicines Agency," en-GB, *BBC News*, Dec. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.bbc.com/news/technology-55249353.

[202] T. Pornin, "The MAKWA Password Hashing Function," en, 2015, makwa.

[203] S. Preibusch and J. Bonneau, "The password game: Negative externalities from weak password practices," en, in *Decision and Game Theory for Security*, T. Alpcan, L. Buttyán, and J. S. Baras, Eds., ser. Lecture Notes in Computer Science, Berlin, Heidelberg: Springer, 2010, pp. 192–207, ISBN: 978-3-642-17197-0. DOI: 10.1007/978-3-642-17197-0_13.

[204] *Privly*. Accessed: Dec. 18, 2019. [Online]. Available: https://priv.ly.

[205] N. Provos and D. Mazières, "A {Future-Adaptable} Password Scheme," en, bcrypt, 1999. Accessed: May 26, 2024. [Online]. Available: https://www.usenix.org/conference/1999-usenix-annual-technical-conference/future-adaptable-password-scheme.

[206] J. Rando, F. Perez-Cruz, and B. Hitaj, "Passgpt: Password modeling and (guided) generation with large language models," no. arXiv:2306.01545, Jun. 2023, http://arxiv.org/abs/2306.01545. DOI: 10.48550/arXiv.2306.01545. [Online]. Available: http://arxiv.org/abs/2306.01545.

[207] *Report: Popular Gambling App Exposed Millions of Users in Massive Data Leak*, en, Report, Mar. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.vpnmentor.com/blog/report-clubillion-leak/.

[208] B. L. Riddle, M. S. Miron, and J. A. Semo, "Passwords in use in a university timesharing environment," *Computers & Security*, vol. 8, no. 7, pp. 569–579, Nov. 1989, ISSN: 0167-4048. DOI: 10.1016/0167-4048(89)90049-7.

[209] D. Riley, *2.5M credit card records belonging to transaction firm PAAY exposed online*, en-US, Section: NEWS, Apr. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://siliconangle.com/2020/04/22/2-5m-credit-card-records-belonging-transaction-firm-paay-exposed-online/.

[210] C. Roy, M. Remi, and B. Sara, *Gozi Banking Trojan Pivots Towards Italian Banks in February and March*, en, Apr. 2019. Accessed: Dec. 27, 2019. [Online]. Available: https://www.f5.com/labs/articles/threat-intelligence/gozi-banking-trojan-pivots-towards-italian-banks-in-february-and-march.html.

[211] H. Saleem and M. Naveed, "SoK: Anatomy of Data Breaches," en, *Proceedings on Privacy Enhancing Technologies*, vol. 2020, no. 4, pp. 153–174, Oct. 2020, PoPETs, ISSN: 2299-0984. DOI: 10.2478/popets-2020-0067. Accessed: Jun. 8, 2024. [Online]. Available: https://petsymposium.org/popets/2020/popets-2020-0067.php.

[212] *Salesforce's Unified Cloud Commerce Software*, en, Business, Jan. 2021. Accessed: Jan. 11, 2021. [Online]. Available: https://www.salesforce.com/uk/products/commerce-cloud/overview/.

[213] T. Sandle, *Claire's Magecart hit is a serious cyber attack (Includes interview)*, News, Jun. 2020. Accessed: Jan. 7, 2021. [Online]. Available: http://www.digitaljournal.com/tech-and-science/technology/claire-s-magecart-hit-by-serious-cyber-attack/article/573339.

[214] Sansec, *Magecart strikes amid Corona lockdown*, en, Jun. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://sansec.io/research/magecart-corona-lockdown.

[215] T. Sansec, *Cardbleed: A massive Magento1 hack*, en, Business, Sep. 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://sansec.io/research/cardbleed.

[216] C. P. Schnorr, "Efficient Identification and Signatures for Smart Cards," in *Proceedings of the Workshop on the Theory and Application of Cryptographic Techniques on Advances in Cryptology*, ser. Eurocrypt '89, event-place: Houthalen, Belgium, Berlin, Heidelberg: Springer-Verlag, 1990, pp. 688–689, ISBN: 978-3-540-53433-4. Accessed: Dec. 1, 2019. [Online]. Available: http://dl.acm.org/citation.cfm?id=111563.111628.

[217] T. Seals, *Good Heavens! 10M Impacted in Pray.com Data Exposure*, en, News, Nov. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://threatpost.com/10m-impacted-pray-com-data-exposure/161459/.

[218] J. Segura, *Credit card skimmer masquerades as favicon*, en-US, Section: Threat analysis, May 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://blog.malwarebytes.com/threat-analysis/2020/05/credit-card-skimmer-masquerades-as-favicon/.

[219] J. Segura, *Criminals hack Tupperware website with credit card skimmer — Malwarebytes Labs*, en, Tupperware, Mar. 2020. Accessed: Jun. 12, 2024. [Online]. Available: https://www.malwarebytes.com/blog/news/2020/03/criminals-hack-tupperware-website-with-credit-card-skimmer/.

[220] J. Segura, *Web skimmer hides within EXIF metadata, exfiltrates credit cards via image files*, en-US, Section: Threat analysis, Jun. 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://blog.malwarebytes.com/threat-analysis/2020/06/web-skimmer-hides-within-exif-metadata-exfiltrates-credit-cards-via-image-files/.

[221] A. Shamir, "Identity-based cryptosystems and signature schemes," in *Advances in Cryptology*, ser. Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, Aug. 19, 1984, pp. 47–53, ISBN: 978-3-540-15658-1. DOI: 10.1007/3-540-39568-7_5. Accessed: Jul. 28, 2018. [Online]. Available: https://link.springer.com/chapter/10.1007/3-540-39568-7%5C%5F5.

[222] Y. Shen and G. Stringhini, "{ATTACK2VEC}: Leveraging Temporal Word Embeddings to Understand the Evolution of Cyberattacks," en, Usenix, 2019, pp. 905–921, ISBN: 978-1-939133-06-9. Accessed: Jun. 8, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/shen.

[223] P. W. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, Oct. 1997, arXiv: quant-ph/9508027, ISSN: 0097-5397, 1095-7111. DOI: 10.1137/s0097539795293172. Accessed: Oct. 4, 2019. [Online]. Available: http://arxiv.org/abs/quant-ph/9508027.

[224] D. Sinegubko, *CSS-JS Steganography in Fake Flash Player Update Malware*, en-US, Nov. 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://blog.sucuri.net/2020/11/css-js-steganography-in-fake-flash-player-update-malware.html.

[225] Solar Designer, *John the ripper password cracker*, https://www.openwall.com/john/, Openwall Project, 1996.

[226] A. Spadafora, *General Electric suffers data breach after service provider hack*, en, News, Mar. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.techradar.com/uk/news/general-electric-suffers-data-breach-after-service-provider-hack.

[227] J. Steube, *PRINCE (PRobability INfinite Chained Elements) password processor*, Password candidate generator implementing the PRINCE algorithm, hashcat, 2015. [Online]. Available: https://github.com/hashcat/princeprocessor.

[228] J. Steube, *Hashcat - advanced password recovery*, Jun. 2023. Accessed: Oct. 6, 2023. [Online]. Available: https://hashcat.net.

[229] *Sweaty Betty Custom.js - Contains Malware*, Archive, Nov. 2019. Accessed: Jan. 9, 2021. [Online]. Available: https://web.archive.org/web/20191125205916js%5C%5F/https://www.sweatybetty.com/on/demandware.static/-/Library-Sites-sweatybettylibrary/en%5C%5FUS/v1574703272172/js/custom.js.

[230] C. Szongott, B. Henne, and M. Smith, "Evaluating the threat of epidemic mobile malware," in *2012 IEEE 8th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*, Issn: 2160-4886, Oct. 2012, pp. 443–450. DOI: 10.1109/WiMOB.2012.6379111.

[231] S.-Y. Tan, S.-H. Heng, R. C. .-. Phan, and B.-M. Goi, "A Variant of Schnorr Identity-Based Identification Scheme with Tight Reduction," en, in *Future Generation Information Technology*, T.-h. Kim et al., Eds., Berlin, Heidelberg: Springer, 2011, pp. 361–370, ISBN: 978-3-642-27142-7. DOI: 10.1007/978-3-642-27142-7_42.

[232] J. Taylor, "Twitter is ending free SMS two-factor authentication. So what can you use instead?" en-GB, *The Guardian*, Feb. 2023, ISSN: 0261-3077. Accessed: Dec. 17, 2024. [Online]. Available: https://www.theguardian.com/technology/2023/feb/21/twitter-is-ending-free-sms-two-factor-authentication-so-what-can-you-use-instead.

[233] T. I. Team, *Mobile network operator falls into the hands of Fullz House criminal group*, en-US, Section: Web threats, Oct. 2020. Accessed: Jan. 9, 2021. [Online]. Available: https://blog.malwarebytes.com/web-threats/2020/10/mobile-network-operator-falls-into-the-hands-of-fullz-house-criminal-group/.

[234] *Teen Arrested As Mastermind Of Recent High Profile Twitter Breach*, en-US, News, Aug. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.onlyinfotech.com/2020/08/01/teen-arrested-as-mastermind-of-recent-high-profile-twitter-breach/.

[235] *The Business of Organized Cybercrime: Rising Intergang Collaboration in 2018*, en, Mar. 2019. Accessed: Dec. 19, 2019. [Online]. Available: https://securityintelligence.co

m/the-business-of-organized-cybercrime-rising-intergang-collaboration-in-2018/.

[236] *The SABSA Institute - Enterprise Security Architecture*, en-US, Dec. 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://sabsa.org/.

[237] K. Thomas et al., "Data Breaches, Phishing, or Malware? Understanding the Risks of Stolen Credentials," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. Ccs '17, Ccs, New York, NY, USA: Association for Computing Machinery, Oct. 2017, pp. 1421–1434, ISBN: 978-1-4503-4946-8. DOI: 10.1145/3133956.3134067. Accessed: Jun. 8, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3133956.3134067.

[238] J. Tidy, "British Airways fined £20m over data breach," en-GB, *BBC News*, Oct. 2020. Accessed: Jun. 6, 2024. [Online]. Available: https://www.bbc.com/news/technology-54568784.

[239] I. Tolstikhin, O. Bousquet, S. Gelly, and B. Schoelkopf, *Wasserstein Auto-Encoders*, arXiv:1711.01558 [cs, stat], Dec. 2019. DOI: 10.48550/arXiv.1711.01558. Accessed: Mar. 6, 2024. [Online]. Available: http://arxiv.org/abs/1711.01558.

[240] H. Touvron et al., "**Llama 2: Open Foundation and Fine-Tuned Chat Models**," *ArXiv*, Jul. 2023, https://arxiv.org/pdf/2307.09288.pdf.

[241] K. Towsend, *Hanna Andersson Data Breach: Hackers Compromise Website of Children's Clothier — SecurityWeek.Com*, News, Jan. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.securityweek.com/hanna-andersson-data-breach-hackers-compromise-website-childrens-clothier.

[242] L. Tran, T. Nguyen, C. Seo, H. Kim, and D. Choi, *A survey on password guessing*, 2022. arXiv: 2212.08796. [Online]. Available: https://arxiv.org/abs/2212.08796.

[243] *TransUnion - Credit Scores, Credit Reports & Credit Check*, en-US, https://www.transunion.com. Accessed: Nov. 29, 2019.

[244] *Tufts Health Plan Reports Breach Affecting 60K*, en, News. Accessed: Jan. 7, 2021. [Online]. Available: https://www.healthleadersmedia.com/technology/tufts-health-plan-reports-breach-affecting-60k.

[245] S. Turkle, "Cyberspace and Identity," *Contemporary Sociology*, vol. 28, no. 6, pp. 643–648, 1999, Publisher: [American Sociological Association, Sage Publications, Inc.], ISSN: 0094-3061. DOI: 10.2307/2655534. Accessed: Jun. 14, 2024. [Online]. Available: https://www.jstor.org/stable/2655534.

[246] UK Cabinet Office, *National Cyber Strategy 2022*, en, Policy paper, 2022. Accessed: Jun. 8, 2024. [Online]. Available: https://www.gov.uk/government/publications/national-cyber-strategy-2022/national-cyber-security-strategy-2022.

[247] Uk Gov, *Cyber Essentials Scheme Overview*, en, Jan. 2018. Accessed: Jan. 11, 2021. [Online]. Available: https://www.gov.uk/government/publications/cyber-essentials-scheme-overview.

[248] B. Ur et al., "Measuring real-world accuracies and biases in modeling password guessability," en, https://www.usenix.org/system/files/conference/usenixsecurity15/sec15-paper-ur.pdf, 2015, pp. 463–481, ISBN: 978-1-939133-11-3. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/ur.

[249] B. E. Ur, "Supporting Password-Security Decisions with Data," en, password, thesis, Carnegie Mellon University, Sep. 2016. Accessed: Jun. 8, 2024. [Online]. Available: https://kilthub.cmu.edu/articles/thesis/Supporting%5C%5FPassword-Security%5C%5FDecisions%5C%5Fwith%5C%5FData/6723416/1.

[250] Verizon, *2024 Data Breach Investigations Report*, en, 2024. Accessed: May 13, 2024. [Online]. Available: https://www.verizon.com/business/en-gb/resources/reports/dbir/.

[251] J. Wakefield, "EasyJet admits data of nine million hacked," en-GB, *BBC News*, May 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://www.bbc.com/news/technology-52722626.

[252] D. Wang, H. Cheng, P. Wang, X. Huang, and G. Jian, "Zipf's law in passwords," *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 11, pp. 2776–2791, Nov. 2017, ISSN: 1556-6021. DOI: 10.1109/tifs.2017.2721359.

[253] D. Wang, X. Shan, Q. Dong, Y. Shen, and C. Jia, "No single silver bullet: Measuring the accuracy of password strength meters," en, 2023, pp. 947–964, ISBN: 978-1-939133-37-3. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/wang-ding-silver-bullet.

[254] D. Wang, X. Shan, Q. Dong, Y. Shen, and C. Jia, "No Single Silver Bullet: Measuring the Accuracy of Password Strength Meters," en, 2023, pp. 947–964, ISBN: 978-1-939133-37-3. Accessed: Oct. 24, 2023. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/wang-ding-silver-bullet.

[255] D. Wang, Z. Zhang, P. Wang, J. Yan, and X. Huang, "Targeted Online Password Guessing: An Underestimated Threat," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. Ccs '16, New York, NY, USA: Association for Computing Machinery, Oct. 2016, pp. 1242–1254, ISBN: 978-1-4503-4139-4. DOI: 10.1145/2976749.2978339. Accessed: Sep. 12, 2023. [Online]. Available: https://dl.acm.org/doi/10.1145/2976749.2978339.

[256] D. Wang, Y. Zou, Y.-A. Xiao, S. Ma, and X. Chen, "Pass2Edit: A Multi-Step Generative Model for Guessing Edited Passwords," in *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA: USENIX Association, Aug. 2023, pp. 983–1000, ISBN: 978-1-939133-37-3. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/wang-ding-pass2edit.

[257] D. Wang, Y. Zou, Z. Zhang, and K. Xiu, "Password Guessing Using Random Forest," in *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA: USENIX Association, Aug. 2023, pp. 965–982, ISBN: 978-1-939133-37-3. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/wang-ding-password-guessing.

[258] Waqas, *ShinyHunters hacker leaks 5.22GB worth of Mashable.com database*, en-US, News, Section: Hacking News, Nov. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://www.hackread.com/shinyhunters-hacker-leaks-mashable-database/.

[259] *Web Authentication: An API for accessing Public Key Credentials - Level 2*, 2021. Accessed: Jun. 24, 2024. [Online]. Available: https://www.w3.org/TR/webauthn-2/.

[260] M. Weir, S. Aggarwal, B. d. Medeiros, and B. Glodek, "Password Cracking Using Probabilistic Context-Free Grammars," in *2009 30th IEEE Symposium on Security and Privacy*, https://ieeexplore.ieee.org/document/5207658, May 2009, pp. 391–405. DOI: 10.1109/sp.2009.8.

[261] D. L. Wheeler, "Zxcvbn: Low-Budget password strength estimation," in *25th USENIX Security Symposium (USENIX Security 16)*, Austin, TX: USENIX Association, Aug. 2016, pp. 157–173, ISBN: 978-1-931971-32-4. [Online]. Available: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/wheeler.

[262] Z. Whittaker, *Alcohol delivery service Drizly hit by data breach*, en-US, Jul. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://social.techcrunch.com/2020/07/28/drizly-data-breach/.

[263] Z. Whittaker, *J.Crew says a hacker accessed customer accounts*, en-US, News, Mar. 2020. Accessed: Jan. 7, 2021. [Online]. Available: https://social.techcrunch.com/2020/03/04/jcrew-unauthorized-accounts/.

[264] N. Wiener, *Cybernetics: Or Control and Communication in the Animal and the Machine*, 1st. Cambridge, MA: MIT Press, 1948.

[265] K. Wiggers, *Passwordless tech startup Beyond Identity raises $75 million*, en-US, Dec. 2020. Accessed: Jan. 11, 2021. [Online]. Available: https://venturebeat.com/2020/12/08/passwordless-tech-startup-beyond-identity-raises-75-million/.

[266] K. Wiggers, *Passwordless authentication startup SecureW2 raises $80M from Insight Partners*, en-US, Oct. 2023. Accessed: Jun. 4, 2024. [Online]. Available: https://techcrunch.com/2023/10/18/passwordless-authentication-startup-securew2-raises-80m-from-insight-partners/.

[267] S. S. Woo, K. J. Jung, and B. J. Choi, "Survey on Current Password Composition Policies," eng, *Review of KIISC*, vol. 28, no. 1, pp. 43–47, 2018, real_world_eval, ISSN: 1598-3978. Accessed: May 27, 2024. [Online]. Available: https://koreascience.kr/article/JAKO201809253685093.page.

[268] M. Xu, C. Wang, J. Yu, J. Zhang, K. Zhang, and W. Han, "Chunk-level password guessing: Towards modeling refined password composition representations," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. Ccs '21, New York, NY, USA: Association for Computing Machinery, Nov. 2021, pp. 5–20, ISBN: 978-1-4503-8454-4. DOI: 10.1145/3460120.3484743. [Online]. Available: https://dl.acm.org/doi/10.1145/3460120.3484743.

[269] M. Xu et al., "Improving real-world password guessing attacks via bi-directional transformers," en, 2023, pp. 1001–1018, ISBN: 978-1-939133-37-3. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/xu-ming.

[270] M. Xu et al., "Improving real-world password guessing attacks via bi-directional transformers," in *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA: USENIX Association, Aug. 2023, pp. 1001–1018, ISBN: 978-1-939133-37-3. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/xu-ming.

[271] A. Yakubov, W. Shbair, and R. State, "BlockPGP: A Blockchain-Based Framework for PGP Key Servers," in *2018 Sixth International Symposium on Computing and Networking Workshops (CANDARW)*, Nov. 2018, pp. 316–322. DOI: 10.1109/candarw.2018.00065.

[272] J. Ye, M. Jin, G. Gong, R. Shen, and H. Lu, "PassTCN-PPLL: A Password Guessing Model Based on Probability Label Learning and Temporal Convolutional Neural Network," *Sensors*, vol. 22, no. 17, 2022, ISSN: 1424-8220. DOI: 10.3390/s22176484. [Online]. Available: https://www.mdpi.com/1424-8220/22/17/6484.

[273] F. Yu and M. V. Martin, "GNPassGAN: Improved Generative Adversarial Networks For Trawling Offline Password Guessing," in *2022 IEEE European Symposium on Security*

and Privacy Workshops (EuroS&PW), Issn: 2768-0657, Jun. 2022, pp. 10–18. DOI: 10.1109/EuroSPW55150.2022.00009.

[274] W. Yu et al., "A Systematic Review on Password Guessing Tasks," en, *Entropy*, vol. 25, no. 9, p. 1303, Sep. 2023, Number: 9 Publisher: Multidisciplinary Digital Publishing Institute, ISSN: 1099-4300. DOI: 10.3390/e25091303. Accessed: Feb. 8, 2024. [Online]. Available: https://www.mdpi.com/1099-4300/25/9/1303.

[275] T. Zhou, H.-T. Wu, H. Lu, P. Xu, Y.-M. Cheung, and T.-T.-H. Le, "Password Guessing Based on GAN with Gumbel-Softmax," *Security and Communication Networks*, vol. 2022, Jan. 2022, ISSN: 1939-0114. DOI: 10.1155/2022/5670629. Accessed: Sep. 20, 2023. [Online]. Available: https://doi.org/10.1155/2022/5670629.

[276] Q. Zhuang, Y. Liu, L. Chen, and Z. Ai, "Proof of Reputation: A Reputation-based Consensus Protocol for Blockchain Based Systems," in *Proceedings of the 2019 International Electronics Communication Conference*, ser. Iecc '19, event-place: Okinawa, Japan, New York, NY, USA: Acm, 2019, pp. 131–138, ISBN: 978-1-4503-7177-3. DOI: 10.1145/3343147.3343169. Accessed: Nov. 29, 2019.